

# A Monograph-Reorganized Companion to Tensor-Train Sequential Filtering, Fixed-Branch Gradients, Large-Scale Validation, And An Expanded Algorithm 5(c.2) Derivation

Chak Wong

2026-06-12

## Abstract

This note gives a self-contained mathematical companion to tensor-train sequential filtering in the style of Zhao and Cui [4]. It follows the state-space, tensor-train, squared-density, conditional Knothe–Rosenblatt, and preconditioning constructions in their paper, while expanding the intermediate equations and implementation meanings that are often compressed in the original presentation. The second half develops a fixed branch variant: all ranks, points, domains, shifts, and fitting choices are declared before differentiation, so the resulting analytical derivative is the derivative of a named approximate filtering scalar rather than of the adaptive algorithm itself. The final part gives a mathematical validation protocol for large-scale state-space applications: the core test models follow Zhao and Cui’s numerical section, and the additional tests record accuracy, memory, runtime, stability, and fixed-branch derivative behavior without claiming that these diagnostics prove exact posterior accuracy.

## Contents

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Orientation And Scope</b>                                       | <b>3</b>  |
| <b>2</b> | <b>Running Example And Notation</b>                                | <b>5</b>  |
| 2.1      | Threaded Nonlinear Example Used Throughout The Note . . . . .      | 6         |
| <b>3</b> | <b>Exact Filtering Target And Recursive Bottleneck</b>             | <b>7</b>  |
| 3.1      | State-Space Model And Notation Used In This Note . . . . .         | 8         |
| 3.2      | The Four Marginal Learning Problems . . . . .                      | 9         |
| 3.3      | The Exact Recursive Bottleneck . . . . .                           | 10        |
| <b>4</b> | <b>Tensor-Train Approximation Toolkit</b>                          | <b>11</b> |
| <b>5</b> | <b>Why Moderate Tensor-Train Rank Is A Plausible Hypothesis</b>    | <b>15</b> |
| <b>6</b> | <b>Coordinate Systems, Conditional Maps, And Preconditioning</b>   | <b>18</b> |
| 6.1      | One Observation Step Through All Coordinate Systems . . . . .      | 19        |
| 6.2      | Conditional Densities And KR Maps From Marginal Ratios . . . . .   | 21        |
| 6.3      | Forward Conditional Map And Particle-Filter Correction . . . . .   | 25        |
| 6.4      | Backward Conditional Map, Path Estimation, And Smoothing . . . . . | 26        |
| 6.5      | Preconditioning . . . . .                                          | 28        |
| 6.6      | The Five Densities In The Preconditioned Step . . . . .            | 31        |

|          |                                                                                        |           |
|----------|----------------------------------------------------------------------------------------|-----------|
| <b>7</b> | <b>Squared-TT Filtering Object And Operational Recursion</b>                           | <b>37</b> |
| 7.1      | Why TT Marginalization Is Cheap Once The Representation Exists . . . . .               | 39        |
| 7.2      | Algorithm 1, Fully Annotated . . . . .                                                 | 40        |
| 7.3      | Why Nonnegativity Fails And Why Square-Root TT Repairs It . . . . .                    | 44        |
| 7.4      | Squared-TT Density, Defensive Reference, And Normalizer . . . . .                      | 44        |
| 7.5      | Squared-TT Marginalization And Mass Matrices . . . . .                                 | 45        |
| 7.6      | A Small Three-Coordinate Marginalization . . . . .                                     | 47        |
| 7.7      | Algorithm 2, Fully Annotated . . . . .                                                 | 47        |
| <b>8</b> | <b>Fixed-Branch Likelihood Construction</b>                                            | <b>48</b> |
| 8.1      | Fixed-Branch Objects And Array Shapes . . . . .                                        | 49        |
| 8.2      | What A Core Object Looks Like . . . . .                                                | 49        |
| 8.3      | What Must Be Saved For The Next Time Step . . . . .                                    | 50        |
| 8.4      | Consolidated Fixed Least-Squares Formulation . . . . .                                 | 50        |
| 8.5      | Inputs, Outputs, And Stored Fields . . . . .                                           | 51        |
| 8.6      | Initialization And Sweep Order . . . . .                                               | 52        |
| 8.7      | Forward Core Update . . . . .                                                          | 52        |
| 8.8      | Derivative Core Update . . . . .                                                       | 52        |
| 8.9      | Alternating-Sweep Recomputations . . . . .                                             | 53        |
| 8.10     | Stopping Rule And Failure Exits . . . . .                                              | 54        |
| 8.11     | One Concrete Branch, Fully Instantiated . . . . .                                      | 55        |
| 8.12     | Coordinate Order And Domain . . . . .                                                  | 55        |
| 8.13     | Basis, Points, Weights, Ranks . . . . .                                                | 56        |
| 8.14     | Deterministic Rank Ladder Before Branch Freezing . . . . .                             | 56        |
| 8.15     | Defensive Reference, Mass, And Shift . . . . .                                         | 57        |
| 8.16     | Boxed Convention: Shifted Fit, Unshifted Density . . . . .                             | 57        |
| 8.17     | Initialization And Sweep Order . . . . .                                               | 59        |
| 8.18     | Forward-Step Object Flow . . . . .                                                     | 60        |
| 8.19     | Conditional-Map Inversion Protocol . . . . .                                           | 60        |
| 8.20     | Fixed-Branch TT Filtering Recursion . . . . .                                          | 61        |
| 8.21     | End-To-End Fixed-Branch Squared-TT Filter . . . . .                                    | 61        |
| 8.22     | Boxed Algorithm: Fixed-Branch Filter And Derivative Together . . . . .                 | 61        |
| <b>9</b> | <b>Fixed-Branch Derivative Theory</b>                                                  | <b>64</b> |
| 9.1      | Fixed-Branch Gradient Motivation . . . . .                                             | 64        |
| 9.2      | Warmup 1: A Normalizing Constant . . . . .                                             | 65        |
| 9.3      | Warmup 2: A Squared Approximation . . . . .                                            | 66        |
| 9.4      | Warmup 3: Two Coordinates, Rank One . . . . .                                          | 67        |
| 9.5      | Warmup 5: A Fixed Linear Solve . . . . .                                               | 68        |
| 9.6      | The Fixed-Branch Forward Pass . . . . .                                                | 69        |
| 9.7      | Gradient Teaching Layer: What Is Differentiated . . . . .                              | 70        |
| 9.8      | The Fixed-Branch Derivative Pass . . . . .                                             | 71        |
| 9.9      | Target And Square-Root Target . . . . .                                                | 72        |
| 9.10     | Design Matrix And Core Solve . . . . .                                                 | 72        |
| 9.11     | Mass Contraction And Normalizer . . . . .                                              | 74        |
| 9.12     | Carried Filter And Next Time Step . . . . .                                            | 74        |
| 9.13     | Concrete One-Coordinate Carried-Filter Storage . . . . .                               | 75        |
| 9.14     | Proposition 1: The Fixed-Branch Recursion Is A Normalized Approximate Filter . . . . . | 79        |

|          |                                                                                     |            |
|----------|-------------------------------------------------------------------------------------|------------|
| 9.15     | The Story Of The Fixed-Branch Derivative . . . . .                                  | 80         |
| 9.16     | Proposition 2: The Fixed-Branch Derivative Differentiates The Same Scalar . . . . . | 81         |
| 9.17     | What This Does Not Prove . . . . .                                                  | 83         |
| 9.18     | What Must Be Held Fixed For The Derivative . . . . .                                | 84         |
| 9.19     | Four Checklists For Reading The Derivative . . . . .                                | 85         |
| <b>A</b> | <b>Validation And Diagnostics</b>                                                   | <b>86</b>  |
| A.1      | Minimal Mathematical Example . . . . .                                              | 88         |
| A.2      | Large-Scale Validation Models And Test Protocol . . . . .                           | 88         |
| A.3      | What The Tests Must Measure . . . . .                                               | 89         |
| A.4      | Accuracy Metrics With Exact Or Reference Targets . . . . .                          | 90         |
| A.5      | The Exact-Reference Linear Gaussian Benchmark . . . . .                             | 90         |
| A.6      | The Long-Horizon Stochastic-Volatility Benchmark . . . . .                          | 92         |
| A.7      | The Spatial SIR Benchmark For High-Dimensional State Filtering . . . . .            | 93         |
| A.8      | Rank-Calibrated Spatial SIR Route For Fixed-Branch Filtering . . . . .              | 95         |
| A.9      | P53 Route-Class Repair For Spatial SIR Transition Application . . . . .             | 97         |
| A.10     | The Predator-Prey Benchmark For Nonlinear Preconditioning . . . . .                 | 98         |
| A.11     | BayesFilter Stress Ladders Beyond The Paper . . . . .                               | 99         |
| A.12     | Robustness Tests And Veto Conditions . . . . .                                      | 99         |
| A.13     | A Compact Benchmark Table Template . . . . .                                        | 100        |
| A.14     | What Each Benchmark Can And Cannot Establish . . . . .                              | 101        |
| <b>B</b> | <b>Positioning, Limits, And Relation To Zhao And Cui</b>                            | <b>101</b> |
| B.1      | Relation To Zhao And Cui . . . . .                                                  | 102        |

## 1 Orientation And Scope

The Zhao and Cui [4] method is easier to follow if the notation is organized around five objects. They are not five separate algorithms; they are five views of one sequential density approximation:

$$q_t, \quad \phi_t, \quad C_{t,k}, \quad \hat{Z}_t, \quad \partial_\beta \log \hat{Z}_t. \quad (1.1)$$

The first three describe the approximated density. The fourth is the scalar contributed to the likelihood. The fifth is the sensitivity needed by the fixed-branch likelihood specialization.

### Object 1: the unnormalized density

Bayes' rule begins with an unnormalized numerator:

$$\text{posterior density} = \frac{\text{likelihood} \times \text{prior}}{\int \text{likelihood} \times \text{prior}}. \quad (1.2)$$

In filtering, the prior at time  $t$  is the previous filter propagated through the transition model. A typical adjacent-state target has the form

$$q_t(x_t, x_{t-1}; \beta) = \hat{p}_{t-1}(x_{t-1}; \beta) f_\beta(x_t | x_{t-1}) g_\beta(y_t | x_t). \quad (1.3)$$

The implementation question is more concrete than the notation suggests: which coordinates are used, and at which fitting points is  $q_t$  evaluated? A fixed branch stores

$$Z_{\text{fit}} = \begin{bmatrix} z_1^{(1)} & z_2^{(1)} \\ \vdots & \vdots \\ z_1^{(N)} & z_2^{(N)} \end{bmatrix}, \quad \text{shape}(Z_{\text{fit}}) = (N, 2), \quad (1.4)$$

and a target vector

$$\tilde{q}_t^{\text{fit}} = \left( \tilde{q}_t(z^{(1)}; \beta), \dots, \tilde{q}_t(z^{(N)}; \beta) \right)^\top, \quad \text{shape}(\tilde{q}_t^{\text{fit}}) = (N, ). \quad (1.5)$$

## Object 2: the square-root approximation

A direct TT fit of  $q_t$  can be negative. Zhao and Cui [4] therefore fit a square root and square it back:

$$\phi_t(z; \beta) \approx e^{c_t/2} \sqrt{\tilde{q}_t(z; \beta)}, \quad (1.6)$$

$$\hat{q}_t(z; \beta) = e^{-c_t} \phi_t(z; \beta)^2 + \tau_t \lambda_t(z). \quad (1.7)$$

The fitted values are

$$y_t^{\text{fit}}[j] = e^{c_t/2} \sqrt{\tilde{q}_t(z^{(j)}; \beta)}, \quad \text{shape}(y_t^{\text{fit}}) = (N, ), \quad (1.8)$$

and the fixed-branch derivative values are

$$\dot{y}_t^{\text{fit}}[j] = e^{c_t/2} \frac{\dot{\tilde{q}}_t(z^{(j)}; \beta)}{2\sqrt{\tilde{q}_t(z^{(j)}; \beta)}}. \quad (1.9)$$

## Object 3: the tensor-train cores

For two coordinates, the fitted square root can be written

$$\phi_t(z_1, z_2; \beta) = \sum_{r=1}^R \left[ \sum_{\ell=0}^{p-1} C_1[0, \ell, r] b_1(z_1)[\ell] \right] \left[ \sum_{m=0}^{p-1} C_2[r, m, 0] b_2(z_2)[m] \right]. \quad (1.10)$$

The stored arrays are

$$\text{shape}(C_1) = \text{shape}(\dot{C}_1) = (1, p, R), \quad \text{shape}(C_2) = \text{shape}(\dot{C}_2) = (R, p, 1). \quad (1.11)$$

The branch freezes the fitting rule, not the fitted numerical core values:

$$C_k(\beta; B) = \text{core returned by the fixed fitting equations at } \beta. \quad (1.12)$$

## Object 4: the normalizer

The evidence increment is the mass of the declared approximate density:

$$\hat{Z}_t(\beta) = e^{-c_t} R_t(\beta) + \tau_t, \quad R_t(\beta) = \int \phi_t(z; \beta)^2 dz. \quad (1.13)$$

In TT form, this mass is computed by contractions rather than by a full tensor grid. For the two-coordinate case,

$$S_2[r, s] = \sum_{\ell, m} C_2[r, \ell, 0] B_2[\ell, m] C_2[s, m, 0], \quad (1.14)$$

$$R_t = \sum_{r, s, \ell, m} C_1[0, \ell, r] B_1[\ell, m] C_1[0, m, s] S_2[r, s]. \quad (1.15)$$

## Object 5: the derivative of the log normalizer

The derivative used by the fixed-branch likelihood is

$$\partial_\beta \log \hat{Z}_t = \frac{\dot{\hat{Z}}_t}{\hat{Z}_t}. \quad (1.16)$$

Since  $c_t, \tau_t, \lambda_t$  are frozen in the fixed branch,

$$\dot{\hat{Z}}_t = e^{-c_t} \dot{R}_t. \quad (1.17)$$

The derivative of the carried filter is equally important because it becomes part of the next target:

$$\dot{\hat{p}}_t(z_t; \beta) = \frac{\dot{a}_t(z_t; \beta)}{\hat{Z}_t(\beta)} - \frac{a_t(z_t; \beta) \dot{\hat{Z}}_t(\beta)}{\hat{Z}_t(\beta)^2}. \quad (1.18)$$

This front orientation is not a replacement for the derivation below. It is the small map used to read the longer source-order reconstruction.

## 2 Running Example And Notation

The purpose of the note is to make the mathematical mechanism readable without asking the reader to reconstruct missing steps from the original paper. The starting point is the tensor-train sequential learning method of Zhao and Cui [4]; tensor trains themselves follow Oseledets [2], and the transport maps used for conditional sampling are Knothe–Rosenblatt maps in the sense of Rosenblatt’s triangular transformation [3]. Squared inverse Rosenblatt transport ideas also connect to the work of Cui and Dolgov [1].

Zhao and Cui write the unknown static parameter as  $\theta$  [4]. The reconstruction below follows that convention when the parameter is part of a posterior density. The fixed-branch derivative section later writes the differentiated external parameter as  $\beta$ , so that the same symbol is not used both for an integration coordinate and for the argument of an approximate likelihood.

The state dimension is  $m$ , observation dimension is  $n$ , and parameter dimension is  $d$ . Thus

$$x_t = (x_{t,1}, \dots, x_{t,m}) \in \mathcal{X} \subseteq \mathbb{R}^m, \quad y_t = (y_{t,1}, \dots, y_{t,n}) \in \mathcal{Y} \subseteq \mathbb{R}^n,$$

and

$$\theta = (\theta_1, \dots, \theta_d) \in \Theta \subseteq \mathbb{R}^d.$$

For coordinate groups, the paper uses

$$x_{t,<j} = (x_{t,1}, \dots, x_{t,j-1}), \quad x_{t,>j} = (x_{t,j+1}, \dots, x_{t,m}), \quad (2.1)$$

and similarly

$$x_{t,\leq j} = (x_{t,1}, \dots, x_{t,j}), \quad x_{t,\geq j} = (x_{t,j}, \dots, x_{t,m}). \quad (2.2)$$

The point of this notation is conditional density bookkeeping. A lower triangular map conditions coordinate  $j$  on  $x_{<j}$ ; an upper triangular map conditions it on  $x_{>j}$ .

All densities in the reconstruction are assumed absolutely continuous with respect to the relevant Lebesgue measure. Zhao and Cui [4] use  $p$  for normalized posterior densities and  $\pi$  for unnormalized

densities. Their approximations are  $\hat{p}$  and  $\hat{\pi}$ . When  $p$  and  $\hat{p}$  are normalized densities, the Hellinger distance is

$$D_H(p, \hat{p}) = \left[ \frac{1}{2} \int (\sqrt{p(r)} - \sqrt{\hat{p}(r)})^2 dr \right]^{1/2}. \quad (2.3)$$

This distance appears because the squared-TT method approximates square roots of densities.

If  $S : \mathcal{X} \rightarrow \mathcal{U}$  is a diffeomorphism and  $X \sim p$ , then the density of  $U = S(X)$  is the pushforward

$$S_{\#}p(u) = p(S^{-1}(u)) |\det \nabla S^{-1}(u)|. \quad (2.4)$$

If  $U \sim \eta$ , then the density of  $X = S^{-1}(U)$  is the pullback

$$S^{\#}\eta(x) = \eta(S(x)) |\det \nabla S(x)|. \quad (2.5)$$

These two identities are the entire mathematical engine behind the Knothe–Rosenblatt maps and the preconditioning maps in Section 5. The implementation stores an evaluator for  $S$ , an evaluator for  $S^{-1}$ , and the relevant log-Jacobian determinant when the map is used as a density transform.

## 2.1 Threaded Nonlinear Example Used Throughout The Note

This example is intentionally small enough to hold in one line, but nonlinear enough to show why Gaussian projection alone may be too rigid. Let

$$x_0 \sim N(\mu_0, \sigma_0^2), \quad x_t = \beta x_{t-1} + \sigma \epsilon_t, \quad \epsilon_t \sim N(0, 1), \quad (2.6)$$

and

$$y_t = \sin(x_t) + \eta_t, \quad \eta_t \sim N(0, r^2). \quad (2.7)$$

The transition and observation densities are therefore

$$f_{\beta}(x_t | x_{t-1}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[ -\frac{(x_t - \beta x_{t-1})^2}{2\sigma^2} \right], \quad (2.8)$$

and

$$g_{\beta}(y_t | x_t) = \frac{1}{\sqrt{2\pi r^2}} \exp \left[ -\frac{(y_t - \sin x_t)^2}{2r^2} \right]. \quad (2.9)$$

At time 1, the exact unnormalized adjacent-state target is

$$q_1(x_1, x_0; \beta) = p_{\beta}(x_0) f_{\beta}(x_1 | x_0) g_{\beta}(y_1 | x_1). \quad (2.10)$$

The squared-TT approximation fits

$$\phi_1(z_1, z_0; \beta) \approx e^{c_1/2} \sqrt{q_1(\Psi_1(z_1, z_0); \beta) J_1(z_1, z_0)}, \quad (2.11)$$

and defines

$$\hat{q}_1(z_1, z_0; \beta) = e^{-c_1} \phi_1(z_1, z_0; \beta)^2 + \tau_1 \lambda_1(z_1, z_0). \quad (2.12)$$

The first evidence increment and retained filter are

$$\hat{Z}_1(\beta) = \int \hat{q}_1(z_1, z_0; \beta) dz_1 dz_0, \quad \hat{p}_1(z_1; \beta) = \frac{\int \hat{q}_1(z_1, z_0; \beta) dz_0}{\hat{Z}_1(\beta)}. \quad (2.13)$$

At time 2, the same object reappears inside the next target:

$$q_2(x_2, x_1; \beta) = \hat{p}_1(x_1; \beta) f_{\beta}(x_2 | x_1) g_{\beta}(y_2 | x_2). \quad (2.14)$$

If a bridge  $\rho_2(x_2, x_1)$  is used, the preconditioned residual in reference coordinates is

$$q_2^\#(u; \beta) = \frac{q_2(T_2^{-1}(u); \beta)}{\rho_2(T_2^{-1}(u))} \eta(u). \quad (2.15)$$

The fixed-branch scalar for the two-step example is

$$\widehat{\ell}_2(\beta; B) = \log \widehat{Z}_1(\beta; B_1) + \log \widehat{Z}_2(\beta; B_2), \quad (2.16)$$

with derivative

$$\partial_\beta \widehat{\ell}_2(\beta; B) = \frac{\partial_\beta \widehat{Z}_1(\beta; B_1)}{\widehat{Z}_1(\beta; B_1)} + \frac{\partial_\beta \widehat{Z}_2(\beta; B_2)}{\widehat{Z}_2(\beta; B_2)}. \quad (2.17)$$

The recurrence table for this example is

| location           | anchor         | role                                     |        |
|--------------------|----------------|------------------------------------------|--------|
| filtering target   | (2.10), (2.14) | Bayes numerator before fitting           |        |
| squared-TT density | (2.11)–(2.13)  | nonnegative fitted approximation         | (2.18) |
| preconditioning    | (2.15)         | bridge removes dominant geometry         |        |
| gradient           | (2.16)–(2.17)  | same saved-branch scalar and derivative. |        |

### 3 Exact Filtering Target And Recursive Bottleneck

**Mathematical question.** Zhao and Cui [4] begin with a hidden Markov state, observations, and possibly unknown static parameters. The question answered here is: what density must a sequential algorithm update when new data arrive?

The data arrive in time order. At time  $t$  there is a hidden state

$$x_t \in \mathbb{R}^{m_x}$$

and an observation

$$y_t \in \mathbb{R}^{m_y}.$$

There may also be a static unknown parameter

$$\alpha \in \mathbb{R}^{m_\alpha}.$$

The filtering problem is to update the conditional density of the current state and parameter when  $y_t$  arrives:

$$p(x_t, \alpha \mid y_{1:t}).$$

The path problem is to update the density of the whole trajectory and parameter:

$$p(x_{0:t}, \alpha \mid y_{1:t}).$$

The smoothing problem asks for an earlier state after later data have been seen:

$$p(x_k \mid y_{1:t}), \quad k < t.$$

The bottleneck is always an integral. At time  $t - 1$  suppose we know, or have approximated,

$$p(x_{t-1}, \alpha \mid y_{1:t-1}).$$

After seeing  $y_t$ , Bayes' rule introduces the unnormalized three-block density

$$q_t(x_t, \alpha, x_{t-1}) = p(x_{t-1}, \alpha \mid y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t).$$

The next filter is obtained by integrating out  $x_{t-1}$ :

$$p(x_t, \alpha \mid y_{1:t}) = \frac{\int q_t(x_t, \alpha, x_{t-1}) dx_{t-1}}{\int q_t(x_t, \alpha, x_{t-1}) dx_t d\alpha dx_{t-1}}.$$

In low dimension one can attack this integral by quadrature, particles, or Gaussian projection. Zhao and Cui [4]'s idea is different: approximate the whole function  $q_t$  in a separable tensor-train form so that marginal integrals are cheap contractions instead of high-dimensional numerical integrations.

**Implementation object.** A minimal implementation must expose evaluators for  $\hat{p}_{t-1}$ ,  $f_\alpha$ , and  $g_\alpha$ , then build an evaluator for  $q_t$ . The first failure check is that  $q_t$  must be finite and nonnegative at all fitting points; otherwise neither a square root nor an importance weight is meaningful.

### 3.1 State-Space Model And Notation Used In This Note

**Mathematical question.** The first two source equations answer: what conditional independence structure defines the state-space model? The variables  $X_t \in \mathcal{X} \subseteq \mathbb{R}^m$  are latent states,  $Y_t \in \mathcal{Y} \subseteq \mathbb{R}^n$  are observations, and  $\alpha \in \Theta \subseteq \mathbb{R}^d$  is the static parameter.

Zhao and Cui [4] use  $\theta$  for the unknown parameter. In this note the parameter is written  $\alpha$ , because  $\theta$  is often used later as a generic argument in derivative formulas. The paper's model equations (1)–(2) become

$$X_t \mid (X_{0:t-1} = x_{0:t-1}, \alpha) \equiv X_t \mid (X_{t-1} = x_{t-1}, \alpha) \sim f_\alpha(x_t \mid x_{t-1}), \quad (3.1)$$

and

$$Y_t \mid (X_{0:t} = x_{0:t}, Y_{1:t-1} = y_{1:t-1}, \alpha) \equiv Y_t \mid (X_t = x_t, \alpha) \sim g_\alpha(y_t \mid x_t). \quad (3.2)$$

The first statement says the latent state process is Markov once  $\alpha$  is fixed. The second says the new observation only sees the current state, again once  $\alpha$  is fixed.

**Implementation meaning.** Inputs are  $(x_t, x_{t-1}, \alpha)$  for the transition evaluator and  $(y_t, x_t, \alpha)$  for the observation evaluator. Outputs are nonnegative densities or log densities. Quantities to retain are the model specification and the observed  $y_t$ . Diagnostics are finite log-density checks and shape checks for  $m, n, d$ .

**Mathematical question.** The next question is: how do the local transition and observation densities combine into the full joint density?

The joint density of parameter, states, and observations through time  $t$ , corresponding to Zhao and Cui [4], Eq. (3), is

$$p(\alpha, x_{0:t}, y_{1:t}) = p(\alpha) p_\alpha(x_0) \prod_{j=1}^t f_\alpha(x_j \mid x_{j-1}) g_\alpha(y_j \mid x_j). \quad (3.3)$$

This is obtained by the chain rule:

$$\begin{aligned} p(\alpha, x_{0:t}, y_{1:t}) &= p(\alpha) p_\alpha(x_0) \prod_{j=1}^t p(x_j \mid x_{0:j-1}, y_{1:j-1}, \alpha) p(y_j \mid x_{0:j}, y_{1:j-1}, \alpha) \\ &= p(\alpha) p_\alpha(x_0) \prod_{j=1}^t f_\alpha(x_j \mid x_{j-1}) g_\alpha(y_j \mid x_j), \end{aligned}$$



where the last equality uses the Markov and observation assumptions.

The derivation is the probabilistic analogue of multiplying conditional reaction probabilities along a time-ordered chain. Every new time index adds one transition factor and one observation factor. An implementation stores this as a log-density sum, not as a product, to avoid underflow:

$$\log p(\alpha, x_{0:t}, y_{1:t}) = \log p(\alpha) + \log p_\alpha(x_0) + \sum_{j=1}^t \{\log f_\alpha(x_j | x_{j-1}) + \log g_\alpha(y_j | x_j)\}. \quad (3.4)$$

The diagnostic is simple: any impossible model transition should produce  $-\infty$  log density intentionally, not a NaN.

**Mathematical question.** The fourth source equation answers: how does the joint density become a posterior density after data are observed?

Conditioning on data gives Zhao and Cui [4], Eq. (4) in the notation of this note:

$$p(\alpha, x_{0:t} | y_{1:t}) = \frac{p(\alpha, x_{0:t}, y_{1:t})}{p(y_{1:t})}, \quad p(y_{1:t}) = \int p(\alpha, x_{0:t}, y_{1:t}) d\alpha dx_{0:t}. \quad (3.5)$$

The denominator is the evidence. It is a scalar, but it is usually not available analytically because the integral spans the parameter and all states.

Implementation meaning. The numerator is evaluable pointwise. The evidence is a scalar integral. A filtering algorithm that claims a likelihood must specify how it computes or approximates this scalar. The failure check is that the approximated evidence must be positive and finite.

### 3.2 The Four Marginal Learning Problems

**Mathematical question.** The next four source equations answer: which posterior marginals correspond to filtering, parameter learning, path estimation, and smoothing?

Zhao and Cui [4], Eqs. (5)–(8) are all marginalizations of the same posterior  $p(\alpha, x_{0:t} | y_{1:t})$ . Filtering keeps only the current state:

$$p(x_t | y_{1:t}) = \int p(\alpha, x_{0:t} | y_{1:t}) dx_{0:t-1} d\alpha. \quad (3.6)$$

Parameter learning keeps only  $\alpha$ :

$$p(\alpha | y_{1:t}) = \int p(\alpha, x_{0:t} | y_{1:t}) dx_{0:t}. \quad (3.7)$$

Path learning keeps the full state path but removes  $\alpha$ :

$$p(x_{0:t} | y_{1:t}) = \int p(\alpha, x_{0:t} | y_{1:t}) d\alpha. \quad (3.8)$$

Fixed-lag or retrospective smoothing keeps one old state:

$$p(x_k | y_{1:t}) = \int p(\alpha, x_{0:t} | y_{1:t}) dx_{0:k-1} dx_{k+1:t} d\alpha, \quad k < t. \quad (3.9)$$

The important point is not the names. It is that the same high-dimensional posterior density produces all four outputs by integration. A method that stores only a cloud of samples can estimate these integrals by sample averages. A method that stores a Gaussian can compute only Gaussian

marginals. A method that stores a tensor-train density aims to keep a non-Gaussian function representation while making many marginal integrals cheap.

Implementation meaning. The stored object must support integrating out named coordinate blocks. The outputs are marginal density evaluators. The failure mode is inconsistent normalization: if integrating a marginal over its retained coordinates does not give one, the contraction or normalizer is wrong.

### 3.3 The Exact Recursive Bottleneck

**Mathematical question.** These source equations answer: how can the posterior update be performed sequentially without storing the whole old path forever?

We now derive Zhao and Cui [4], Eqs. (9)–(11). Start from Bayes’ rule:

$$p(\alpha, x_{0:t} \mid y_{1:t}) = \frac{p(\alpha, x_{0:t}, y_{1:t})}{p(y_{1:t})}.$$

Using the factorization (BF-3),

$$p(\alpha, x_{0:t}, y_{1:t}) = p(\alpha, x_{0:t-1}, y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t).$$

Also,

$$p(\alpha, x_{0:t-1}, y_{1:t-1}) = p(\alpha, x_{0:t-1} \mid y_{1:t-1}) p(y_{1:t-1}).$$

Therefore

$$\begin{aligned} p(\alpha, x_{0:t} \mid y_{1:t}) &= \frac{p(\alpha, x_{0:t-1} \mid y_{1:t-1}) p(y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t)}{p(y_{1:t})} \\ &= \frac{p(\alpha, x_{0:t-1} \mid y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t)}{p(y_t \mid y_{1:t-1})}. \end{aligned} \quad (3.10)$$

The last step uses  $p(y_{1:t}) = p(y_t \mid y_{1:t-1}) p(y_{1:t-1})$ .

Now integrate (BF-9) over  $x_{0:t-2}$ . The only factor depending on the old path  $x_{0:t-2}$  is  $p(\alpha, x_{0:t-1} \mid y_{1:t-1})$ , so

$$\begin{aligned} p(\alpha, x_t, x_{t-1} \mid y_{1:t}) &= \int p(\alpha, x_{0:t} \mid y_{1:t}) dx_{0:t-2} \\ &= \frac{p(\alpha, x_{t-1} \mid y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t)}{p(y_t \mid y_{1:t-1})}. \end{aligned} \quad (3.11)$$

Finally, integrate over  $x_{t-1}$  to obtain the next parameter-state filter:

$$p(\alpha, x_t \mid y_{1:t}) = \int p(\alpha, x_t, x_{t-1} \mid y_{1:t}) dx_{t-1}. \quad (3.12)$$

The numerator in (BF-10) is pointwise evaluable if the previous filter is evaluable. The denominator and the marginalization in (BF-11) are the hard parts. Zhao and Cui [4] turn this into a function-approximation problem by replacing the unnormalized numerator with a tensor train.

Implementation meaning. 1-Eq9–Eq11. Inputs are the retained evaluator for  $p(\alpha, x_{t-1} \mid y_{1:t-1})$ , the new observation  $y_t$ , and model evaluators  $f_\alpha, g_\alpha$ . The output of the numerator step is a pointwise evaluator for the three-block target. The quantities to retain after marginalization is an evaluator for  $p(\alpha, x_t \mid y_{1:t})$  and an evidence increment. The failure check is that the marginal integral over  $(\alpha, x_t)$  must equal one after normalization.

## 4 Tensor-Train Approximation Toolkit

**Mathematical question.** The question answered here is: what separable function class will replace a full high-dimensional grid?

This section starts from the object a numerical reader can picture: a table of function values. If  $D = 2$ , a function  $h(r_1, r_2)$  sampled on a grid is a matrix. Low-rank matrix approximation writes

$$h(r_1, r_2) \approx \sum_{\rho=1}^R u_{\rho}(r_1) v_{\rho}(r_2).$$

Each term is separable: one factor depends only on  $r_1$ , one factor only on  $r_2$ . A tensor train is the many-variable analogue. It does not require a full  $D$ -dimensional table. It stores a chain of low-rank links, so that information can pass from coordinate 1 to coordinate  $D$  through the rank indices.

Let  $r = (r_1, \dots, r_D) \in \Omega_1 \times \dots \times \Omega_D$ . A functional tensor train approximates a scalar function  $h(r)$  by multiplying matrix-valued one-dimensional functions:

$$h(r) \approx \hat{h}(r) = H_1(r_1) H_2(r_2) \cdots H_D(r_D), \quad (4.1)$$

where

$$H_k(r_k) \in \mathbb{R}^{R_{k-1} \times R_k}, \quad R_0 = R_D = 1.$$

Zhao and Cui [4]’s displayed TT formula includes the endpoint rank indices:

$$\hat{h}(r) = \sum_{\rho_0=1}^{R_0} \sum_{\rho_1=1}^{R_1} \cdots \sum_{\rho_D=1}^{R_D} H_1^{(\rho_0, \rho_1)}(r_1) \cdots H_D^{(\rho_{D-1}, \rho_D)}(r_D), \quad R_0 = R_D = 1. \quad (4.2)$$

The endpoint sums have only one term. They are written so every core has the same two rank labels. A programmer usually evaluates the matrix product (TT-1); a proof often expands the rank contractions as in (TT-1a).

Because the first object has one row and the last has one column, the product is a  $1 \times 1$  scalar. Written with indices,

$$\hat{h}(r) = \sum_{\rho_1=1}^{R_1} \cdots \sum_{\rho_{D-1}=1}^{R_{D-1}} H_1^{1, \rho_1}(r_1) H_2^{\rho_1, \rho_2}(r_2) \cdots H_D^{\rho_{D-1}, 1}(r_D). \quad (4.3)$$

The numbers  $R_k$  are TT ranks. A high rank lets the representation express strong interactions across a split  $(r_1, \dots, r_k)$  and  $(r_{k+1}, \dots, r_D)$ . A low rank is useful only when the function has low-dimensional structure across that split.

To see the role of the ranks, split the variables into a left block and a right block:

$$r_{\leq k} = (r_1, \dots, r_k), \quad r_{> k} = (r_{k+1}, \dots, r_D).$$

Equation (TT-2) can be regrouped as

$$\hat{h}(r_{\leq k}, r_{> k}) = \sum_{\rho=1}^{R_k} L_{\rho}(r_{\leq k}) R_{\rho}(r_{> k}), \quad (4.4)$$

where

$$L_{\rho}(r_{\leq k}) = \sum_{\rho_1, \dots, \rho_{k-1}} H_1^{1, \rho_1}(r_1) \cdots H_k^{\rho_{k-1}, \rho}(r_k),$$

and

$$R_\rho(r_{>k}) = \sum_{\rho_{k+1}, \dots, \rho_{D-1}} H_{k+1}^{\rho, \rho_{k+1}}(r_{k+1}) \cdots H_D^{\rho_{D-1}, 1}(r_D).$$

Thus  $R_k$  is the number of separated terms allowed across that split. If a posterior density has only local or weak long-range dependence in the chosen ordering, moderate ranks can be enough. If a posterior has strong global coupling, the ranks grow and the method becomes expensive.

The low-dimensional analogy is the singular value decomposition of a matrix. For a two-coordinate function, rank  $R$  means  $R$  separable columns times rows. For many coordinates,  $R_k$  plays the same role at the cut after coordinate  $k$ . The failure diagnostic is rank growth: if any fitted  $R_k$  approaches the imposed maximum, the chosen coordinate ordering, basis, or preconditioner is not expressing the density compactly.

## Rank-Count Plausibility Calculation

If every coordinate uses  $p$  basis functions and the function has  $D$  coordinates, a full tensor stores

$$N_{\text{full}} = p^D \quad (4.5)$$

coefficients. This is the algebraic curse of dimensionality. For  $p = 8$  and  $D = 20$ ,

$$8^{20} = 2^{60} \approx 1.15 \times 10^{18}. \quad (4.6)$$

A tensor train with uniform rank  $R$  stores approximately

$$N_{\text{TT}} = \sum_{k=1}^D R_{k-1} p R_k \approx D p R^2, \quad R_0 = R_D = 1. \quad (4.7)$$

For  $p = 8$ ,  $D = 20$ ,  $R = 10$ ,

$$D p R^2 = 20 \cdot 8 \cdot 10^2 = 16000. \quad (4.8)$$

Mass contractions usually cost one extra rank factor:

$$C_{\text{mass}} \approx O(D p R^3). \quad (4.9)$$

This comparison is the reason the method is plausible, not a proof that it will work. The entire empirical hinge is

$$R \text{ stays moderate under the chosen ordering, basis, domain, and preconditioner.} \quad (4.10)$$

If ranks grow toward the full tensor scale, the TT representation has not escaped the curse of dimensionality for that target.

## A Fully Worked Three-Variable Exponential Example

The abstract formula (4.1) becomes much less mysterious when one multiplies through a small example. Consider the three-variable function

$$h(x, y, z) = \exp\{\alpha xy + \beta yz\}, \quad (x, y, z) \in \mathbb{R}^3. \quad (4.11)$$

This function is not separable as  $h_1(x)h_2(y)h_3(z)$ . The middle coordinate  $y$  couples to both  $x$  and  $z$ . The interaction graph is a chain,

$$x \text{ --- } y \text{ --- } z,$$

which is exactly the kind of structure a tensor train can exploit.

Start by approximating each exponential edge by its degree- $M$  Taylor polynomial:

$$E_{12}^{(M)}(x, y) = \sum_{a=0}^M \frac{\alpha^a}{a!} x^a y^a, \quad E_{23}^{(M)}(y, z) = \sum_{b=0}^M \frac{\beta^b}{b!} y^b z^b. \quad (4.12)$$

The truncated target is

$$h_M(x, y, z) = E_{12}^{(M)}(x, y) E_{23}^{(M)}(y, z) = \sum_{a=0}^M \sum_{b=0}^M \frac{\alpha^a \beta^b}{a! b!} x^a y^{a+b} z^b. \quad (4.13)$$

Equation (4.13) already has the tensor-train structure. Use ranks

$$R_0 = R_3 = 1, \quad R_1 = R_2 = M + 1,$$

with rank labels  $a, b \in \{0, \dots, M\}$ . Define

$$H_1(x)[1, a] = \frac{\alpha^a}{a!} x^a, \quad (4.14)$$

$$H_2(y)[a, b] = y^{a+b}, \quad (4.15)$$

$$H_3(z)[b, 1] = \frac{\beta^b}{b!} z^b. \quad (4.16)$$

Then

$$H_1(x)H_2(y)H_3(z) = \sum_{a=0}^M \sum_{b=0}^M \frac{\alpha^a}{a!} x^a y^{a+b} \frac{\beta^b}{b!} z^b = h_M(x, y, z). \quad (4.17)$$

The rank index  $a$  remembers how much  $xy$ -interaction has entered from the left edge. The rank index  $b$  remembers how much  $yz$ -interaction will leave through the right edge. The middle core is where the two interaction orders meet.

Take a concrete numerical case:

$$M = 2, \quad \alpha = \frac{1}{2}, \quad \beta = -\frac{1}{3}, \quad (x, y, z) = \left(2, \frac{1}{2}, -1\right).$$

The three cores are

$$H_1(2) = \begin{bmatrix} 1 & 1 & 1/2 \end{bmatrix}, \quad (4.18)$$

$$H_2(1/2) = \begin{bmatrix} 1 & 1/2 & 1/4 \\ 1/2 & 1/4 & 1/8 \\ 1/4 & 1/8 & 1/16 \end{bmatrix}, \quad (4.19)$$

and

$$H_3(-1) = \begin{bmatrix} 1 \\ 1/3 \\ 1/18 \end{bmatrix}. \quad (4.20)$$

First multiply the first two cores:

$$H_1(2)H_2(1/2) = \begin{bmatrix} 13/8 & 13/16 & 13/32 \end{bmatrix}. \quad (4.21)$$

Then multiply by the final core:

$$\begin{aligned} H_1(2)H_2(1/2)H_3(-1) &= \frac{13}{8} + \frac{13}{16} \frac{1}{3} + \frac{13}{32} \frac{1}{18} \\ &= \frac{1105}{576} \approx 1.9184. \end{aligned} \quad (4.22)$$

The true exponential value is

$$\exp \left\{ \frac{1}{2} \cdot 2 \cdot \frac{1}{2} - \frac{1}{3} \cdot \frac{1}{2} \cdot (-1) \right\} = e^{2/3} \approx 1.9477. \quad (4.23)$$

The difference is not a tensor-train multiplication error. It is the Taylor truncation error in replacing  $h$  by  $h_M$ . Raising  $M$  increases the TT ranks and reduces this particular approximation error. This is the same tradeoff that later appears in filtering: richer cores and higher ranks can represent more interaction, but they cost more and can become harder to fit.

## How The Least-Squares Core Regression Looks In The Same Example

The fitting sections later in the note use rows of the form

$$\widehat{h}(r^{(j)}) = A_{j,k} g_k,$$

where  $g_k = \text{vec}(C_k)$  is one unknown core and the other cores are held fixed. The exponential example shows exactly what this row means.

Use the first-order truncation  $M = 1$ :

$$h_1(x, y, z) = (1 + \alpha xy)(1 + \beta yz) = 1 + \alpha xy + \beta yz + \alpha \beta xy^2 z. \quad (4.24)$$

The TT ranks are  $R_1 = R_2 = 2$ . Use local basis vectors

$$b_1(x) = \begin{bmatrix} 1 & x \end{bmatrix}, \quad b_2(y) = \begin{bmatrix} 1 & y & y^2 \end{bmatrix}, \quad b_3(z) = \begin{bmatrix} 1 & z \end{bmatrix}. \quad (4.25)$$

One exact set of cores is

$$H_1(x) = \begin{bmatrix} 1 & \alpha x \end{bmatrix}, \quad H_3(z) = \begin{bmatrix} 1 \\ \beta z \end{bmatrix}, \quad (4.26)$$

and

$$H_2(y) = \begin{bmatrix} 1 & y \\ y & y^2 \end{bmatrix}. \quad (4.27)$$

If the middle core is unknown, write it as

$$C_2[a, \ell, b], \quad a, b \in \{0, 1\}, \quad \ell \in \{0, 1, 2\},$$

so that

$$H_2(y)[a, b] = \sum_{\ell=0}^2 C_2[a, \ell, b] y^\ell. \quad (4.28)$$

The exact middle core for (4.27) has four nonzero entries:

$$C_2[0, 0, 0] = 1, \quad C_2[0, 1, 1] = 1, \quad C_2[1, 1, 0] = 1, \quad C_2[1, 2, 1] = 1. \quad (4.29)$$

At a fitting point  $r^{(j)} = (x_j, y_j, z_j)$ , the left and right environments for the middle core are

$$L_j = H_1(x_j) = \begin{bmatrix} 1 & \alpha x_j \end{bmatrix}, \quad R_j = H_3(z_j) = \begin{bmatrix} 1 \\ \beta z_j \end{bmatrix}. \quad (4.30)$$

The least-squares coefficient multiplying  $C_2[a, \ell, b]$  is

$$A_{j,2}[a, \ell, b] = L_j[a] y_j^\ell R_j[b]. \quad (4.31)$$

This is the concrete version of the environment formula used later in (7.27) and (7.36).

Use the same numerical point as above with  $M = 1$ :

$$\alpha = \frac{1}{2}, \quad \beta = -\frac{1}{3}, \quad (x_j, y_j, z_j) = \left(2, \frac{1}{2}, -1\right).$$

Then

$$L_j = \begin{bmatrix} 1 & 1 \end{bmatrix}, \quad b_2(y_j) = \begin{bmatrix} 1 & 1/2 & 1/4 \end{bmatrix}, \quad R_j = \begin{bmatrix} 1 \\ 1/3 \end{bmatrix}. \quad (4.32)$$

The row  $A_{j,2}$  is the list of all products

$$L_j[a] b_2(y_j)[\ell] R_j[b].$$

When dotted with the exact middle-core vector from (4.29), only four entries contribute:

$$\begin{aligned} A_{j,2}g_2 &= 1 + \frac{1}{2} \cdot \frac{1}{3} + 1 \cdot \frac{1}{2} + 1 \cdot \frac{1}{4} \cdot \frac{1}{3} \\ &= \frac{7}{4}. \end{aligned} \quad (4.33)$$

Direct multiplication gives the same value:

$$H_1(2)H_2(1/2)H_3(-1) = (1 + \alpha xy)(1 + \beta yz) = \left(1 + \frac{1}{2}\right) \left(1 + \frac{1}{6}\right) = \frac{7}{4}. \quad (4.34)$$

Now suppose the target values in the least-squares problem are not the first-order truncation  $h_1$ , but the true exponential  $h$ . With fitting points  $r^{(j)}$  and weights  $w_j$ , the middle-core update solves

$$\min_{g_2} \sum_j w_j (A_{j,2}g_2 - h(r^{(j)}))^2 + \rho \|g_2\|_2^2. \quad (4.35)$$

The regression is therefore not mysterious interpolation jargon. It is the ordinary problem of choosing the middle-core coefficients so that the TT product best matches the target values at the fitting points, while the left and right cores are held fixed. Alternating regression repeats this operation for core 1, then core 2, then core 3, and then sweeps back. In the full filtering algorithm, the target values  $h(r^{(j)})$  are replaced by unnormalized filtering-density values or square-root density values evaluated at the chosen fitting points.

## 5 Why Moderate Tensor-Train Rank Is A Plausible Hypothesis

Storage counts explain why tensor trains are attractive, but they do not by themselves explain why the rank should remain moderate for a nonlinear posterior. The mathematical reason to try the

method is that many filtering densities have structure across coordinate splits. Write a transformed filtering target as

$$h(r_1, \dots, r_D) = q_t(r_1, \dots, r_D) J_t(r_1, \dots, r_D), \quad (5.1)$$

where  $r$  denotes physical or transformed coordinates and  $J_t$  is the Jacobian factor from the chosen domain map. At the split after coordinate  $k$ , define

$$r_{\leq k} = (r_1, \dots, r_k), \quad r_{> k} = (r_{k+1}, \dots, r_D). \quad (5.2)$$

A rank- $R_k$  separated approximation across this split has the form

$$h(r_{\leq k}, r_{> k}) \approx \sum_{a=1}^{R_k} U_a(r_{\leq k}) V_a(r_{> k}). \quad (5.3)$$

The TT rank  $R_k$  is the number of left-right interaction channels needed at that split. It is not a property of one coordinate in isolation; it is a property of the dependence between the first  $k$  coordinates and the remaining coordinates.

For a two-block discretization, collect  $h(r_{\leq k}, r_{> k})$  on a tensor product grid and reshape it into a matrix

$$H_k[i, j] = h(r_{\leq k}^{(i)}, r_{> k}^{(j)}). \quad (5.4)$$

The best matrix rank- $R_k$  approximation is controlled by the singular values

$$H_k = \sum_{a \geq 1} \sigma_{k,a} u_{k,a} v_{k,a}^\top, \quad \|H_k - H_{k,R}\|_F^2 = \sum_{a > R} \sigma_{k,a}^2. \quad (5.5)$$

The continuous TT split rank is the same idea applied successively to all left-right splits. Moderate rank is plausible when the split singular values decay quickly after a good coordinate ordering and preconditioning map.

Filtering models often create this kind of structure. If the transition density has local coupling,

$$f(x_t \mid x_{t-1}) \approx \prod_{j=1}^m f_j(x_{t,j} \mid x_{t-1,j}, x_{t-1,j-1}, x_{t-1,j+1}), \quad (5.6)$$

and the observation density is conditionally local,

$$g(y_t \mid x_t) \approx \prod_{\ell=1}^n g_\ell(y_{t,\ell} \mid x_{t,\mathcal{I}_\ell}), \quad (5.7)$$

then a coordinate order that keeps neighboring variables near one another can leave only a limited amount of dependence crossing each split. The TT ranks then measure the width of the statistical interaction crossing the ordering, not the ambient dimension alone.

Here is the same rank idea in a physical high-dimensional example. Imagine measurements from a one-dimensional sensor array, a polymer chain, or a reaction coordinate discretized into neighboring sites. Let  $x_j$  be the unknown local state at site  $j$ . A common local energy model has the form

$$\log q(x_1, \dots, x_m) = \sum_{j=1}^m a_j(x_j) + \sum_{j=1}^{m-1} b_j(x_j, x_{j+1}), \quad (5.8)$$



where  $a_j$  is the single-site evidence and  $b_j$  is the nearest-neighbor interaction. Across the split after coordinate  $k$ , all terms in (5.8) are entirely left or entirely right except the one bridge term  $b_k(x_k, x_{k+1})$ :

$$q(x_1, \dots, x_m) = \exp \left[ \sum_{j \leq k} a_j(x_j) + \sum_{j < k} b_j(x_j, x_{j+1}) \right] \times \exp[b_k(x_k, x_{k+1})] \exp \left[ \sum_{j > k} a_j(x_j) + \sum_{j > k} b_j(x_j, x_{j+1}) \right]. \quad (5.9)$$

Only the middle factor crosses the split. If that bridge factor has a separated expansion

$$\exp[b_k(x_k, x_{k+1})] \approx \sum_{a=1}^{R_{\text{edge}}} u_{k,a}(x_k) v_{k,a}(x_{k+1}), \quad (5.10)$$

then the whole density has a split-separated representation with rank at most  $R_{\text{edge}}$  at that cut:

$$q(x_{\leq k}, x_{> k}) \approx \sum_{a=1}^{R_{\text{edge}}} U_{k,a}(x_{\leq k}) V_{k,a}(x_{> k}). \quad (5.11)$$

For a two-dimensional grid ordered by rows, a vertical cut crosses several nearest-neighbor edges rather than one. If  $w_k$  edges cross the split and each edge needs at most  $R_{\text{edge}}$  channels, the crude product bound is

$$R_k \lesssim R_{\text{edge}}^{w_k}. \quad (5.12)$$

This bound is not a sharp theorem, but it makes the hypothesis vivid: moderate TT rank is plausible when the coordinate order keeps the number and complexity of cross-split physical interactions moderate. It is much less plausible when one observation creates a sharp constraint involving nearly all coordinates at once.

The same equations show when the method should fail. If a likelihood creates a global constraint such as

$$y_t \approx \sum_{j=1}^m a_j x_{t,j} \quad (5.13)$$

with very small noise, then many coordinates are coupled through one sharp surface. The split singular values in (5.5) may decay slowly, and rank growth becomes a diagnostic rather than a surprise. A multimodal posterior with modes separated along many coordinate directions can have the same effect.

Preconditioning is included precisely because the raw target may have poor rank behavior. Suppose a map  $T_t$  sends a simpler reference density  $\eta$  toward the dominant shape of  $q_t$ . In  $u$ -coordinates the residual target is

$$h_t^\#(u) = \frac{q_t(T_t^{-1}(u))}{\rho_t(T_t^{-1}(u))} \eta(u), \quad (5.14)$$

where  $\rho_t$  is the density represented by the map. If  $\rho_t$  captures the main location, scale, and correlation structure, the ratio

$$\omega_t(u) = \frac{q_t(T_t^{-1}(u))}{\rho_t(T_t^{-1}(u))} \quad (5.15)$$

is flatter than  $q_t$ , and the residual TT is asked to learn a correction rather than the whole posterior geometry. That is the mathematical reason preconditioning can reduce rank pressure.

The practical hypothesis is therefore

$$\max_k R_k \leq R_{\max} \quad \text{and} \quad \frac{\|Ag - y\|_W}{\|y\|_W} \leq \varepsilon_{\text{fit}} \quad \text{with stable condition numbers.} \quad (5.16)$$

The rank assumption is not a theorem for arbitrary nonlinear posteriors. It is a structural and empirical claim checked by ranks, residuals, conditioning, defensive mass, bridge ratios, and finite-difference consistency.

## 6 Coordinate Systems, Conditional Maps, And Preconditioning

The method uses several coordinate systems because different operations are easiest in different spaces. Let

$$r = (x_t, \theta, x_{t-1}) \quad (6.1)$$

denote physical filtering coordinates. A domain map sends a reference cube coordinate  $z \in [-1, 1]^D$  to physical coordinates:

$$r = \Psi_t(z), \quad J_{\Psi,t}(z) = |\det \nabla \Psi_t(z)|. \quad (6.2)$$

The target seen by the TT in reference coordinates is

$$\tilde{q}_t(z) = q_t(\Psi_t(z)) J_{\Psi,t}(z). \quad (6.3)$$

Thus  $z$  is the coordinate in which basis functions, fitting points, and mass matrices are usually declared.

Preconditioning introduces another coordinate  $u$ . Let  $T_t$  be an invertible map from physical coordinates to a preconditioned coordinate:

$$u = T_t(r), \quad r = T_t^{-1}(u). \quad (6.4)$$

If  $\eta(u)$  is the reference density used in  $u$ -space, then the pushforward/pullback identities give

$$q_t(r) dr = q_t(T_t^{-1}(u)) |\det \nabla T_t^{-1}(u)| du. \quad (6.5)$$

The preconditioned residual in (5.14) is a density in  $u$ -coordinates. A second domain map may still send a cube coordinate  $z$  to  $u$ :

$$u = \Psi_t^\sharp(z), \quad \hat{q}_t^\sharp(z) = h_t^\sharp(\Psi_t^\sharp(z)) |\det \nabla \Psi_t^\sharp(z)|. \quad (6.6)$$

The retained coordinate  $z_t$  is not another global transform. It is the part of the reference coordinate retained after marginalizing the previous state:

$$z = (z_t, z_{t-1}), \quad a_t(z_t) = \int \hat{q}_t(z_t, z_{t-1}) dz_{t-1}, \quad (6.7)$$

and

$$\hat{p}_t(z_t) = \frac{a_t(z_t)}{\hat{Z}_t}. \quad (6.8)$$

If the next time step evaluates the previous filter at a physical point  $x_t$ , the implementation first maps that point to the retained reference coordinate:

$$z_t = \Psi_{t,\text{ret}}^{-1}(x_t), \quad \hat{p}_t(x_t) = \hat{p}_t(z_t) |\det \nabla \Psi_{t,\text{ret}}^{-1}(x_t)|. \quad (6.9)$$

| Symbol | Space                                | Main density identity                                    | What is stored                                     |
|--------|--------------------------------------|----------------------------------------------------------|----------------------------------------------------|
| $r$    | physical<br>state/parameter<br>space | $q_t(r) \, dr$                                           | model evaluator and<br>physical domains            |
| $z$    | reference cube                       | $\tilde{q}_t(z)$<br>$q_t(\Psi_t(z))J_{\Psi,t}(z)$        | = basis values, points,<br>TT cores, mass matrices |
| $u$    | preconditioned<br>space              | $q_t(T_t^{-1}(u)) \det \nabla T_t^{-1}(u) $              | preconditioner,<br>bridge density, log<br>Jacobian |
| $z_t$  | retained reference<br>coordinates    | $a_t(z_t)$<br>$\int \hat{q}_t(z_t, z_{t-1}) \, dz_{t-1}$ | = retained numerators/<br>filter and derivative    |

Most implementation mistakes in this method are coordinate mistakes: using a physical density in reference coordinates without the Jacobian, applying a KR map in  $u$ -space to a point still in  $r$ -space, or carrying a retained filter in  $z_t$  while evaluating it as if it were already a density in  $x_t$ .

## 6.1 One Observation Step Through All Coordinate Systems

The symbols  $r, z, u, z_t$  become easier to track when one observation update is written as a single chain. Begin in physical coordinates

$$r = (x_t, \theta, x_{t-1}), \quad q_t(r) = \hat{p}_{t-1}(x_{t-1}, \theta) f_\theta(x_t \mid x_{t-1}) g_\theta(y_t \mid x_t). \quad (6.10)$$

This formula answers the physical inference question: after observing  $y_t$ , which triples  $(x_t, \theta, x_{t-1})$  are plausible?

The TT does not fit  $q_t$  directly on an infinite physical domain. A domain map

$$r = \Psi_t(z), \quad z \in [-1, 1]^D, \quad J_{\Psi,t}(z) = |\det \nabla \Psi_t(z)| \quad (6.11)$$

turns the physical density into a reference-coordinate density

$$\tilde{q}_t(z) = q_t(\Psi_t(z)) J_{\Psi,t}(z). \quad (6.12)$$

The equality of masses is

$$\int_{\Psi_t(A)} q_t(r) \, dr = \int_A \tilde{q}_t(z) \, dz. \quad (6.13)$$

Thus the Jacobian is not a numerical decoration; it is what keeps probability mass unchanged after changing coordinates.

The squared-TT step fits the square root in reference coordinates:

$$\sqrt{\tilde{q}_t(z)} \approx e^{-c_t/2} \phi_t(z), \quad (6.14)$$

and then defines the nonnegative approximation

$$\hat{q}_t(z) = e^{-c_t} \phi_t(z)^2 + \tau_t \lambda_t(z). \quad (6.15)$$

The carried filter is not the full joint object. If

$$z = (z_t, z_{t-1}), \quad (6.16)$$

then the retained numerator and normalizer are

$$a_t(z_t) = \int \widehat{q}_t(z_t, z_{t-1}) dz_{t-1}, \quad \widehat{Z}_t = \int a_t(z_t) dz_t, \quad (6.17)$$

and the retained reference filter is

$$\widehat{p}_t^{\text{ref}}(z_t) = \frac{a_t(z_t)}{\widehat{Z}_t}. \quad (6.18)$$

If a later formula needs the filter as a physical density in  $x_t, \theta$ , write

$$(x_t, \theta) = \Psi_{t,\text{ret}}(z_t), \quad \widehat{p}_t^{\text{phys}}(x_t, \theta) = \widehat{p}_t^{\text{ref}}(\Psi_{t,\text{ret}}^{-1}(x_t, \theta)) |\det \nabla \Psi_{t,\text{ret}}^{-1}(x_t, \theta)|. \quad (6.19)$$

A useful bookkeeping rule is

$$\begin{aligned} \widetilde{q}_t(z), a_t(z_t), \widehat{Z}_t, \widehat{p}_t^{\text{ref}}(z_t) & \text{ are reference-coordinate objects integrated with } dz, \\ \widehat{p}_t^{\text{phys}}(r_t) = \widehat{p}_t^{\text{ref}}(\Psi_{t,\text{ret}}^{-1}(r_t)) |\det \nabla \Psi_{t,\text{ret}}^{-1}(r_t)| & \text{ is the corresponding physical density.} \end{aligned} \quad (6.20)$$

Preconditioning inserts one additional coordinate  $u$ . A bridge density  $\rho_t(r)$  and map  $T_t : r \mapsto u$  are chosen so that the dominant shape of  $q_t$  is easier in  $u$ -space. The residual target is

$$q_t^\sharp(u) = \frac{q_t(T_t^{-1}(u))}{\widehat{\rho}_t(T_t^{-1}(u))} \eta(u). \quad (6.21)$$

This is still the same observation step. The difference is that the TT now fits the residual density  $q_t^\sharp$  rather than the raw density  $q_t$ . The pullback relation after fitting is

$$\widehat{q}_t(r) = \frac{\widehat{\nu}_t^\sharp(T_t(r))}{\eta(T_t(r))} \widehat{\rho}_t(r). \quad (6.22)$$

The next observation uses the retained filter by querying it at the previous state coordinate of the next fitting point. If a next-step fitting point is

$$z_{t+1}^{(j)} = (z_{t+1}^{(j)}, z_t^{(j)}), \quad (6.23)$$

then the target value at that point uses

$$\widehat{p}_t^{\text{ref}}(z_t^{(j)}) \quad (6.24)$$

inside

$$\widetilde{q}_{t+1}(z_{t+1}^{(j)}) = \widehat{p}_t^{\text{ref}}(z_t^{(j)}) f_\theta(x_{t+1}^{(j)} | x_t^{(j)}) g_\theta(y_{t+1} | x_{t+1}^{(j)}) J_{t+1}(z_{t+1}^{(j)}). \quad (6.25)$$

This is the full coordinate story in one line of work:

$$q_t(r) \rightarrow \widetilde{q}_t(z) \rightarrow \widehat{q}_t(z) \rightarrow \widehat{p}_t^{\text{ref}}(z_t) \rightarrow \widetilde{q}_{t+1}(z_{t+1}, z_t). \quad (6.26)$$

The preconditioned route adds

$$q_t(r) \rightarrow q_t^\sharp(u) \rightarrow \widehat{\nu}_t^\sharp(u) \rightarrow \widehat{q}_t(r), \quad (6.27)$$

but it does not change the filtering task. It changes the coordinate system in which the hard part of the density is approximated.

## 6.2 Conditional Densities And KR Maps From Marginal Ratios

**Mathematical question.** The question answered here is: how do marginal densities become a transport map that can sample from the approximate density?

For a normalized density  $\widehat{p}(r_1, \dots, r_D)$ , the chain rule gives

$$\widehat{p}(r) = \widehat{p}(r_1) \prod_{k=2}^D \widehat{p}(r_k \mid r_{<k}), \quad \widehat{p}(r_k \mid r_{<k}) = \frac{\widehat{p}(r_{\leq k})}{\widehat{p}(r_{<k})}. \quad (6.28)$$

The squared-TT marginal formula (7.60) supplies both numerator and denominator. Thus each conditional density is computable from TT contractions.

Define conditional distribution functions

$$F_1(r_1) = \int_{-\infty}^{r_1} \widehat{p}(s_1) \, ds_1, \quad (6.29)$$

and, for  $k \geq 2$ ,

$$F_k(r_k \mid r_{<k}) = \int_{-\infty}^{r_k} \widehat{p}(s_k \mid r_{<k}) \, ds_k. \quad (6.30)$$

The lower triangular Knothe–Rosenblatt map is

$$F(r) = (F_1(r_1), F_2(r_2 \mid r_1), \dots, F_D(r_D \mid r_{<D})). \quad (6.31)$$

If  $R \sim \widehat{p}$ , then  $F(R)$  is uniform on  $[0, 1]^D$ . Conversely, if  $\Xi \sim \text{Unif}([0, 1]^D)$ , then  $F^{-1}(\Xi) \sim \widehat{p}$ .

The proof is the triangular Jacobian calculation. The Jacobian of  $F$  is lower triangular, and

$$\frac{\partial F_k(r_k \mid r_{<k})}{\partial r_k} = \widehat{p}(r_k \mid r_{<k}).$$

Therefore

$$|\det \nabla F(r)| = \prod_{k=1}^D \widehat{p}(r_k \mid r_{<k}) = \widehat{p}(r). \quad (6.32)$$

The pullback of the uniform density by  $F$  is exactly  $|\det \nabla F(r)| = \widehat{p}(r)$ .

### Why This Transport Is The Right Device

The KR map is useful because it turns the question “how do we sample from a high-dimensional density?” into a sequence of one-dimensional questions. Start with the ordinary one-dimensional probability integral transform:

$$U = F_X(X) = \int_{-\infty}^X p_X(s) \, ds, \quad X \sim p_X, \quad (6.33)$$

which implies  $U \sim \text{Unif}(0, 1)$ . Conversely,

$$X = F_X^{-1}(U), \quad U \sim \text{Unif}(0, 1), \quad (6.34)$$

has density  $p_X$ . The multivariate construction repeats this idea after conditioning on the coordinates that have already been chosen:

$$\begin{aligned} u_1 &= F_1(r_1), \\ u_2 &= F_2(r_2 \mid r_1), \\ u_3 &= F_3(r_3 \mid r_1, r_2), \\ &\vdots \\ u_D &= F_D(r_D \mid r_1, \dots, r_{D-1}). \end{aligned} \quad (6.35)$$

The inverse construction is equally sequential:

$$\begin{aligned}
r_1 &= F_1^{-1}(u_1), \\
r_2 &= F_2^{-1}(u_2 \mid r_1), \\
r_3 &= F_3^{-1}(u_3 \mid r_1, r_2), \\
&\vdots \\
r_D &= F_D^{-1}(u_D \mid r_1, \dots, r_{D-1}).
\end{aligned} \tag{6.36}$$

Thus a product draw  $u \sim \text{Unif}([0, 1]^D)$  becomes a dependent draw  $r \sim \hat{p}$ . This is the reason the map is a transport map: it transports the simple product measure to the fitted dependent density.

The triangular form is the essential feature. Its Jacobian matrix has zeros above the diagonal,

$$\nabla F(r) = \begin{bmatrix} \partial_{r_1} F_1 & 0 & \cdots & 0 \\ \partial_{r_1} F_2 & \partial_{r_2} F_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \partial_{r_1} F_D & \partial_{r_2} F_D & \cdots & \partial_{r_D} F_D \end{bmatrix}, \tag{6.37}$$

so the determinant is just the product of one-dimensional conditional densities:

$$\det \nabla F(r) = \prod_{k=1}^D \partial_{r_k} F_k(r_k \mid r_{<k}) = \prod_{k=1}^D \hat{p}(r_k \mid r_{<k}) = \hat{p}(r). \tag{6.38}$$

For the fitted squared TT, the hard part is not this determinant identity; it is obtaining the conditional densities in the middle product. The squared-TT mass contractions provide exactly those marginal ratios:

$$\hat{p}(r_k \mid r_{<k}) = \frac{\hat{p}(r_{\leq k})}{\hat{p}(r_{<k})} = \frac{\text{TT contraction retaining } r_{\leq k}}{\text{TT contraction retaining } r_{<k}}. \tag{6.39}$$

This is why conditional/KR maps are not an ornament added after fitting. They are the natural way to convert the fitted nonnegative density into a sampler, a particle proposal, or a backward smoother.

## Worked Two-Dimensional KR Example

For  $D = 2$ , write the normalized approximate density as

$$\hat{p}(r_1, r_2) \geq 0, \quad \iint \hat{p}(r_1, r_2) \, dr_1 \, dr_2 = 1. \tag{6.40}$$

The first marginal is

$$\hat{p}_1(r_1) = \int \hat{p}(r_1, s_2) \, ds_2, \tag{6.41}$$

and the second conditional density is

$$\hat{p}_{2|1}(r_2 \mid r_1) = \frac{\hat{p}(r_1, r_2)}{\hat{p}_1(r_1)} \quad \text{when } \hat{p}_1(r_1) > 0. \tag{6.42}$$

The two conditional distribution functions are

$$F_1(r_1) = \int_{a_1}^{r_1} \hat{p}_1(s_1) \, ds_1, \tag{6.43}$$

and

$$F_2(r_2 \mid r_1) = \int_{a_2}^{r_2} \widehat{p}_{2|1}(s_2 \mid r_1) \, ds_2. \quad (6.44)$$

The lower triangular map is

$$F(r_1, r_2) = (F_1(r_1), F_2(r_2 \mid r_1)). \quad (6.45)$$

Its Jacobian matrix is

$$\nabla F(r_1, r_2) = \begin{bmatrix} \partial_{r_1} F_1(r_1) & 0 \\ \partial_{r_1} F_2(r_2 \mid r_1) & \partial_{r_2} F_2(r_2 \mid r_1) \end{bmatrix}. \quad (6.46)$$

Because the matrix is triangular,

$$\det \nabla F = \widehat{p}_1(r_1) \widehat{p}_{2|1}(r_2 \mid r_1) = \widehat{p}(r_1, r_2). \quad (6.47)$$

Now connect this analytic conditional density to the object stored by a squared-TT implementation. In two dimensions, write

$$\widehat{p}(r_1, r_2) = \frac{\phi(r_1, r_2)^2 + \tau \lambda(r_1, r_2)}{\widehat{Z}}. \quad (6.48)$$

Suppose the square-root TT has two cores,

$$\phi(r_1, r_2) = H_1(r_1) H_2(r_2), \quad H_1(r_1) \in \mathbb{R}^{1 \times R_1}, \quad H_2(r_2) \in \mathbb{R}^{R_1 \times 1}. \quad (6.49)$$

The square-mass object for the second coordinate is

$$G_2 = \int H_2(s) H_2(s)^\top \, ds \in \mathbb{R}^{R_1 \times R_1}. \quad (6.50)$$

Therefore the unnormalized first marginal is

$$n_1(r_1) = H_1(r_1) G_2 H_1(r_1)^\top + \tau \lambda_1(r_1), \quad \lambda_1(r_1) = \int \lambda(r_1, s) \, ds. \quad (6.51)$$

Dividing by  $\widehat{Z}$  gives  $\widehat{p}_1(r_1) = n_1(r_1)/\widehat{Z}$ . The conditional density evaluator used by the CDF machinery is

$$d_2(s \mid r_1) = \widehat{p}_{2|1}(s \mid r_1) = \frac{\phi(r_1, s)^2 + \tau \lambda(r_1, s)}{n_1(r_1)}. \quad (6.52)$$

The global normalizer cancels. This is the exact two-dimensional bridge from TT square-mass contraction to the operational conditional density in (6.62)–(6.64).

To sample from  $\widehat{p}$ , draw  $u_1, u_2 \sim \text{Unif}(0, 1)$  and solve

$$F_1(r_1) = u_1, \quad F_2(r_2 \mid r_1) = u_2. \quad (6.53)$$

Endpoint and monotonicity diagnostics are

$$F_1(a_1) = 0, \quad F_1(b_1) = 1, \quad F_2(a_2 \mid r_1) = 0, \quad F_2(b_2 \mid r_1) = 1, \quad (6.54)$$

and

$$\partial_{r_1} F_1(r_1) \geq 0, \quad \partial_{r_2} F_2(r_2 \mid r_1) \geq 0. \quad (6.55)$$

If these fail, the map is not a valid sampler until the conditional CDF representation is repaired.

**Mathematical question.** Remark 3 answers: what changes if the conditional map is built from the right end rather than from the left end?

The paper also states the reverse-order construction. If instead we compute right-to-left marginals

$$\widehat{p}(r_{\geq k}) = \int \widehat{p}(r) \, dr_{<k}, \quad (6.56)$$

then the conditional densities are

$$\widehat{p}(r_k \mid r_{>k}) = \frac{\widehat{p}(r_{\geq k})}{\widehat{p}(r_{>k})}, \quad (6.57)$$

and the upper triangular KR map is

$$F^u(r) = (F_1^u(r_1 \mid r_{>1}), \dots, F_{D-1}^u(r_{D-1} \mid r_D), F_D^u(r_D)). \quad (6.58)$$

This is not a different density approximation. It is the same density integrated in the opposite direction. Zhao and Cui [4] use the lower map for backward smoothing and the upper map for forward particle proposals.

**Implementation meaning.** Inputs are the same squared-TT density and the choice of reverse integration direction. Outputs are right-to-left mass contractions and upper-triangular conditional CDFs. Quantities to retain must record the direction, because using lower-map contractions inside an upper-map proposal silently changes the proposal density. The failure check is endpoint normalization of every reverse conditional CDF.

The computational costs in the paper separate preprocessing from per-sample map evaluation. Building all conditional densities from the squared-TT mass recursions costs

$$O(DpR^3). \quad (6.59)$$

For  $N$  samples, evaluating conditional densities along a triangular map costs approximately

$$O(NDpR^2), \quad (6.60)$$

and constructing/evaluating one-dimensional distribution functions adds basis and root-finding work. If  $c_{\text{root}}$  is the fixed number of root-finding iterations, inverse-map evaluation scales as

$$O(DpR^3 + NDpR^2 + NDp(\log p + R) + NDp c_{\text{root}}). \quad (6.61)$$

The formula is less important than the decomposition: build the conditional objects once, then pay a per-sample triangular evaluation cost.

**Implementation meaning. 1-KR-ratios.** Inputs are marginal density evaluators from Proposition 2. Outputs are conditional CDF evaluators and inverse-CDF samplers. Quantities to retain includes integration direction, mass matrices, one-dimensional CDF coefficients, and root-finding tolerances. Diagnostics are monotonicity of each CDF, endpoint values near 0 and 1, and inverse residuals.

## Operational KR Construction Details

For each conditional coordinate  $k$ , store a conditional density evaluator

$$d_k(s \mid r_{<k}) = \widehat{p}(r_k = s \mid r_{<k}). \quad (6.62)$$

Represent the conditional CDF by quadrature on the coordinate interval  $[a_k, b_k]$ :

$$F_k(s \mid r_{<k}) = \int_{a_k}^s d_k(u \mid r_{<k}) \, du. \quad (6.63)$$



A deterministic baseline is Gauss–Legendre subinterval quadrature. For nodes  $\xi_\nu$  and weights  $\omega_\nu$  on  $[a_k, s]$ ,

$$F_k(s \mid r_{<k}) \approx \sum_{\nu=1}^{N_q} \omega_\nu d_k(\xi_\nu \mid r_{<k}). \quad (6.64)$$

After evaluation, clip only the reported probability, not the density:

$$F_k^{\text{clip}}(s \mid r_{<k}) = \min\{1, \max\{0, F_k(s \mid r_{<k})\}\}. \quad (6.65)$$

The branch records whether clipping occurred. Clipping is a diagnostic, not a mathematical cure.

The inverse problem is

$$F_k(s \mid r_{<k}) - u = 0. \quad (6.66)$$

The baseline inverse rule is bisection:

$$[a^{(0)}, b^{(0)}] = [a_k, b_k], \quad m^{(n)} = \frac{a^{(n)} + b^{(n)}}{2}. \quad (6.67)$$

If  $F_k(m^{(n)} \mid r_{<k}) < u$ , set

$$a^{(n+1)} = m^{(n)}, \quad b^{(n+1)} = b^{(n)}; \quad (6.68)$$

otherwise set

$$a^{(n+1)} = a^{(n)}, \quad b^{(n+1)} = m^{(n)}. \quad (6.69)$$

Stop when

$$b^{(n)} - a^{(n)} \leq \varepsilon_{\text{root}} \quad \text{or} \quad |F_k(m^{(n)} \mid r_{<k}) - u| \leq \varepsilon_{\text{cdf}}. \quad (6.70)$$

The operational diagnostic vector is

$$\mathcal{D}_{\text{KR}} = \{F_k(a_k), F_k(b_k), \min \Delta F_k, \max |F_k(F_k^{-1}(u)) - u|, \text{clip count}\}. \quad (6.71)$$

If  $F_k(b_k)$  differs from 1, if  $\min \Delta F_k < -\varepsilon_{\text{mono}}$ , or if inverse residuals exceed tolerance, the conditional map is rejected for that branch.

**Local assumptions for KR exactness.** This note uses the triangular CDF identity under positive conditional densities, well-defined marginal ratios, and invertible one-dimensional conditional CDFs. If a conditional CDF is flat, discontinuous, numerically non-monotone, or unsupported by the reference domain, the KR sampler is not valid until that failure is fixed.

### 6.3 Forward Conditional Map And Particle-Filter Correction

**Mathematical question.** These equations answer: how does the squared-TT approximation become a particle proposal, and how is approximation bias corrected?

The same  $\hat{q}_t(x_t, \alpha, x_{t-1})$  defines a conditional density for the new state given the old state and parameter:

$$\hat{p}_t(x_t \mid \alpha, x_{t-1}, y_{1:t}) = \frac{\hat{q}_t(x_t, \alpha, x_{t-1})}{\int \hat{q}_t(s, \alpha, x_{t-1}) \, ds}. \quad (6.72)$$

Constructing the corresponding upper triangular conditional KR map gives a proposal sampler:

$$\hat{X}_t^{(i)} = (F_{t,t}^u)^{-1} \left( \Xi_t^{(i)} \mid \hat{\alpha}^{(i)}, \hat{x}_{t-1}^{(i)} \right), \quad \Xi_t^{(i)} \sim \text{Unif}([0, 1]^{m_x}). \quad (6.73)$$

This is Zhao and Cui [4], Eq. (21) in the fixed-branch notation used here.

The proposal density for the full path after sampling is

$$\hat{p}_t(x_t \mid \alpha, x_{t-1}, y_{1:t})p(\alpha, x_{0:t-1} \mid y_{1:t-1}), \quad (6.74)$$

which corresponds to Zhao and Cui [4], Eq. (22). The exact posterior recursion is

$$p(\alpha, x_{0:t} \mid y_{1:t}) \propto p(\alpha, x_{0:t-1} \mid y_{1:t-1})f_\alpha(x_t \mid x_{t-1})g_\alpha(y_t \mid x_t).$$

Dividing exact target by proposal gives the unnormalized correction factor

$$\omega_f(\alpha, x_{t-1:t}) = \frac{f_\alpha(x_t \mid x_{t-1})g_\alpha(y_t \mid x_t)}{\hat{p}_t(x_t \mid \alpha, x_{t-1}, y_{1:t})}. \quad (6.75)$$

This is Zhao and Cui [4], Eq. (23). The correction is conceptually important: the TT proposal can be biased, but if its conditional density is evaluable and covers the target, importance weights can correct expectations.

With incoming normalized weights  $W_{t-1}^{(i)}$ , the particle-filter weight update is

$$\widetilde{W}_t^{(i)} = W_{t-1}^{(i)}\omega_f(\hat{\alpha}^{(i)}, \hat{x}_{t-1:t}^{(i)}), \quad (6.76)$$

followed by

$$W_t^{(i)} = \frac{\widetilde{W}_t^{(i)}}{\sum_{j=1}^N \widetilde{W}_t^{(j)}}. \quad (6.77)$$

Algorithm 3 is therefore an ordinary importance sampler whose proposal is defined by the upper conditional KR map of the TT approximation. The implementation stores the conditional-map object, the sampled  $x_t^{(i)}$ , the proposal density evaluator, and the normalized weights. A standard failure diagnostic is effective sample size,

$$\text{ESS}_t = \frac{1}{\sum_{i=1}^N (W_t^{(i)})^2}. \quad (6.78)$$

Low ESS means that the TT proposal is not close enough to the exact recursive posterior for stable correction.

**Implementation meaning.** Inputs: weighted particles at time  $t - 1$ , the upper conditional map, model density evaluators, and proposal density evaluator. Outputs: new particles and normalized weights. Quantities to retain: particles, weights, ESS, and optional resampling history. Operation: inverse triangular sampling plus importance-weight update. Diagnostics: ESS, infinite/zero proposal density, root-finding failure, and weight underflow.

## 6.4 Backward Conditional Map, Path Estimation, And Smoothing

**Mathematical question.** These equations answer: how can the same sequence of local TT approximations produce full paths and smoothed old states after all data are observed?

The lower conditional map goes the other direction:

$$\hat{p}_t(x_{t-1} \mid x_t, \alpha, y_{1:t}) = \frac{\hat{q}_t(x_t, \alpha, x_{t-1})}{\int \hat{q}_t(x_t, \alpha, s) \mathrm{d}s}. \quad (6.79)$$

The corresponding inverse conditional KR map samples

$$\hat{X}_{t-1}^{(i)} = (F_{t,t-1}^l)^{-1} \left( \Xi_{t-1}^{(i)} \mid \hat{X}_t^{(i)}, \hat{\alpha}^{(i)} \right). \quad (6.80)$$

This is Zhao and Cui [4], Eq. (24).

Starting at final time  $T$ , draw

$$(\hat{X}_T, \hat{\alpha}, \hat{X}_{T-1}) \sim \hat{p}_T(x_T, \alpha, x_{T-1} \mid y_{1:T}),$$

then move backward using (6.80). The resulting approximate path density factors as Zhao and Cui [4], Eq. (25):

$$\hat{p}(\alpha, x_{0:T} \mid y_{1:T}) = \hat{p}_T(x_T, \alpha, x_{T-1} \mid y_{1:T}) \prod_{t=1}^{T-1} \hat{p}_t(x_{t-1} \mid x_t, \alpha, y_{1:t}). \quad (6.81)$$

The exact smoothing posterior has the same Markov factorization form with exact conditionals:

$$p(\alpha, x_{0:T} \mid y_{1:T}) = p(x_T, \alpha, x_{T-1} \mid y_{1:T}) \prod_{t=1}^{T-1} p(x_{t-1} \mid x_t, \alpha, y_{1:t}). \quad (6.82)$$

The approximation can therefore be corrected by an importance ratio. The unnormalized backward weight, Zhao and Cui [4], Eq. (26), is

$$\omega_b(\alpha, x_{0:T}) = \frac{p(\alpha) p_\alpha(x_0) \prod_{t=1}^T f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t)}{\hat{p}_T(x_T, \alpha, x_{T-1} \mid y_{1:T}) \prod_{t=1}^{T-1} \hat{p}_t(x_{t-1} \mid x_t, \alpha, y_{1:t})}. \quad (6.83)$$

The numerator is the exact unnormalized path posterior. The denominator is the proposal density generated by the backward TT maps.

Algorithm 4 normalizes the backward weights in the same way:

$$W^{(i)} = \frac{\omega_b(\hat{\alpha}^{(i)}, \hat{x}_{0:T}^{(i)})}{\sum_{j=1}^N \omega_b(\hat{\alpha}^{(j)}, \hat{x}_{0:T}^{(j)})}. \quad (6.84)$$

The paper emphasizes a Markov identity used by the backward recursion:

$$p(x_{t-1} \mid \alpha, x_t, y_{1:t}) = p(x_{t-1} \mid \alpha, x_t, y_{1:T}), \quad t < T. \quad (6.85)$$

The reason is conditional independence. Once  $(\alpha, x_t)$  are known, the future observations  $y_{t+1:T}$  depend on the past state  $x_{t-1}$  only through  $x_t$ . This is why a conditional object built at time  $t$  can be used inside a final-time smoothing path.

Zhao and Cui [4] also give the computational reason for the variable order  $(x_t, \alpha, x_{t-1})$ . Marginalizing a squared TT from an end costs

$$O(pR^3) \text{ per coordinate}, \quad (6.86)$$

because it updates one boundary mass matrix. Marginalizing a coordinate in the middle can cost

$$O(pR^6) \text{ per coordinate}, \quad (6.87)$$

because left and right rank couplings must be paired. The order  $(x_t, \alpha, x_{t-1})$  lets Algorithm 2 integrate  $x_{t-1}$  from the right end and lets the smoothing map reuse the same direction. The upper map for particle filtering goes in the opposite direction and therefore incurs an additional  $O(mpR^3)$ -type preprocessing cost.

Implementation meaning. Inputs: final-time approximation, lower conditional maps for all times, model density evaluators, and reference uniform samples. Outputs: path samples and normalized smoothing weights. Quantities to retain: map sequence, samples, weights, and ESS. Operation: final joint sample, then backward inverse-map sampling. Diagnostics: backward proposal support, weight variance, root residuals, and mismatch between source-time conditionals and final-time path use.

## 6.5 Preconditioning

**Mathematical question.** These equations answer: how can a hard target density be transformed into a less concentrated residual before fitting a TT?

The direct target  $q_t(x_t, \alpha, x_{t-1})$  can be sharply concentrated. A TT may then need high rank. Zhao and Cui [4], Section 5 introduces a change of variables that makes the function closer to a product reference density before fitting.

Let

$$r = (x_t, \alpha, x_{t-1}), \quad u = T_t(r),$$

and let  $\eta(u) = \eta_1(u_1) \cdots \eta_D(u_D)$  be a product reference density. Choose a bridging density  $\rho_t(r)$  that is easier to approximate than  $q_t(r)$ , and construct  $T_t$  so that the pushforward of  $\rho_t$  is close to  $\eta$ :

$$(T_t)_\# \rho_t(u) = \rho_t(T_t^{-1}(u)) |\det \nabla T_t^{-1}(u)| \propto \eta(u). \quad (6.88)$$

This is Zhao and Cui [4], Eq. (30).

The pushforward of  $q_t$  is

$$(T_t)_\# q_t(u) = q_t(T_t^{-1}(u)) |\det \nabla T_t^{-1}(u)|.$$

Using (6.88), the Jacobian factor is proportional to  $\eta(u)/\rho_t(T_t^{-1}(u))$ , so the function to fit is

$$q_t^\#(u) = \frac{q_t(T_t^{-1}(u))}{\rho_t(T_t^{-1}(u))} \eta(u). \quad (6.89)$$

This is Zhao and Cui [4], Eq. (31). If  $\rho_t$  captures the main shape of  $q_t$ , then  $q_t/\rho_t$  is flatter than  $q_t$ , and TT ranks may be smaller.

In density language, the preconditioner should move the hard geometry into the map  $T_t$ . The residual  $q_t^\#$  then asks the TT to learn only what the bridge missed. The implementation failure mode is division by a bridge that is too small:  $\rho_t(T_t^{-1}(u))$  or  $\hat{\rho}_t(T_t^{-1}(u))$  must not vanish on target-relevant points.

### Before/After Flattening Calculation

The preconditioning claim can be written as a before/after ratio. Before preconditioning, the TT sees the target

$$q(r). \quad (6.90)$$

Choose a positive bridge  $\rho(r)$  and write

$$q(r) = \rho(r) m(r), \quad m(r) = \frac{q(r)}{\rho(r)}. \quad (6.91)$$

If  $\rho$  captures the main concentration, then the multiplier  $m$  varies more slowly than  $q$ . In logarithmic form,

$$\log q(r) = \log \rho(r) + \log m(r). \quad (6.92)$$

The bridge map  $T$  moves  $\rho$  to a product reference  $\eta$ :

$$\rho(T^{-1}(u)) |\det \nabla T^{-1}(u)| = Z_\rho \eta(u). \quad (6.93)$$

Therefore the transformed target is

$$q^\#(u) = q(T^{-1}(u)) |\det \nabla T^{-1}(u)| = Z_\rho m(T^{-1}(u)) \eta(u). \quad (6.94)$$

The TT now fits the residual multiplier times a product reference, not the full concentrated target:

$$\sqrt{q^\sharp(u)} = \sqrt{Z_\rho} \sqrt{m(T^{-1}(u))} \sqrt{\eta(u)}. \quad (6.95)$$

The rank-pressure statement is qualitative but precise:

if  $m(T^{-1}(u))$  has weaker cross-coordinate coupling than  $q(r)$ , then smaller TT ranks may suffice. (6.96)

The failure case is equally explicit. If

$$\inf_{r \in \Omega_{\text{fit}}} \rho(r) \approx 0 \quad \text{while} \quad q(r) > 0, \quad (6.97)$$

then  $m = q/\rho$  can explode and the residual is harder, not easier. The bridge-support diagnostic is therefore

$$\kappa_{\text{bridge}} = \max_{r \in Z_{\text{fit}}} \frac{q(r)}{\rho(r) + \varepsilon_{\text{bridge}}}, \quad \kappa_{\text{bridge}} < \kappa_{\text{max}}. \quad (6.98)$$

For the running example (2.6)–(2.15), the unpreconditioned target is

$$q_t(r) = \hat{p}_{t-1}(x_{t-1}) \varphi(x_t; \beta x_{t-1}, \sigma^2) \varphi(y_t; \sin x_t, \tau_y^2), \quad r = (x_t, x_{t-1}). \quad (6.99)$$

A Gaussian bridge may keep a local mean and covariance for this two-dimensional target while missing the sinusoidal likelihood curvature. The residual after the bridge is therefore

$$m(T^{-1}(u)) = \frac{\hat{p}_{t-1}(x_{t-1}(u)) \varphi(x_t(u); \beta x_{t-1}(u), \sigma^2) \varphi(y_t; \sin x_t(u), \tau_y^2)}{\rho(x_t(u), x_{t-1}(u))}. \quad (6.100)$$

This is (6.94) with the concrete factors from the running example substituted. The bridge helps only if the remaining multiplier in (6.100) is flatter and less coupled than the original target.

Fit a square-root TT in  $u$ -space:

$$\sqrt{q_t^\sharp(u)} \approx \phi_t^\sharp(u),$$

and define

$$\hat{\nu}_t^\sharp(u) = \phi_t^\sharp(u)^2 + \tau_t^\sharp \eta(u). \quad (6.101)$$

This is Zhao and Cui [4], Eq. (32). Pulling the approximation back to physical coordinates gives

$$\hat{\pi}_t(r) = \frac{\hat{\nu}_t^\sharp(T_t(r))}{\eta(T_t(r))} \rho_t(r). \quad (6.102)$$

This is Zhao and Cui [4], Eq. (33). The normalizer is unchanged by the change of variables:

$$\int \hat{\pi}_t(r) \, dr = \int \hat{\nu}_t^\sharp(u) \, du. \quad (6.103)$$

## Why Preconditioning And KR Maps Belong Together

The preconditioner and the KR map solve complementary problems. The KR map can sample from a fitted density once the density has been represented well. The preconditioner tries to make the density easier to represent before the TT fit is performed. In equations, let the exact target be decomposed as

$$q(r) = \rho(r) m(r), \quad m(r) = q(r)/\rho(r), \quad (6.104)$$

and choose  $T$  so that  $\rho$  is mapped to a product reference  $\eta$ :

$$\rho(r) dr = \eta(u) du, \quad u = T(r). \quad (6.105)$$

Substituting  $r = T^{-1}(u)$  gives

$$q(r) dr = m(T^{-1}(u))\eta(u) du. \quad (6.106)$$

The TT therefore sees the residual multiplier  $m(T^{-1}(u))$ , not the entire original density geometry. If  $\rho$  captures the main posterior ellipsoid, ridge, or concentration region, then

$$\text{osc}\{\log m(T^{-1}(u)) : u \in \Omega_u\} < \text{osc}\{\log q(r) : r \in \Omega_r\} \quad (6.107)$$

on the fitting domain, where  $\text{osc}(f) = \sup f - \inf f$ . This is the concrete mathematical sense in which the residual can be flatter.

After the residual has been fitted, a KR map can be built in the easier  $u$ -coordinates:

$$F^\sharp(u) = (F_1^\sharp(u_1), F_2^\sharp(u_2 | u_1), \dots, F_D^\sharp(u_D | u_{<D})). \quad (6.108)$$

If  $\xi \sim \text{Unif}([0, 1]^D)$ , then the two-step sampler is

$$u = (F^\sharp)^{-1}(\xi), \quad r = T^{-1}(u). \quad (6.109)$$

The density calculation is the product of the two Jacobian identities:

$$\left| \det \nabla F^\sharp(u) \right| \left| \det \nabla T(r) \right| = \widehat{\nu}^\sharp(u) \left| \det \nabla T(r) \right| \approx q(r). \quad (6.110)$$

This is the teach-back version of the method: the preconditioner removes the dominant geometry, the squared TT fits the remaining density, and the KR map turns the fitted density into sequential conditional inversions.

Equivalently, the bridge answers a coarse geometric question and the KR map answers a sampling question:

$$\rho \approx q \implies q^\sharp(u) \approx \eta(u) \times \text{moderate residual}, \quad F^\sharp : q^\sharp \longrightarrow \text{Unif}([0, 1]^D). \quad (6.111)$$

The inverse proposal then has the readable form

$$\xi \sim \text{Unif}([0, 1]^D) \mapsto u = (F^\sharp)^{-1}(\xi) \mapsto r = T^{-1}(u). \quad (6.112)$$

So the bridge captures the coarse posterior geometry, the squared TT captures what the bridge missed, and the KR inverse converts that fitted residual density into a sequential one-dimensional proposal construction.

Implementation meaning. 1-Eq30–Eq33. Inputs are  $q_t$ , bridge  $\rho_t$  or  $\widehat{\rho}_t$ , map  $T_t$ , reference density  $\eta$ , and residual TT  $\widehat{\nu}_t^\sharp$ . Outputs are a physical-coordinate approximate density evaluator  $\widehat{\pi}_t$ , a normalizer, and later marginal/conditional map objects. Quantities to retain: map evaluators, inverse map evaluators, log-Jacobians or density ratios, bridge TT, residual TT, and normalizer. Diagnostics: bridge support ratio, map invertibility, normalizer equality between physical and reference coordinates, and residual rank.

**Mathematical question.** This paragraph answers: what is the simplest concrete preconditioner?

A simple bridge is Gaussian. Estimate a mean and covariance for  $r$ , set  $\rho_t = \mathcal{N}(\mu_t, \Sigma_t)$ , and use an affine Gaussian whitening map. If

$$\Sigma_t = L_t L_t^\top \quad (6.113)$$

is a Cholesky factorization with  $L_t$  lower triangular, then the lower linear KR whitening map is

$$T_t^\ell(r) = L_t^{-1}(r - \mu_t), \quad (6.114)$$

and  $T_t^\ell(R) \sim N(0, I)$  when  $R \sim N(\mu_t, \Sigma_t)$ . The reference density  $\eta$  is then the standard Gaussian product density. The implementation stores  $\mu_t$ ,  $L_t$ , triangular solves for  $L_t^{-1}$ , and the log determinant

$$\log |\det \nabla T_t^\ell| = - \sum_j \log L_t[j, j]. \quad (6.115)$$

An upper triangular linear map is obtained by permuting or reversing the coordinate order before applying the same whitening idea.

Implementation meaning. 2. Inputs: particles or quadrature samples used to estimate  $\mu_t, \Sigma_t$ . Outputs: triangular factor  $L_t$ , whitening map  $T_t^\ell$ , inverse map  $r = \mu_t + L_t u$ , and log determinant. Quantities to retain:  $\mu_t, L_t$  and any covariance regularization. Diagnostics: positive-definiteness of  $\Sigma_t$ , conditioning of  $L_t$ , and whether the Gaussian bridge assigns negligible density to target-relevant samples.

**Mathematical question.** This paragraph answers: how can the bridge retain nonlinear structure from the previous filter, transition, and likelihood?

A nonlinear bridge uses powers of the three factors in  $q_t$ :

$$\rho_t(r) = \hat{p}_{t-1}(x_{t-1}, \alpha)^{\beta_\pi} f_\alpha(x_t | x_{t-1})^{\beta_f} g_\alpha(y_t | x_t)^{\beta_g}, \quad \beta_\pi, \beta_f, \beta_g \in [0, 1]. \quad (6.116)$$

This is Zhao and Cui [4], Eq. (34). A squared-TT approximation of this bridge is

$$\hat{\rho}_t(r) = \phi_{\rho,t}(r)^2 + \tau_{\rho,t} \lambda(r). \quad (6.117)$$

This is Zhao and Cui [4], Eq. (35). The same algebra as (6.102) applies with  $\rho_t$  replaced by  $\hat{\rho}_t$ .

Implementation meaning. 3. Inputs: exponents  $\beta_\pi, \beta_f, \beta_g$ , previous filter evaluator, transition evaluator, likelihood evaluator, and bridge TT controls. Outputs:  $\hat{\rho}_t$  and its KR map. Quantities to retain: exponents, bridge TT cores, defensive mass, and bridge mass contractions. Diagnostics: bridge fit residual, bridge rank, support coverage, and whether the powered factors are finite at fitting points.

## 6.6 The Five Densities In The Preconditioned Step

**Mathematical question.** This subsection expands the compact Section 5 notation into the five densities an implementer must distinguish.

The preconditioned step is easy to lose because the paper moves between physical coordinates  $r = (x_t, \theta, x_{t-1})$ , reference coordinates  $u = (u_t, u_\theta, u_{t-1})$ , an exact bridge, an approximate bridge, and a residual density. The complete chain is as follows.

First, there is the exact target we ultimately want to approximate:

$$q_t(r) = \hat{\pi}_{t-1}(x_{t-1}, \theta) f_\theta(x_t | x_{t-1}) g_\theta(y_t | x_t). \quad (6.118)$$

Second, there is an exact bridge  $\rho_t(r)$ . It is not the final answer. It is a simpler density chosen to resemble the main shape of  $q_t$ . In the tempered construction it is the powered product in (6.116). In the Gaussian construction it is  $N(\mu_t, \Sigma_t)$ .

Third, because the exact bridge may still be nonseparable, Algorithm 5(b.1) approximates the bridge itself by a squared TT:

$$\sqrt{\rho_t(r)} \approx \phi_{\rho,t}(r), \quad \hat{\rho}_t(r) = \phi_{\rho,t}(r)^2 + \tau_{\rho,t}\lambda(r). \quad (6.119)$$

This is the first line of Algorithm 5 in mathematical form. The implementation stores the bridge TT cores, the defensive mass  $\tau_{\rho,t}$ , the reference density  $\lambda$ , the bridge normalizer, and the mass contractions needed to construct conditional maps from  $\hat{\rho}_t$ .

Fourth, build the preconditioning map from the bridge approximation. Let

$$T_t : r \mapsto u \quad (6.120)$$

be a KR-based map satisfying

$$(T_t)_\# \hat{\rho}_t(u) \propto \eta(u). \quad (6.121)$$

Using  $\hat{\rho}_t$ , not the unavailable exact bridge, the residual target to approximate in reference coordinates is

$$q_t^\#(u) = \frac{q_t(T_t^{-1}(u))}{\hat{\rho}_t(T_t^{-1}(u))} \eta(u). \quad (6.122)$$

This formula answers the question “what remains after the bridge has removed the main shape?” If the bridge is good, the ratio  $q_t/\hat{\rho}_t$  is flatter than  $q_t$ , so the residual may need smaller TT ranks.

Fifth, approximate the square root of the residual:

$$\sqrt{q_t^\#(u)} \approx \phi_t^\#(u), \quad \hat{v}_t^\#(u) = \phi_t^\#(u)^2 + \tau_t^\# \eta(u). \quad (6.123)$$

Finally pull this residual approximation back to physical coordinates:

$$\hat{\pi}_t(r) = \frac{\hat{v}_t^\#(T_t(r))}{\eta(T_t(r))} \hat{\rho}_t(r). \quad (6.124)$$

To check the algebra, substitute  $u = T_t(r)$ . Since  $(T_t)_\# \hat{\rho}_t \propto \eta$ , the factor  $\hat{\rho}_t(r)/\eta(T_t(r))$  restores the physical-coordinate density scale. The normalizer is computed in reference space:

$$\int \hat{\pi}_t(r) dr = \int \hat{v}_t^\#(u) du. \quad (6.125)$$

Algorithm 5 stores exactly the objects in this chain:

$$\hat{\rho}_t, \quad T_t, \quad q_t^\# \text{ evaluator}, \quad \hat{v}_t^\#, \quad \hat{\pi}_t \text{ evaluator}, \quad \text{marginal and conditional maps.} \quad (6.126)$$

This is the preconditioning story in one line of thought: approximate the bridge, use the bridge to flatten the target, approximate the flattened residual, then pull the residual back.

Failure checks for the five-density chain. Verify all of the following:

$$q_t(r) \geq 0, \quad \hat{\rho}_t(r) > 0 \text{ on fitted target points}, \quad q_t^\#(u) \geq 0, \quad (6.127)$$

$$0 < \int \hat{v}_t^\#(u) du < \infty, \quad \left| \int \hat{\pi}_t(r) dr - \int \hat{v}_t^\#(u) du \right| \leq \varepsilon_{\text{norm}}. \quad (6.128)$$

These are not cosmetic checks. They assess whether the density chain still defines the same scalar after moving between coordinate systems.



**Mathematical question.** Algorithm 5(b.1) answers: what approximation is built before the residual target is fitted?

The bridge approximation is  $\hat{\rho}_t$ , already defined in (6.119). Its mathematical input-output relation is

$$(r \mapsto \rho_t(r), \lambda, \tau_{\rho,t}, \text{basis}, \text{ranks}) \mapsto (\hat{\rho}_t, \text{bridge mass contractions}). \quad (6.129)$$

Failure checks: bridge fit residual, positivity of  $\hat{\rho}_t$ , and nonzero bridge values on target fitting points.

**Mathematical question.** Algorithm 5(b.2) answers: how does the bridge approximation become a lower preconditioning map and a bridge marginal?

Build a KR map  $R_t$  from  $\hat{\rho}_t$  to a uniform variable:

$$(R_t)_\# \hat{\rho}_t(\xi_t, \xi_\theta, \xi_{t-1}) \propto \text{Unif}(\xi_t, \xi_\theta, \xi_{t-1}). \quad (6.130)$$

If  $D$  is a diagonal coordinate map satisfying

$$D_\# \eta(\xi_t, \xi_\theta, \xi_{t-1}) = \text{Unif}(\xi_t, \xi_\theta, \xi_{t-1}), \quad (6.131)$$

then

$$T_t = D^{-1} \circ R_t \quad (6.132)$$

pushes the bridge approximation to the product reference  $\eta$ :

$$(T_t)_\# \hat{\rho}_t(u_t, u_\theta, u_{t-1}) \propto \eta(u_t, u_\theta, u_{t-1}). \quad (6.133)$$

For the lower-triangular version, write

$$T_t^\ell(x_t, \theta, x_{t-1}) = (T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta \mid x_t), T_{t,t-1}^\ell(x_{t-1} \mid x_t, \theta)). \quad (6.134)$$

The same bridge mass contractions also provide the bridge marginal

$$\hat{\rho}_t(x_t, \theta) = \int \hat{\rho}_t(x_t, \theta, x_{t-1}) dx_{t-1}. \quad (6.135)$$

Mathematical input and output:

$$\hat{\rho}_t \mapsto (T_t^\ell, (T_t^\ell)^{-1}, \hat{\rho}_t(x_t, \theta), T_{t,t-1}^\ell). \quad (6.136)$$

Failure checks: monotone conditional CDFs, inverse-map residuals, and agreement between the bridge marginal and direct bridge contraction.

**Mathematical question.** Algorithm 5(b.3) answers: after the bridge map is available, what residual density is fitted in reference coordinates?

With this nonlinear preconditioner, the residual target becomes

$$q_t^\sharp(u) = \frac{q_t(T_t^{-1}(u))}{\hat{\rho}_t(T_t^{-1}(u))} \eta(u), \quad (6.137)$$

and the pulled-back unnormalized posterior approximation is

$$\hat{\pi}_t(r) = \frac{\hat{\nu}_t^\sharp(T_t(r))}{\eta(T_t(r))} \hat{\rho}_t(r). \quad (6.138)$$

Mathematical input and output:

$$(q_t, T_t^{-1}, \hat{\rho}_t, \eta, \text{basis}, \text{ranks}) \mapsto (\hat{\nu}_t^\sharp, S_t^\ell, \text{residual mass contractions}). \quad (6.139)$$

Failure checks: finite ratio  $q_t/\hat{\rho}_t$ , residual rank growth, and normalizer  $\int \hat{\nu}_t^\sharp(u) du \in (0, \infty)$ .

**Mathematical question.** Algorithm 5(c.1) answers: what residual marginal is needed before returning to physical coordinates?

Use the lower triangular decompositions

$$S_t^\ell(u_t, u_\theta, u_{t-1}) = \begin{bmatrix} S_{t,t}^\ell(u_t) \\ S_{t,\theta}^\ell(u_\theta | u_t) \\ S_{t,t-1}^\ell(u_{t-1} | u_t, u_\theta) \end{bmatrix}, \quad T_t^\ell(x_t, \theta, x_{t-1}) = \begin{bmatrix} T_{t,t}^\ell(x_t) \\ T_{t,\theta}^\ell(\theta | x_t) \\ T_{t,t-1}^\ell(x_{t-1} | x_t, \theta) \end{bmatrix}. \quad (6.140)$$

Step (c.1) integrates the last block ( $u_{t-1}$ ) of  $\hat{\nu}_t^\#$  to obtain

$$\hat{\nu}_t^\#(u_t, u_\theta) = \int \hat{\nu}_t^\#(u_t, u_\theta, u_{t-1}) du_{t-1}. \quad (6.141)$$

This is an endpoint squared-TT marginalization in  $u$ -space, so it uses the same mass-matrix algebra as Proposition 2. It also creates the residual lower conditional map  $S_{t,t-1}^\ell(u_{t-1} | u_t, u_\theta)$ .

**Mathematical question.** Algorithm 5(c.2) answers: how does the residual marginal become the retained physical-coordinate filter?

Zhao and Cui's Algorithm 5(c.2) is compact because it uses the lower triangular map  $T_t^\ell$  and the residual marginal from step (c.1) in one line. We now derive that line slowly. The goal is to prove the physical retained marginal

$$\hat{\pi}_t(x_t, \theta | y_{1:t})$$

from the three-block pulled-back density, without losing the Jacobian direction.

First name the two retained physical variables and the old-state variable:

$$a = (x_t, \theta), \quad b = x_{t-1}. \quad (6.142)$$

The lower bridge map is block triangular:

$$T_t^\ell(a, b) = (T_A(a), T_B(b | a)), \quad T_A(a) = (T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta | x_t)), \quad (6.143)$$

where  $T_B(b | a) = T_{t,t-1}^\ell(x_{t-1} | x_t, \theta)$ . Write

$$u_A = (u_t, u_\theta), \quad u_B = u_{t-1}.$$

Thus  $u_A = T_A(a)$  and  $u_B = T_B(b | a)$ .

Using

$$(u_t, u_\theta) = (T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta | x_t)), \quad (6.144)$$

start from the pulled-back three-block density (6.138):

$$\hat{\pi}_t(x_t, \theta, x_{t-1}) = \frac{\hat{\nu}_t^\#(T_t^\ell(x_t, \theta, x_{t-1}))}{\eta(T_t^\ell(x_t, \theta, x_{t-1}))} \hat{\rho}_t(x_t, \theta, x_{t-1}). \quad (6.145)$$

The source formula to reach is

$$\hat{\pi}_t(a) = \frac{\hat{\nu}_t^\#(T_A(a))}{\eta_A(T_A(a))} \hat{\rho}_{t,A}(a), \quad (6.146)$$

where

$$\hat{\rho}_{t,A}(a) = \int \hat{\rho}_t(a, b) db, \quad \hat{\nu}_t^\#(u_A) = \int \hat{\nu}_t^\#(u_A, u_B) du_B, \quad (6.147)$$

and

$$\eta_A(u_A) = \int \eta(u_A, u_B) du_B. \quad (6.148)$$

For Zhao and Cui's product reference,  $\eta(u_A, u_B) = \eta_A(u_A)\eta_B(u_B)$ , so  $\eta_A$  is just the reference density of the retained transformed coordinates.

The only delicate point is how  $\hat{\rho}_t(a, b) db$  transforms when  $a$  is fixed. Because  $T_t^\ell$  pushes the bridge approximation to the reference density,

$$\hat{\rho}_t(a, b) da db = C_\rho \eta(u_A, u_B) du_A du_B, \quad (6.149)$$

for a bridge normalizing constant  $C_\rho > 0$ . Marginalizing both sides over  $b$  or  $u_B$  gives the retained bridge marginal identity

$$\hat{\rho}_{t,A}(a) da = C_\rho \eta_A(u_A) du_A. \quad (6.150)$$

Dividing (6.149) by (6.150) gives the conditional change-of-variables identity

$$\hat{\rho}_t(b | a) db = \eta_B(u_B | u_A) du_B. \quad (6.151)$$

For a product reference,  $\eta_B(u_B | u_A) = \eta_B(u_B)$ , but we keep the conditional notation because it shows what must be true even for a block-triangular non-product reference.

Now change variables only in the old-state conditional block. For fixed  $(x_t, \theta)$ , the lower bridge map sends

$$x_{t-1} \mapsto u_{t-1} = T_{t,t-1}^\ell(x_{t-1} | x_t, \theta). \quad (6.152)$$

Write

$$u_t = T_{t,t}^\ell(x_t), \quad u_\theta = T_{t,\theta}^\ell(\theta | x_t). \quad (6.153)$$

The conditional part of the triangular pushforward identity says that, for fixed  $(x_t, \theta)$ ,

$$\hat{\rho}_t(x_{t-1} | x_t, \theta) dx_{t-1} = \eta(u_{t-1} | u_t, u_\theta) du_{t-1}. \quad (6.154)$$

This is the conditional KR transport for the old-state block: once the retained block  $(x_t, \theta)$  is held fixed, the monotone lower-triangular map  $x_{t-1} \mapsto u_{t-1}$  changes the conditional bridge law into the conditional reference law. Since the reference density factorizes in the transformed coordinates,

$$\eta(u_t, u_\theta, u_{t-1}) = \eta(u_{t-1} | u_t, u_\theta) \eta(u_t, u_\theta), \quad (6.155)$$

the conditional density in (6.154) can be written as

$$\eta(u_{t-1} | u_t, u_\theta) = \frac{\eta(T_t^\ell(x_t, \theta, x_{t-1}))}{\eta(T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta | x_t))}. \quad (6.156)$$

Multiplying (6.154) by the bridge marginal  $\hat{\rho}_t(x_t, \theta)$  gives the local bridge decomposition

$$\hat{\rho}_t(x_t, \theta, x_{t-1}) dx_{t-1} = \frac{\hat{\rho}_t(x_t, \theta)}{\eta(T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta | x_t))} \eta(T_t^\ell(x_t, \theta, x_{t-1})) du_{t-1}, \quad (6.157)$$

where the Jacobian of the conditional map is included in the differential  $du_{t-1}$ . In the compact notation  $(a, b, u_A, u_B)$ , the same identity is

$$\hat{\rho}_t(a, b) db = \frac{\hat{\rho}_{t,A}(a)}{\eta_A(T_A(a))} \eta(T_A(a), u_B) du_B. \quad (6.158)$$

This is the cancellation identity. It is where the Jacobian enters the derivation; it is not an extra factor to be appended later.

Substituting (6.158) into the pulled-back density (6.145) and using  $T_t^\ell(a, b) = (T_A(a), u_B)$  gives

$$\begin{aligned}\hat{\pi}_t(a, b) db &= \frac{\hat{\nu}_t^\sharp(T_A(a), u_B)}{\eta(T_A(a), u_B)} \hat{\rho}_t(a, b) db \\ &= \frac{\hat{\nu}_t^\sharp(T_A(a), u_B)}{\eta(T_A(a), u_B)} \frac{\hat{\rho}_{t,A}(a)}{\eta_A(T_A(a))} \eta(T_A(a), u_B) du_B \\ &= \frac{\hat{\rho}_{t,A}(a)}{\eta_A(T_A(a))} \hat{\nu}_t^\sharp(T_A(a), u_B) du_B.\end{aligned}\tag{6.159}$$

The full three-block reference density cancels before the old-state variable is integrated. Integrating (6.159) over  $u_B$  gives

$$\hat{\pi}_t(a) = \frac{\hat{\rho}_{t,A}(a)}{\eta_A(T_A(a))} \int \hat{\nu}_t^\sharp(T_A(a), u_B) du_B.\tag{6.160}$$

This is exactly step (c.1) followed by step (c.2): first compute the residual marginal in  $u$ -space, then pull it back to the retained physical block using the bridge marginal.

Returning from  $a = (x_t, \theta)$  notation to the paper's notation, integrating over  $x_{t-1}$  gives

$$\begin{aligned}\hat{\pi}_t(x_t, \theta \mid y_{1:t}) &= \int \hat{\pi}_t(x_t, \theta, x_{t-1}) dx_{t-1} \\ &= \frac{\hat{\rho}_t(x_t, \theta)}{\eta(T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta \mid x_t))} \int \hat{\nu}_t^\sharp(u_t, u_\theta, u_{t-1}) du_{t-1}.\end{aligned}\tag{6.161}$$

Using (6.141) with  $(u_t, u_\theta) = (T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta \mid x_t))$ , the retained marginal density for the next filtering step is

$$\hat{\pi}_t(x_t, \theta \mid y_{1:t}) = \frac{\hat{\nu}_t^\sharp(T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta \mid x_t))}{\eta(T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta \mid x_t))} \hat{\rho}_t(x_t, \theta).\tag{6.162}$$

Here the two-argument  $\eta$  in the denominator is the retained reference marginal  $\eta_A(T_A(a))$ , not the full three-block reference density  $\eta(T_A(a), u_B)$ . This is the source formula in Algorithm 5(c.2) written without relying on paper shorthand. The implementation stores the two maps, their marginal evaluators, and a retained evaluator for  $\hat{\pi}_t(x_t, \theta)$ .

**Two-block sanity check.** If the residual approximation is perfect in the sense that

$$\hat{\nu}_t^\sharp(u_A, u_B) = \frac{q_t(T_t^{-1}(u_A, u_B))}{\hat{\rho}_t(T_t^{-1}(u_A, u_B))} \eta(u_A, u_B),\tag{6.163}$$

then (6.138) gives  $\hat{\pi}_t(a, b) = q_t(a, b)$ . Formula (6.160) therefore reduces to

$$\hat{\pi}_t(a) = \int q_t(a, b) db,\tag{6.164}$$

which is ordinary marginalization. This check does not prove numerical accuracy, but it verifies the direction of the density ratio: a perfect residual fit returns the exact retained marginal.

As a byproduct, the lower conditional map for path estimation is the last block of the composite map:

$$F_{t,t-1}^\ell(x_{t-1} \mid x_t, \theta) = S_{t,t-1}^\ell\left(T_{t,t-1}^\ell(x_{t-1} \mid x_t, \theta) \mid T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta \mid x_t)\right).\tag{6.165}$$

This identity matters because preconditioning does not eliminate smoothing; it changes which conditional maps must be composed and stored.

**Boxed Algorithm 5 dataflow.**

| Step | Evaluator signature                                                                     | Quantities retained                 | Main diagnostic       |
|------|-----------------------------------------------------------------------------------------|-------------------------------------|-----------------------|
| b.1  | $\rho_t(r) \mapsto \widehat{\rho}_t(r)$                                                 | bridge TT, $\tau_{\rho,t}, \lambda$ | bridge residual/rank  |
| b.2  | $\widehat{\rho}_t \mapsto T_t^\ell, (T_t^\ell)^{-1}$                                    | bridge maps, bridge marginal        | CDF monotonicity      |
| b.3  | $(q_t, T_t^{-1}, \widehat{\rho}_t, \eta) \mapsto \widehat{\nu}_t^\#$                    | residual TT, $S_t^\ell$             | ratio finite          |
| c.1  | $\widehat{\nu}_t^\#(u_t, u_\theta, u_{t-1}) \mapsto \widehat{\nu}_t^\#(u_t, u_\theta)$  | residual marginal, $S_{t,t-1}^\ell$ | mass contraction      |
| c.2  | $(\widehat{\nu}_t^\#, T_t^\ell, \widehat{\rho}_t) \mapsto \widehat{\pi}_t(x_t, \theta)$ | retained filter evaluator           | normalizer invariance |

## 7 Squared-TT Filtering Object And Operational Recursion

The reason to introduce tensor trains is not storage arithmetic alone. The starting mathematical object is a posterior density. At time  $t$ , before normalization, the exact adjacent-state filtering target has the form

$$q_t(x_t, \theta, x_{t-1}) = p(x_{t-1}, \theta \mid y_{1:t-1}) f_\theta(x_t \mid x_{t-1}) g_\theta(y_t \mid x_t). \quad (7.1)$$

If the state has  $m$  components and the parameter has  $d$  components, then

$$(x_t, \theta, x_{t-1}) \in \mathbb{R}^{2m+d}. \quad (7.2)$$

Thus the exact object is a function of many variables, not a vector of probabilities. Filtering is the operation

$$p(x_t, \theta \mid y_{1:t}) = \frac{\int q_t(x_t, \theta, x_{t-1}) dx_{t-1}}{\int q_t(x_t, \theta, x_{t-1}) dx_{t-1} dx_t d\theta}. \quad (7.3)$$

The denominator is the evidence increment and the numerator is the retained filter. A high-dimensional filtering method must therefore answer two questions at the same time:

$$\text{How do we represent } q_t? \quad \text{How do we integrate the represented object?} \quad (7.4)$$

A Gaussian filter answers the first question by replacing the posterior with a mean and covariance:

$$p(x_t, \theta \mid y_{1:t}) \approx N\left(\begin{bmatrix} x_t \\ \theta \end{bmatrix}; \mu_t, \Sigma_t\right). \quad (7.5)$$

This is efficient, but it assumes the relevant posterior shape can be captured by ellipsoids. In the threaded example,

$$y_t = \sin(x_t) + \eta_t, \quad (7.6)$$

the likelihood is

$$g(y_t \mid x_t) \propto \exp\left[-\frac{(y_t - \sin x_t)^2}{2r^2}\right]. \quad (7.7)$$

For  $y_t$  near zero, this function can have high density near several separated values of  $x_t$ . The posterior may be skewed, curved, or multimodal. A single Gaussian can still be useful as a preconditioner, but it is not a full representation of the nonlinear density.

Tensor trains answer the first question by approximating a function with separated interaction channels. After mapping physical coordinates to reference coordinates  $z = (z_1, \dots, z_D)$ , write

$$\tilde{q}_t(z) = q_t(\Psi_t(z)) |\det \nabla \Psi_t(z)|. \quad (7.8)$$

The TT approximation seeks a product of low-rank matrix-valued univariate functions:

$$\tilde{q}_t(z) \approx H_1(z_1)H_2(z_2) \cdots H_D(z_D). \quad (7.9)$$

This does not assume independence. Independence would be the rank-one factorization

$$\tilde{q}_t(z_1, \dots, z_D) = \prod_{k=1}^D h_k(z_k). \quad (7.10)$$

A rank- $R$  tensor train allows  $R$  interaction channels to pass information across coordinate splits. At a split  $z_{\leq k} \mid z_{> k}$ , the local meaning is

$$\tilde{q}_t(z_{\leq k}, z_{> k}) \approx \sum_{a=1}^{R_k} U_{k,a}(z_{\leq k}) V_{k,a}(z_{> k}). \quad (7.11)$$

Thus TT rank is a measure of how much dependence crosses the split. It is a controlled relaxation of independence, not a return to independent coordinates.

The second question, integration, is answered by using basis functions and mass matrices. If

$$H_k(z_k) = \sum_{\ell=0}^{p_k-1} C_k[:, \ell, :] b_{k,\ell}(z_k), \quad (7.12)$$

then one-dimensional integrals such as

$$B_k[\ell, m] = \int_{-1}^1 b_{k,\ell}(z) b_{k,m}(z) \, dz \quad (7.13)$$

turn high-dimensional squared-TT integrals into repeated small matrix contractions. The approximation family is therefore matched to the filtering task: it represents a nonlinear density and gives a way to marginalize it.

The method is plausible when the posterior geometry has limited interaction width after ordering and preconditioning. A local transition model has the schematic form

$$f(x_t \mid x_{t-1}) \approx \prod_{j=1}^m f_j(x_{t,j} \mid x_{t-1,j}, x_{t-1, \mathcal{N}(j)}), \quad (7.14)$$

and a local observation model has

$$g(y_t \mid x_t) \approx \prod_{\ell=1}^n g_\ell(y_{t,\ell} \mid x_{t, \mathcal{I}_\ell}). \quad (7.15)$$

If the coordinate order keeps statistically neighboring variables close, then many splits separate only a moderate number of direct interactions. The rank diagnostic

$$\max_k R_k \leq R_{\max} \quad (7.16)$$

is a check of this structural hypothesis. This locality argument is a modeling and numerical heuristic for why low or moderate TT ranks may approximate  $\tilde{q}_t$  well in structured cases; it is not a theorem that such ranks suffice uniformly across models, observations, coordinate orderings, or preconditioners.

The method should not be expected to work automatically. A sharp global constraint,

$$y_t \approx \sum_{j=1}^m a_j x_{t,j}, \quad r^2 \ll 1, \quad (7.17)$$

can create dependence crossing many splits. A many-mode posterior can do the same. In those cases the practical signs are

$$R_k \text{ grows, } \frac{\|Ag - y\|_W}{\|y\|_W} \text{ stays large, } \text{cond}(A^\top W A + \rho I) \text{ grows.} \quad (7.18)$$

These are failure diagnostics, not embarrassments. They tell the panel that the approximation family is being tested against the posterior geometry rather than assumed correct.

Preconditioning explains why a Gaussian object can still be useful without being the final approximation. Let  $\rho_t$  be a bridge density capturing location, scale, and dominant correlations. The residual target

$$q_t^\#(u) = \frac{q_t(T_t^{-1}(u))}{\rho_t(T_t^{-1}(u))} \eta(u) \quad (7.19)$$

asks the TT to represent a correction factor. If  $\rho_t$  captures the main shape, then

$$\frac{q_t(T_t^{-1}(u))}{\rho_t(T_t^{-1}(u))} \quad (7.20)$$

is flatter than the original density, and lower TT rank becomes more plausible. This is the conceptual reason the Zhao–Cui method combines tensor trains, squared densities, conditional maps, and preconditioning rather than relying on one device alone.

## 7.1 Why TT Marginalization Is Cheap Once The Representation Exists

**Mathematical question.** The question answered here is: once a function is in TT form, how does the algorithm integrate out one coordinate without returning to a full grid?

Suppose  $h$  is represented by (TT-1). If we need to integrate out coordinate  $r_k$ , the product structure gives

$$\int \hat{h}(r_1, \dots, r_D) dr_k = H_1(r_1) \cdots H_{k-1}(r_{k-1}) \left( \int H_k(r_k) dr_k \right) H_{k+1}(r_{k+1}) \cdots H_D(r_D). \quad (7.21)$$

Only the  $k$ -th core changes. Its integrated matrix is

$$\bar{H}_k = \int H_k(r_k) dr_k, \quad \bar{H}_k[a, b] = \sum_{\ell=0}^{p_k-1} C_k[a, \ell, b] \int \psi_{k,\ell}(r_k) dr_k. \quad (7.22)$$

If the one-dimensional basis integrals are precomputed, marginalizing one coordinate is a small matrix contraction. Repeatedly applying (TT-4) integrates whole blocks of variables.

This explains why Zhao and Cui [4] target a TT representation of  $(x_t, \alpha, x_{t-1})$ . Once that three-block function is separable enough, the new filter  $p(x_t, \alpha \mid y_{1:t})$  is obtained by integrating the old-state block  $x_{t-1}$ , one coordinate at a time.

Implementation meaning. 2-core-integration. Inputs are the TT cores and precomputed one-dimensional basis integrals. The operation replaces one core by  $\overline{H}_k$  and contracts adjacent matrices. The retained object is a lower-dimensional TT or evaluator. The failure check is dimension/order consistency: integrating the wrong block silently returns the wrong filter.

## 7.2 Algorithm 1, Fully Annotated

**Mathematical question.** Algorithm 1(a) answers: what target can be evaluated at time  $t$  using only the retained approximation from time  $t - 1$ ?

The exact recursion (BF-10) contains the previous exact filter. A numerical algorithm has the previous approximation. Let  $\hat{\pi}_{t-1}(x_{t-1}, \alpha)$  denote an unnormalized approximation of  $p(x_{t-1}, \alpha \mid y_{1:t-1})$ . Zhao and Cui [4], Eq. (12) becomes

$$q_t(x_t, \alpha, x_{t-1}) = \hat{\pi}_{t-1}(x_{t-1}, \alpha) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t). \quad (7.23)$$

This is not separable in general. Even if  $\hat{\pi}_{t-1}$  is a TT over  $(\alpha, x_{t-1})$ , the transition and likelihood couple the new and old state coordinates.

Zhao and Cui [4] use approximate proportionality before normalization. In explicit terms,  $h_1 \propto_{\text{approx}} h_2$  means

$$\frac{h_1(r)}{\int h_1(s) ds} \approx \frac{h_2(r)}{\int h_2(s) ds}. \quad (7.24)$$

This is why the algorithm can store an unnormalized TT. The normalization is not forgotten; it is postponed until the complete TT contraction  $\hat{Z}_t$  is computed.

Algorithm 1 has three mathematical steps.

**Step 1: build a pointwise target.** Given  $r_t = (x_t, \alpha, x_{t-1})$ , evaluate  $q_t(r_t)$  by multiplying the previous approximation, transition density, and likelihood:

$$r_t \mapsto \hat{\pi}_{t-1}(x_{t-1}, \alpha) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t).$$

Storage needed: an evaluator for the previous TT approximation and evaluators for  $f_\alpha$  and  $g_\alpha$ .

Implementation meaning. Inputs:  $\hat{\pi}_{t-1}$ ,  $f_\alpha$ ,  $g_\alpha$ ,  $y_t$ , and a point  $(x_t, \alpha, x_{t-1})$ . Output:  $q_t$  or  $\log q_t$ . Quantities to retain: none beyond the evaluator. Operation: multiply three factors or add three log factors. Diagnostic: reject NaN, negative density values, or inconsistent coordinate block sizes.

**Step 2: re-approximate by a TT.**

**Mathematical question.** This paragraph answers: how does the nonseparable target become a separable object that can be marginalized?

Choose a variable order, in the paper usually

$$(x_t, \alpha, x_{t-1}).$$

Fit a TT

$$\hat{\pi}_t(x_t, \alpha, x_{t-1}) = F_{t,1}(x_{t,1}) \cdots F_{t,m_x}(x_{t,m_x}) A_{t,1}(\alpha_1) \cdots A_{t,m_\alpha}(\alpha_{m_\alpha}) H_{t,1}(x_{t-1,1}) \cdots H_{t,m_x}(x_{t-1,m_x}) \quad (7.25)$$



so that  $\hat{\pi}_t \approx q_t$ . Zhao and Cui [4] use functional TT tools with alternating approximation and rank adaptation. For implementation one must choose a domain map, basis, fitting points, rank rule, and stopping rule. The paper’s research code supplies one such family; the fixed-branch section below supplies a differentiable fixed-branch specialization.

The most direct implementation view is least squares. Choose fitting points

$$r^{(j)} = (x_t^{(j)}, \alpha^{(j)}, x_{t-1}^{(j)}), \quad j = 1, \dots, N_{\text{fit}},$$

and define target values

$$y_j = q_t(r^{(j)}).$$

For fixed TT ranks and fixed all cores except core  $k$ , the fitted function is linear in the entries of core  $C_k$ . There is a design row  $A_{j,k}$  such that

$$\hat{\pi}_t(r^{(j)}) = A_{j,k} g_k, \quad g_k = \text{vec}(C_k). \quad (7.26)$$

Let  $L_{j,k}$  be the left environment obtained by multiplying all cores before  $k$  at point  $r^{(j)}$ , and let  $R_{j,k}$  be the right environment from all cores after  $k$ . Let  $\psi_k(r_k^{(j)})$  be the local basis vector. Then

$$A_{j,k} = R_{j,k}^\top \otimes \psi_k(r_k^{(j)})^\top \otimes L_{j,k}. \quad (7.27)$$

This formula is worth lingering over. The left environment tells the current core what the earlier coordinates have already contributed. The right environment tells it what later coordinates will contribute. The basis vector gives the local coordinate dependence. The Kronecker product stitches these three pieces into a row that multiplies the vectorized core.

For a concrete version of this row, see the three-variable exponential example in Section 4. There the middle-core row is written entry by entry as

$$A_{j,2}[a, \ell, b] = L_j[a] y_j^\ell R_j[b],$$

and multiplying that row by the vectorized middle core reproduces the TT value at the fitting point. The general formula (7.27) is the same calculation with arbitrary ranks and arbitrary local basis functions.

With weights  $w_j > 0$ , the core update solves

$$\min_{g_k} \sum_{j=1}^{N_{\text{fit}}} w_j (A_{j,k} g_k - y_j)^2 + \rho \|g_k\|_2^2, \quad (7.28)$$

or equivalently

$$(A_k^\top W A_k + \rho I) g_k = A_k^\top W y. \quad (7.29)$$

An alternating scheme sweeps through the cores, updating one core at a time. Zhao and Cui [4]’s adaptive implementation is more sophisticated, but this least-squares view is the easiest way to see how a TT approximation becomes an actual numerical object.

Implementation meaning. Inputs: target evaluator  $q_t$ , domains, basis objects, fitting points, weights, ranks, and sweep settings. Output: cores  $F, A, H$  for the fitted TT. Quantities to retain: cores, rank vector, basis definition, fit residual, and any scaling shift. Operation: alternating core least-squares or cross approximation. Diagnostic: weighted residual, condition number of each normal matrix, rank growth, and negative values if this basic non-squared algorithm is interpreted as a density.

## Deterministic Fixed-Rank Sweep Protocol

For the fixed-branch derivative, the sweep rule must be a finite deterministic map. The branch declares

$$\mathcal{S}_{\text{fit}} = \{Z_{\text{fit}}, W, R_{0:D}, p_{1:D}, \rho, S_{\text{max}}, \varepsilon_{\text{rel}}, \varepsilon_{\text{abs}}, s_{\text{rescale}}\}. \quad (7.30)$$

Here  $Z_{\text{fit}}$  are fitting points,  $W$  are weights,  $R_{0:D}$  are fixed ranks,  $p_{1:D}$  are basis sizes,  $\rho$  is the ridge parameter,  $S_{\text{max}}$  is the maximum number of bidirectional sweeps,  $\varepsilon_{\text{rel}}, \varepsilon_{\text{abs}}$  are stopping tolerances, and  $s_{\text{rescale}}$  is the declared rescaling convention.

For each fitting point  $z^{(j)} = (z_1^{(j)}, \dots, z_D^{(j)})$ , define the target value used by the sweep as

$$y_j = e^{ct/2} \sqrt{\tilde{q}_t(z^{(j)}; \beta)}, \quad y = (y_1, \dots, y_{N_{\text{fit}}})^\top. \quad (7.31)$$

The current core matrices at that point are

$$H_k^{(j)} = \sum_{\ell=0}^{p_k-1} \psi_{k,\ell}(z_k^{(j)}) C_k[:, \ell, :]. \quad (7.32)$$

Before updating core  $k$ , build the left and right environments from the current values of the other cores:

$$L_{j,1} = 1, \quad L_{j,k+1} = L_{j,k} H_k^{(j)}, \quad (7.33)$$

and

$$R_{j,D+1} = 1, \quad R_{j,k} = H_k^{(j)} R_{j,k+1}. \quad (7.34)$$

Thus  $L_{j,k} \in \mathbb{R}^{1 \times R_{k-1}}$  summarizes coordinates left of the core being updated, while  $R_{j,k+1} \in \mathbb{R}^{R_k \times 1}$  summarizes coordinates right of it. With the local basis row

$$b_{j,k}^\top = (\psi_{k,0}(z_k^{(j)}), \dots, \psi_{k,p_k-1}(z_k^{(j)})), \quad (7.35)$$

the least-squares row for the vectorized core  $g_k = \text{vec}(C_k)$  is

$$A_k[j, :] = R_{j,k+1}^\top \otimes b_{j,k}^\top \otimes L_{j,k}. \quad (7.36)$$

The weighted normal-equation pieces are therefore

$$N_k = A_k^\top W A_k + \rho I, \quad d_k = A_k^\top W y. \quad (7.37)$$

After solving for  $g_k$ , set  $C_k = \text{vec}^{-1}(g_k)$ , recompute  $H_k^{(j)}$ , and update the affected environment before moving to the next core:

$$\text{forward pass: } L_{j,k+1} \leftarrow L_{j,k} H_k^{(j)}, \quad \text{backward pass: } R_{j,k} \leftarrow H_k^{(j)} R_{j,k+1}. \quad (7.38)$$

This is the minimal deterministic sweep object. Adaptive TT-cross can choose new interpolation points, pivots, or ranks; the fixed-branch derivative here does not.

One forward sweep applies the core updates

$$k = 1, 2, \dots, D, \quad N_k g_k = d_k, \quad N_k = A_k^\top W A_k + \rho I. \quad (7.39)$$

One backward sweep applies the same update in the reverse order

$$k = D, D-1, \dots, 1. \quad (7.40)$$

After a complete bidirectional sweep  $s$ , compute the weighted residual

$$r_s^2 = \frac{\sum_{j=1}^{N_{\text{fit}}} w_j [\phi_s(z^{(j)}) - y_j]^2}{\sum_{j=1}^{N_{\text{fit}}} w_j y_j^2 + \varepsilon_{\text{abs}}}. \quad (7.41)$$

The deterministic stopping rule is

$$s = s_{\text{max}} \quad \text{or} \quad r_s \leq \varepsilon_{\text{rel}} \quad \text{or} \quad |r_{s-1} - r_s| \leq \varepsilon_{\text{abs}}. \quad (7.42)$$

For a same-scalar derivative, the realized number of sweeps is frozen:

$$S = S(\beta_0), \quad S(\beta_0 + h) = S(\beta_0 - h) = S. \quad (7.43)$$

Thus finite differences compare the same finite composition of solve maps. If the residual criterion would stop at a different sweep count for a perturbed parameter, the fixed-branch diagnostic must continue to the saved  $S$  rather than changing the branch.

Optional rescaling is allowed only if it is declared. A simple convention is to normalize each core after its update and carry the product of normalization constants into the global shift:

$$C_k \leftarrow C_k / \alpha_k, \quad c_t \leftarrow c_t - 2 \sum_k \log \alpha_k. \quad (7.44)$$

If rescaling is not used, record  $s_{\text{rescale}} = \text{none}$ . The failure recovery rule for the fixed-rank protocol is not to increase rank silently. Instead record

$$r_S > \varepsilon_{\text{rel}} \quad \Rightarrow \quad \text{fixed-rank fit failure for this branch.} \quad (7.45)$$

Rank adaptation may be studied later, but it is a different branch.

### Step 3: integrate the old state.

**Mathematical question.** This paragraph answers: after the three-block target is fitted, what is retained for the next sequential update?

The next approximate filter is

$$\hat{\pi}_t(x_t, \alpha) = \int \hat{\pi}_t(x_t, \alpha, x_{t-1}) dx_{t-1}. \quad (7.46)$$

Because  $x_{t-1}$  occupies the last block, this integral is a sequence of core integrals:

$$\hat{\pi}_t(x_t, \alpha) = F_{t,1}(x_{t,1}) \cdots F_{t,m_x}(x_{t,m_x}) A_{t,1}(\alpha_1) \cdots A_{t,m_\alpha}(\alpha_{m_\alpha}) \overline{H}_{t,1} \cdots \overline{H}_{t,m_x}. \quad (7.47)$$

The approximate normalizer is

$$\hat{Z}_t = \int \hat{\pi}_t(x_t, \alpha, x_{t-1}) dx_t d\alpha dx_{t-1}, \quad (7.48)$$

again a complete TT contraction.

The same fitted TT gives the parameter marginal and filtering marginal:

$$\hat{p}_t(\alpha) = \frac{\int \hat{\pi}_t(x_t, \alpha, x_{t-1}) dx_t dx_{t-1}}{\hat{Z}_t}, \quad \hat{p}_t(x_t) = \frac{\int \hat{\pi}_t(x_t, \alpha, x_{t-1}) d\alpha dx_{t-1}}{\hat{Z}_t}. \quad (7.49)$$

The retained object for the next time step is not merely a scalar normalizer. It is the evaluable two-block density

$$\hat{\pi}_t(x_t, \alpha) = \int \hat{\pi}_t(x_t, \alpha, x_{t-1}) dx_{t-1}, \quad (7.50)$$

because time  $t + 1$  will plug this function into the next target  $q_{t+1}$ .

Implementation meaning. Inputs: fitted three-block TT and block ordering  $(x_t, \alpha, x_{t-1})$ . Outputs: retained evaluator  $\hat{\pi}_t(x_t, \alpha)$ , normalizer  $\hat{Z}_t$ , optional marginals  $\hat{p}_t(x_t)$  and  $\hat{p}_t(\alpha)$ . Quantities to retain: retained TT/evaluator and contraction data. Operation: endpoint TT core integration. Diagnostic:  $\hat{Z}_t$  must be finite and positive, and normalizing retained marginals must integrate to one.

The main failure mode of Algorithm 1 is sign. A fitted TT is a real-valued function approximation. Truncation or interpolation can make  $\hat{\pi}_t(r_t) < 0$ , which is impossible for a density. Negative values also break transport maps and importance-sampling ratios. This motivates the squared-TT construction.

### 7.3 Why Nonnegativity Fails And Why Square-Root TT Repairs It

**Mathematical question.** This paragraph answers: why is Algorithm 1 not enough for transport maps and importance correction?

Approximating a nonnegative function by a truncated expansion does not preserve nonnegativity. The one-dimensional analogy is simple:

$$h(x) \geq 0 \quad \text{but} \quad \hat{h}(x) = \sum_{\ell=0}^{p-1} c_\ell \psi_\ell(x) \text{ may be negative.}$$

The tensor-train case has the same issue. A low-rank approximation can reduce error in a least-squares sense while still creating small negative regions.

Zhao and Cui [4]’s repair is to approximate the square root of an unnormalized density. If

$$\sqrt{\pi(r)} \approx \phi(r),$$

then

$$\phi(r)^2 \geq 0$$

for all  $r$ . A small defensive density is then added so that the proposal density is positive wherever the target might be positive.

### 7.4 Squared-TT Density, Defensive Reference, And Normalizer

**Mathematical question.** The question answered here is: how can a TT approximation be forced to define a nonnegative density whose support covers the target?

Zhao and Cui [4], Eq. (13) becomes the following. Let  $\pi(r) \geq 0$  be an unnormalized target on a domain  $\Omega \subseteq \mathbb{R}^D$ , with normalizer

$$Z = \int_{\Omega} \pi(r) dr.$$

Let  $\lambda(r)$  be a normalized reference density on  $\Omega$ ,

$$\lambda(r) \geq 0, \quad \int_{\Omega} \lambda(r) dr = 1,$$

and choose  $\tau > 0$ . Fit a TT  $\phi(r)$  to  $\sqrt{\pi(r)}$ . Define

$$\hat{\pi}(r) = \phi(r)^2 + \tau\lambda(r), \quad \hat{Z} = \int_{\Omega} \hat{\pi}(r) dr, \quad \hat{p}(r) = \frac{\hat{\pi}(r)}{\hat{Z}}. \quad (7.51)$$

Since  $\lambda$  is normalized,

$$\hat{Z} = \int_{\Omega} \phi(r)^2 dr + \tau. \quad (7.52)$$

The defensive term has two jobs. First, it keeps the approximation positive even if  $\phi$  is zero somewhere. Second, if  $\pi/\lambda$  is bounded, then

$$\frac{p(r)}{\hat{p}(r)} = \frac{\pi(r)/Z}{\hat{\pi}(r)/\hat{Z}} = \frac{\hat{Z}}{Z} \frac{\pi(r)}{\phi(r)^2 + \tau\lambda(r)} \leq \frac{\hat{Z}}{Z\tau} \frac{\pi(r)}{\lambda(r)}. \quad (7.53)$$

This is the importance-sampling reason for the defensive density: the true target is absolutely continuous with respect to the approximation when the reference covers the target support.

Zhao and Cui [4]’s Lemma 1 ties the square-root fit to density error. In the notation above, suppose

$$\|\phi - \sqrt{\pi}\|_{L^2} \leq \varepsilon, \quad \sqrt{\tau} \leq \|\phi - \sqrt{\pi}\|_{L^2}. \quad (7.54)$$

Then the square-root error of the defensive approximation satisfies

$$\|\sqrt{\hat{\pi}} - \sqrt{\pi}\|_{L^2} \leq 2\varepsilon, \quad (7.55)$$

and the normalized Hellinger distance obeys

$$D_H(\hat{p}, p) \leq \frac{\sqrt{2}}{\sqrt{Z}} \|\sqrt{\hat{\pi}} - \sqrt{\pi}\|_{L^2} \leq \frac{2\sqrt{2}\varepsilon}{\sqrt{Z}}. \quad (7.56)$$

For this note, the important lesson is local: fitting the square root in  $L^2$  controls the normalized density in Hellinger distance after normalization. The lemma does not say ranks will be small, nor does it make an adaptive implementation globally differentiable.

**Local assumptions for Lemma 1.** This note uses only the following local content: if the square-root fit is small in  $L^2$ , then the normalized defensive density is close in Hellinger distance under the stated support and finite-normalizer conditions. P18 does not re-prove the external Cui and Dolgov [1] theorem, does not claim a rank bound, and does not claim adaptive differentiability.

Implementation meaning. 1-Eq13. Inputs are a square-root target evaluator, a reference density  $\lambda$ , a defensive mass  $\tau$ , and a fitted TT  $\phi$ . Outputs are  $\hat{\pi}$ ,  $\hat{Z}$ , and  $\hat{p}$ . Quantities to retain is  $\phi$ ’s cores,  $\lambda$ ,  $\tau$ , and  $\hat{Z}$ . Diagnostics are positivity, finite  $\hat{Z}$ , support coverage by  $\lambda$ , and the defensive ratio  $\tau/\hat{Z}$ .

## 7.5 Squared-TT Marginalization And Mass Matrices

**Mathematical question.** This proposition answers: how do we marginalize a squared TT without expanding the square into a full tensor?

The square changes the marginalization algebra. Let

$$\phi(r) = H_1(r_1) \cdots H_D(r_D), \quad H_k(r_k) \in \mathbb{R}^{R_{k-1} \times R_k}.$$

For a split after coordinate  $k$ , define

$$H_{\leq k}(r_{\leq k}) = H_1(r_1) \cdots H_k(r_k) \in \mathbb{R}^{1 \times R_k},$$

and

$$H_{> k}(r_{> k}) = H_{k+1}(r_{k+1}) \cdots H_D(r_D) \in \mathbb{R}^{R_k \times 1}.$$

Then

$$\phi(r) = H_{\leq k}(r_{\leq k}) H_{> k}(r_{> k}).$$

The marginal square term is

$$\begin{aligned} \int \phi(r)^2 dr_{> k} &= \int [H_{\leq k}(r_{\leq k}) H_{> k}(r_{> k})]^2 dr_{> k} \\ &= \sum_{a=1}^{R_k} \sum_{b=1}^{R_k} H_{\leq k, a}(r_{\leq k}) \left[ \int H_{> k, a}(r_{> k}) H_{> k, b}(r_{> k}) dr_{> k} \right] H_{\leq k, b}(r_{\leq k}). \end{aligned} \quad (7.57)$$

Define the right mass matrix

$$M_{> k}[a, b] = \int H_{> k, a}(r_{> k}) H_{> k, b}(r_{> k}) dr_{> k}. \quad (7.58)$$

This matrix is positive semidefinite. If  $M_{> k} = L_{> k} L_{> k}^\top$  is a Cholesky or square-root factorization, then

$$\int \phi(r)^2 dr_{> k} = \sum_{\gamma=1}^{R_k} \left[ \sum_{a=1}^{R_k} H_{\leq k, a}(r_{\leq k}) L_{> k}[a, \gamma] \right]^2. \quad (7.59)$$

Including the defensive reference, if  $\lambda(r) = \prod_{j=1}^D \lambda_j(r_j)$ , gives

$$\hat{p}(r_{\leq k}) = \frac{\sum_{\gamma=1}^{R_k} \left[ \sum_{a=1}^{R_k} H_{\leq k, a}(r_{\leq k}) L_{> k}[a, \gamma] \right]^2 + \tau \prod_{j=1}^k \lambda_j(r_j)}{\hat{Z}}. \quad (7.60)$$

This is Zhao and Cui [4], Eq. (14) in the notation of this note.

The practical recursion starts from the right end. Set  $M_{> D} = 1$ . Moving from  $j = D$  down to 1, compute

$$M_{> j-1} = \int H_j(r_j) M_{> j} H_j(r_j)^\top dr_j. \quad (7.61)$$

In basis coefficients this integral is finite-dimensional. If

$$H_j^{a, b}(r_j) = \sum_{\ell} C_j[a, \ell, b] \psi_{j, \ell}(r_j),$$

and

$$B_j[\ell, \ell'] = \int \psi_{j, \ell}(r_j) \psi_{j, \ell'}(r_j) dr_j,$$

then

$$M_{> j-1}[a, a'] = \sum_{b, b'=1}^{R_j} \sum_{\ell, \ell'} C_j[a, \ell, b] M_{> j}[b, b'] C_j[a', \ell', b'] B_j[\ell, \ell']. \quad (7.62)$$

This formula is the bridge from the paper to code: marginalization stores mass matrices or their square roots and never enumerates a full tensor grid.

Implementation meaning. 1-Proposition2. Inputs are TT cores, basis mass matrices  $B_j$ , defensive reference factors  $\lambda_j$ , and  $\tau$ . Outputs are marginal density evaluators and mass contractions  $M_{>k}$  or their Cholesky factors. Quantities to retain is the list of mass matrices needed by later conditional CDFs. Diagnostics are symmetry error  $\|M - M^\top\|$ , Cholesky failure, negative eigenvalues beyond tolerance, and normalizer mismatch between independent contraction paths.

**Local assumptions for Proposition 2.** This note uses the finite-dimensional contraction identity for a squared TT with integrable basis products and positive semidefinite mass matrices. The note derives the contraction algebra explicitly in (7.57)–(7.65). It does not claim that every numerical Cholesky factorization is stable; that is a diagnostic obligation.

## 7.6 A Small Three-Coordinate Marginalization

Take  $D = 3$ , and suppose

$$\phi(r_1, r_2, r_3) = H_1(r_1)H_2(r_2)H_3(r_3).$$

If  $H_1 \in \mathbb{R}^{1 \times R_1}$ ,  $H_2 \in \mathbb{R}^{R_1 \times R_2}$ , and  $H_3 \in \mathbb{R}^{R_2 \times 1}$ , then marginalizing out  $r_3$  gives

$$\int \phi(r_1, r_2, r_3)^2 dr_3 = H_1(r_1)H_2(r_2) \left[ \int H_3(r_3)H_3(r_3)^\top dr_3 \right] H_2(r_2)^\top H_1(r_1)^\top. \quad (7.63)$$

The bracketed quantity is an  $R_2 \times R_2$  matrix. It is not a function of  $r_1$  or  $r_2$ ; it is a stored contraction. If we also marginalize out  $r_2$ , then

$$M_{>1} = \int H_2(r_2) \left[ \int H_3(r_3)H_3(r_3)^\top dr_3 \right] H_2(r_2)^\top dr_2, \quad (7.64)$$

and the one-coordinate marginal is

$$\int \phi(r_1, r_2, r_3)^2 dr_2 dr_3 = H_1(r_1)M_{>1}H_1(r_1)^\top. \quad (7.65)$$

This is exactly what the recursive mass formula (M5) does. The notation in the paper is compact because it assumes the reader already sees this matrix chain. The implementation sees it as repeated multiplication and integration of small matrices.

## 7.7 Algorithm 2, Fully Annotated

**Mathematical question.** Algorithm 2 answers: how does the sequential recursion change when the fitted object is the square root and the stored density is squared?

Algorithm 2 is Algorithm 1 with a square-root TT and a defensive density. At time  $t-1$ , assume the previous filter approximation is evaluable as

$$\widehat{p}_{t-1}(x_{t-1}, \alpha).$$

The pointwise unnormalized target, Zhao and Cui [4], Eq. (15), is

$$q_t(x_t, \alpha, x_{t-1}) = \widehat{p}_{t-1}(x_{t-1}, \alpha) f_\alpha(x_t | x_{t-1}) g_\alpha(y_t | x_t). \quad (7.66)$$

Fit a TT to the square root:

$$\sqrt{q_t(x_t, \alpha, x_{t-1})} \approx \phi_t(x_t, \alpha, x_{t-1}). \quad (7.67)$$

Then define Zhao and Cui [4], Eq. (16):

$$\hat{q}_t(x_t, \alpha, x_{t-1}) = \phi_t(x_t, \alpha, x_{t-1})^2 + \tau_t \lambda_t(x_t) \lambda_\alpha(\alpha) \lambda_{t-1}(x_{t-1}). \quad (7.68)$$

The approximate normalizer is

$$\hat{Z}_t = \int \hat{q}_t(x_t, \alpha, x_{t-1}) dx_t d\alpha dx_{t-1}. \quad (7.69)$$

The next normalized approximate filter is

$$\hat{p}_t(x_t, \alpha) = \frac{\int \hat{q}_t(x_t, \alpha, x_{t-1}) dx_{t-1}}{\hat{Z}_t}. \quad (7.70)$$

Everything is now nonnegative. The approximation risk has moved: the method must fit the square root accurately enough, choose a defensive term that is not too small for stability or too large for accuracy, and keep TT ranks manageable.

Implementation meaning. Inputs: previous retained density evaluator, model evaluators, reference factors  $\lambda_t, \lambda_\alpha, \lambda_{t-1}$ ,  $\tau_t$ , basis/rank/fitting controls, and observation  $y_t$ . Outputs: square-root TT  $\phi_t$ , nonnegative density evaluator  $\hat{q}_t$ , normalizer  $\hat{Z}_t$ , retained filter  $\hat{p}_t(x_t, \alpha)$ , and mass contractions for conditional maps. Quantities to retain: all of those objects. Diagnostics: square-root fit residual, defensive mass ratio,  $\hat{Z}_t > 0$ , rank growth, and Cholesky/mass-matrix health.

## 8 Fixed-Branch Likelihood Construction

The equations above explain Zhao and Cui [4]’s adaptive sequential algorithm: tensor-train approximation, squared-TT nonnegativity, conditional KR maps, particle correction, smoothing, and preconditioning. The fixed-branch likelihood specialization freezes approximation choices so that an analytical derivative can differentiate the same scalar computed by the forward pass. Zhao and Cui [4]’s adaptive rank, pivot, fitting, preconditioning, and branch choices should not be read as globally differentiable.

### Reconciliation: Paper Objects Versus Fixed-Branch Objects

The paper’s main sequential-learning density carries a learned static parameter as a random coordinate:

$$r_t^{\text{ZC}} = (x_t, \theta, x_{t-1}). \quad (8.1)$$

The fixed-branch likelihood specialization conditions on an external parameter value  $\beta$  and differentiates with respect to that external argument:

$$r_t^{\text{FB}} = (x_t, x_{t-1}; \beta). \quad (8.2)$$

The semicolon is deliberate:  $\beta$  is not integrated out by the TT density. It is the input at which the approximate likelihood is evaluated.

The exact mapping of objects is:

$$\begin{aligned} \hat{\pi}_{t-1}(x_{t-1}, \theta) &\mapsto \hat{p}_{t-1}(x_{t-1}; \beta), \\ f(x_t \mid x_{t-1}, \theta) &\mapsto f_\beta(x_t \mid x_{t-1}), \quad g(y_t \mid x_t, \theta) \mapsto g_\beta(y_t \mid x_t), \\ \phi_t(x_t, \theta, x_{t-1}) &\mapsto \phi_t(x_t, x_{t-1}; \beta), \\ \tau_t \lambda(x_t) \lambda(\theta) \lambda(x_{t-1}) &\mapsto \tau_t \lambda_t(x_t, x_{t-1}), \\ \hat{Z}_t = \int \hat{q}_t(x_t, \theta, x_{t-1}) &\mapsto \hat{Z}_t(\beta) = \int \hat{q}_t(x_t, x_{t-1}; \beta), \\ \text{adaptive ranks/pivots/maps} &\mapsto \text{fixed ranks/points/maps/sweeps.} \end{aligned} \quad (8.3)$$



The first line keeps the retained-filter role but drops the integrated  $\theta$  coordinate. The second line keeps the model evaluators, evaluated at the external  $\beta$ . The third and fourth lines keep the squared-TT and defensive-density forms in a lower-dimensional state space. The fifth line keeps the evidence-increment role. The last line is the essential change: adaptive choices are replaced by frozen branch choices before differentiation. Thus the fixed-branch section does not differentiate the adaptive Zhao and Cui [4] algorithm. It differentiates a lower-dimensional scalar constructed by applying the same squared-TT filtering algebra after freezing all discrete approximation choices. If one wants a fixed-branch derivative while also learning a static parameter as a random coordinate, one can append that coordinate to the state; the external derivative variable must still be distinguished from the integrated coordinate.

## 8.1 Fixed-Branch Objects And Array Shapes

A reader implementing a minimal method must store more than an equation. At time  $t$ , a squared-TT filter object contains:

$$\mathcal{B}_t = \{\Omega_{t,k}, \Psi_{t,k}, \psi_{t,k,\ell}, C_{t,k}, R_{t,k}, B_{t,k}, \lambda_{t,k}, \tau_t, c_t, \text{mass contractions, map data}\}.$$

Here  $\Omega_{t,k}$  is the coordinate domain,  $\Psi_{t,k}$  maps reference coordinates to physical coordinates,  $\psi_{t,k,\ell}$  are basis functions,  $C_{t,k}$  are TT cores,  $R_{t,k}$  are ranks,  $B_{t,k}$  are mass matrices,  $\lambda_{t,k}$  are reference densities,  $\tau_t$  is the defensive mass, and  $c_t$  is any log-scaling shift used during fitting.

For the adaptive Zhao and Cui [4] algorithm these objects may change with the data, with samples, and with approximation decisions. For a differentiable same-scalar target, the realized branch must be frozen before differentiation.

## 8.2 What A Core Object Looks Like

For coordinate  $k$ , store

$$C_k \in \mathbb{R}^{R_{k-1} \times p_k \times R_k}.$$

Given a scalar coordinate value  $r_k$ , first compute the basis vector

$$b_k(r_k) = (\psi_{k,0}(r_k), \dots, \psi_{k,p_k-1}(r_k))^\top.$$

Then form the matrix

$$H_k(r_k)[a, b] = \sum_{\ell=0}^{p_k-1} C_k[a, \ell, b] b_k(r_k)[\ell]. \quad (8.4)$$

Evaluation of the TT is the loop

$$v_0 = 1, \quad v_k = v_{k-1} H_k(r_k), \quad k = 1, \dots, D, \quad \hat{h}(r) = v_D. \quad (8.5)$$

The square-density object additionally stores mass contractions

$$M_{>k} = \int H_{k+1}(r_{k+1}) \cdots H_D(r_D) H_D(r_D)^\top \cdots H_{k+1}(r_{k+1})^\top dr_{k+1:D}. \quad (8.6)$$

These matrices are enough to evaluate marginals from the left. Analogous left-side mass contractions evaluate reverse-order marginals.

### 8.3 What Must Be Saved For The Next Time Step

After time  $t$ , the next step needs to evaluate

$$\hat{p}_t(x_t; \beta) \quad \text{and} \quad \partial_{\beta_i} \hat{p}_t(x_t; \beta)$$

at future fitting points. It is not enough to store a cloud of samples or only  $\hat{Z}_t$ . A fixed-branch derivative implementation stores:

$$\left\{ C_{t,k}, \dot{C}_{t,k}^{(i)}, M_{t,>k}, \dot{M}_{t,>k}^{(i)}, \hat{Z}_t, \dot{\hat{Z}}_t^{(i)}, \Psi_t, \lambda_t, \tau_t \right\}. \quad (8.7)$$

The derivative of the normalized marginal has the quotient form

$$\partial_{\beta_i} \hat{p}_t(x_t; \beta) = \frac{\partial_{\beta_i} a_t(x_t; \beta)}{\hat{Z}_t(\beta)} - \frac{a_t(x_t; \beta) \partial_{\beta_i} \hat{Z}_t(\beta)}{\hat{Z}_t(\beta)^2}, \quad (8.8)$$

where

$$a_t(x_t; \beta) = \int \hat{q}_t(x_t, x_{t-1}; \beta) dx_{t-1} \quad (8.9)$$

is the unnormalized retained numerator. Equation (8.8) is the carried-filter derivative needed by the next-step product-rule derivative.

**Lemma 1** (Carried filter object feeds the next target). *Fix the branch at time  $t$ . Suppose the stored object contains*

$$\mathcal{F}_t = \{a_t, \dot{a}_t^{(i)}, \hat{Z}_t, \dot{\hat{Z}}_t^{(i)}, \Psi_t, \lambda_t, \tau_t\}.$$

*Define  $\hat{p}_t$  and  $\dot{\hat{p}}_t^{(i)}$  by (8.8). Then the next transformed target and its derivative at any next-step fitting point are computable from  $\mathcal{F}_t$ , the model density evaluators, and their derivatives.*

*Proof.* At time  $t+1$ , the fixed-branch target has the form

$$\tilde{q}_{t+1}(z; \beta) = \hat{p}_t(x_t(z); \beta) f_\beta(x_{t+1}(z) | x_t(z)) g_\beta(y_{t+1} | x_{t+1}(z)) J_{t+1}(z).$$

The stored object evaluates  $\hat{p}_t$ . The model supplies  $f_\beta$  and  $g_\beta$ , and the branch supplies  $J_{t+1}$ . Differentiating gives the corresponding product-rule expression at time  $t+1$ . The first term uses  $\dot{\hat{p}}_t^{(i)}$ , which is the quotient derivative in (8.8). No old samples or external source objects are needed.  $\square$

### 8.4 Consolidated Fixed Least-Squares Formulation

The implementable formulation used for the fixed-branch derivative is a fixed least-squares formulation. It is not adaptive TT-cross. Adaptive TT-cross chooses pivots, interpolation sets, and ranks from the target. The construction here declares the branch first, then uses deterministic ridge least squares to fit the TT cores. In symbols,

$$\text{fixed least-squares derivative formulation} = \text{fixed points} + \text{fixed ranks} + \text{fixed sweep order} + \text{ridge least squares}. \quad (8.10)$$

This narrower construction is sufficient to define a scalar and a derivative of that same scalar.

## 8.5 Inputs, Outputs, And Stored Fields

For one time step, the inputs are

$$\mathcal{I}_t = \{y_t, \beta, \hat{p}_{t-1}, \dot{\hat{p}}_{t-1}, f_\beta, \dot{f}_\beta, g_\beta, \dot{g}_\beta, \Psi_t, Z_{\text{fit}}^{(t)}, W^{(t)}, b_k, R_k, S, \rho, c_t, \tau_t\}. \quad (8.11)$$

Here  $S$  is the declared number of sweeps and  $\rho$  is the ridge scale. The forward outputs are

$$\mathcal{O}_t = \{C_{t,k}, M_{t,>k}, M_{t,<k}, \hat{Z}_t, a_t, \hat{p}_t\}_{k=1}^D, \quad (8.12)$$

and the derivative outputs are

$$\dot{\mathcal{O}}_t = \{\dot{C}_{t,k}, \dot{M}_{t,>k}, \dot{M}_{t,<k}, \dot{\hat{Z}}_t, \dot{a}_t, \dot{\hat{p}}_t\}_{k=1}^D. \quad (8.13)$$

The frozen branch fields are

$$\mathcal{B}_{t,\text{frozen}} = \{\Psi_t, Z_{\text{fit}}^{(t)}, W^{(t)}, b_k, R_k, S, \rho, c_t, \tau_t, \lambda_t, \text{sweep direction sequence}\}. \quad (8.14)$$

The differentiable fields are recomputed when  $\beta$  changes:

$$\mathcal{B}_{t,\text{diff}}(\beta) = \{y_j, A_k, N_k, d_k, g_k, C_{t,k}, M_{t,>k}, \hat{Z}_t, a_t, \hat{p}_t\}. \quad (8.15)$$

The following shape table fixes the array contract used by the equations below:

| object                       | shape                                        |
|------------------------------|----------------------------------------------|
| $C_k$                        | $R_{k-1} \times p_k \times R_k$              |
| $L_{j,k}$                    | $1 \times R_{k-1}$                           |
| $R_{j,k}$                    | $R_k \times 1$                               |
| $A_k$                        | $n_{\text{fit}} \times (R_{k-1} p_k R_k)$    |
| $g_k = \text{vec}(C_k), d_k$ | $R_{k-1} p_k R_k$                            |
| $N_k$                        | $(R_{k-1} p_k R_k) \times (R_{k-1} p_k R_k)$ |

(8.16)

The vectorization convention is

$$\iota_k(a, \ell, b) = (a-1)p_k R_k + \ell R_k + b, \quad g_k[\iota_k(a, \ell, b)] = C_k[a, \ell, b], \quad (8.17)$$

where  $a = 1, \dots, R_{k-1}$ ,  $\ell = 0, \dots, p_k - 1$ , and  $b = 1, \dots, R_k$ . With the one-dimensional basis mass matrix

$$B_k[\ell, m] = \int b_{k,\ell}(z) b_{k,m}(z) dz, \quad (8.18)$$

the left-to-right squared-mass contraction is

$$G_k[b, b'] = \sum_{a, a', \ell, m} G_{k-1}[a, a'] C_k[a, \ell, b] B_k[\ell, m] C_k[a', m, b'], \quad G_0[1, 1] = 1. \quad (8.19)$$

For a pure squared TT, the final square mass is  $G_D[1, 1]$ ; the defensive component adds the declared  $\tau_t$ -mass separately.

## 8.6 Initialization And Sweep Order

Initialize every core with the declared ranks:

$$C_k^{(0)} \in \mathbb{R}^{R_{k-1} \times p_k \times R_k}, \quad R_0 = R_D = 1. \quad (8.20)$$

The initialization rule must be deterministic, for example

$$C_k^{(0)}[a, \ell, b] = \sigma_{\text{init}} \sin((a+1)(\ell+1)(b+1)), \quad (8.21)$$

followed by endpoint normalization. The exact deterministic rule is less important than saving it in the branch, because changing initialization changes the fixed fitting map.

One sweep consists of a left-to-right pass followed by a right-to-left pass:

$$1, 2, \dots, D, D-1, \dots, 1. \quad (8.22)$$

After  $S$  sweeps the fitted cores are

$$(C_1, \dots, C_D) = \mathcal{S}_{B_t}(\beta), \quad (8.23)$$

where  $\mathcal{S}_{B_t}$  denotes the finite composition of fixed core-update maps.

## 8.7 Forward Core Update

At fitting point  $z^{(j)}$ , define environments for core  $k$ :

$$L_{j,k} = H_1(z_1^{(j)}) \cdots H_{k-1}(z_{k-1}^{(j)}), \quad R_{j,k} = H_{k+1}(z_{k+1}^{(j)}) \cdots H_D(z_D^{(j)}). \quad (8.24)$$

For endpoint ranks this means  $L_{j,k} \in \mathbb{R}^{1 \times R_{k-1}}$  and  $R_{j,k} \in \mathbb{R}^{R_k \times 1}$ . The design row for coefficient  $(a, \ell, b)$  is

$$A_k[j, (a, \ell, b)] = L_{j,k}[1, a] b_k(z_k^{(j)})[\ell] R_{j,k}[b, 1]. \quad (8.25)$$

The local ridge system is

$$N_k g_k = d_k, \quad N_k = A_k^\top W A_k + \rho I, \quad d_k = A_k^\top W y. \quad (8.26)$$

After solving (8.26), reshape  $g_k$  into  $C_k$ :

$$C_k[a, \ell, b] = g_k[(a, \ell, b)]. \quad (8.27)$$

## 8.8 Derivative Core Update

The derivative environments are obtained by differentiating the matrix products:

$$\dot{L}_{j,k} = \sum_{r=1}^{k-1} H_1 \cdots H_{r-1} \dot{H}_r H_{r+1} \cdots H_{k-1}, \quad (8.28)$$

$$\dot{R}_{j,k} = \sum_{r=k+1}^D H_{k+1} \cdots H_{r-1} \dot{H}_r H_{r+1} \cdots H_D. \quad (8.29)$$

The derivative design row is

$$\begin{aligned} \dot{A}_k[j, (a, \ell, b)] &= \dot{L}_{j,k}[1, a] b_k(z_k^{(j)})[\ell] R_{j,k}[b, 1] \\ &\quad + L_{j,k}[1, a] b_k(z_k^{(j)})[\ell] \dot{R}_{j,k}[b, 1], \end{aligned} \quad (8.30)$$

because the basis and fitting points are frozen. If a later branch allows parameter-dependent domain maps or basis functions, additional terms are required; they are not part of this fixed branch.

The derivative normal equations are

$$\dot{N}_k = \dot{A}_k^\top W A_k + A_k^\top W \dot{A}_k, \quad \dot{d}_k = \dot{A}_k^\top W y + A_k^\top W \dot{y}. \quad (8.31)$$

Differentiating  $N_k g_k = d_k$  gives

$$N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k. \quad (8.32)$$

After solving (8.32), reshape

$$\dot{C}_k[a, \ell, b] = \dot{g}_k[(a, \ell, b)]. \quad (8.33)$$

## 8.9 Alternating-Sweep Computations

The symbols  $L_{j,k}$ ,  $R_{j,k}$ ,  $A_k$ ,  $N_k$ , and  $d_k$  are not static arrays during an alternating sweep. They are recomputed from the current cores at the moment a core is updated. Let

$$H_{j,k} = H_k(z_k^{(j)}; C_k), \quad \dot{H}_{j,k} = H_k(z_k^{(j)}; \dot{C}_k), \quad (8.34)$$

where the basis values and fitting point are fixed and only the coefficients change. For a current core collection  $C^{\text{cur}}$ , define prefix and suffix products by

$$L_{j,1}^{\text{cur}} = 1, \quad L_{j,k}^{\text{cur}} = L_{j,k-1}^{\text{cur}} H_{j,k-1}^{\text{cur}}, \quad k = 2, \dots, D, \quad (8.35)$$

and

$$R_{j,D}^{\text{cur}} = 1, \quad R_{j,k}^{\text{cur}} = H_{j,k+1}^{\text{cur}} R_{j,k+1}^{\text{cur}}, \quad k = D-1, \dots, 1. \quad (8.36)$$

The dotted prefix and suffix products are computed by the same recursions with one product-rule term added:

$$\dot{L}_{j,1}^{\text{cur}} = 0, \quad \dot{L}_{j,k}^{\text{cur}} = \dot{L}_{j,k-1}^{\text{cur}} H_{j,k-1}^{\text{cur}} + L_{j,k-1}^{\text{cur}} \dot{H}_{j,k-1}^{\text{cur}}, \quad (8.37)$$

and

$$\dot{R}_{j,D}^{\text{cur}} = 0, \quad \dot{R}_{j,k}^{\text{cur}} = \dot{H}_{j,k+1}^{\text{cur}} R_{j,k+1}^{\text{cur}} + H_{j,k+1}^{\text{cur}} \dot{R}_{j,k+1}^{\text{cur}}. \quad (8.38)$$

At a core update  $k$ , the design matrix and normal equations are built from the current environments:

$$A_k^{\text{cur}} = A_k(L_{\cdot,k}^{\text{cur}}, R_{\cdot,k}^{\text{cur}}), \quad N_k^{\text{cur}} = (A_k^{\text{cur}})^\top W A_k^{\text{cur}} + \rho I, \quad d_k^{\text{cur}} = (A_k^{\text{cur}})^\top W y. \quad (8.39)$$

Solve  $N_k^{\text{cur}} g_k^{\text{new}} = d_k^{\text{cur}}$  and replace only the current core:

$$C_k^{\text{cur}} \leftarrow C_k^{\text{new}}, \quad C_\ell^{\text{cur}} \text{ unchanged for } \ell \neq k. \quad (8.40)$$

The derivative objects for the same update are then

$$\begin{aligned} \dot{A}_k^{\text{cur}} &= \dot{A}_k(\dot{L}_{\cdot,k}^{\text{cur}}, L_{\cdot,k}^{\text{cur}}, \dot{R}_{\cdot,k}^{\text{cur}}, R_{\cdot,k}^{\text{cur}}), \\ \dot{N}_k^{\text{cur}} &= (\dot{A}_k^{\text{cur}})^\top W A_k^{\text{cur}} + (A_k^{\text{cur}})^\top W \dot{A}_k^{\text{cur}}, \\ \dot{d}_k^{\text{cur}} &= (\dot{A}_k^{\text{cur}})^\top W y + (A_k^{\text{cur}})^\top W \dot{y}. \end{aligned} \quad (8.41)$$

The derivative core is updated by

$$N_k^{\text{cur}} \dot{g}_k^{\text{new}} = \dot{d}_k^{\text{cur}} - \dot{N}_k^{\text{cur}} g_k^{\text{new}}, \quad \dot{C}_k^{\text{cur}} \leftarrow \dot{C}_k^{\text{new}}. \quad (8.42)$$

The next environment update depends on sweep direction. In a left-to-right pass, after replacing  $C_k$ , the next core is  $k + 1$ , so

$$L_{j,k+1}^{\text{cur}} = L_{j,k}^{\text{cur}} H_{j,k}^{\text{new}}, \quad \dot{L}_{j,k+1}^{\text{cur}} = \dot{L}_{j,k}^{\text{cur}} H_{j,k}^{\text{new}} + L_{j,k}^{\text{cur}} \dot{H}_{j,k}^{\text{new}}. \quad (8.43)$$

The suffix  $R_{j,k+1}^{\text{cur}}$  is already valid because it involves only cores  $k + 2, \dots, D$ . In a right-to-left pass, after replacing  $C_k$ , the next core is  $k - 1$ , so

$$R_{j,k-1}^{\text{cur}} = H_{j,k}^{\text{new}} R_{j,k}^{\text{cur}}, \quad \dot{R}_{j,k-1}^{\text{cur}} = \dot{H}_{j,k}^{\text{new}} R_{j,k}^{\text{cur}} + H_{j,k}^{\text{new}} \dot{R}_{j,k}^{\text{cur}}. \quad (8.44)$$

The prefix  $L_{j,k-1}^{\text{cur}}$  is already valid because it involves only cores  $1, \dots, k - 2$ . If an implementation caches all prefixes and suffixes for all  $k$ , then after updating  $C_k$  every cached prefix with index  $\ell > k$  and every cached suffix with index  $\ell < k$  is stale:

$$C_k \text{ updated} \implies \{L_{j,\ell} : \ell > k\} \text{ and } \{R_{j,\ell} : \ell < k\} \text{ must be recomputed before use.} \quad (8.45)$$

This is the concrete bookkeeping behind the statement that  $A_k$ ,  $N_k$ , and  $d_k$  are recomputed during each alternating sweep.

## 8.10 Stopping Rule And Failure Exits

After each full sweep compute

$$\varepsilon_s = \frac{\|A(C^{(s)}) - y\|_W}{\max\{\|y\|_W, \epsilon_{\text{abs}}\}}, \quad \Delta_s = \frac{\|C^{(s)} - C^{(s-1)}\|}{\max\{\|C^{(s-1)}\|, \epsilon_{\text{abs}}\}}. \quad (8.46)$$

The sweep stops successfully if

$$\varepsilon_s \leq \epsilon_{\text{fit}} \quad \text{and} \quad \Delta_s \leq \epsilon_{\text{sweep}}. \quad (8.47)$$

If this does not happen by  $S$  sweeps, the declared branch reports

$$\mathcal{F}_t = \text{sweep failure.} \quad (8.48)$$

| Failure condition                        | Mathematical test                                                                                | Report field       |
|------------------------------------------|--------------------------------------------------------------------------------------------------|--------------------|
| Domain miss                              | $\delta_{\text{out}} > \delta_{\text{max}}$                                                      | domain failure     |
| Rank ladder fail                         | no $R$ satisfies residual/conditioning/floor tests                                               | rank failure       |
| Ill-conditioned solve                    | $\text{cond}(N_k) > \kappa_{\text{max}}$                                                         | solve failure      |
| Sweep stagnation                         | $\varepsilon_s > \epsilon_{\text{fit}}$ or $\Delta_s > \epsilon_{\text{sweep}}$ after $S$ sweeps | sweep failure      |
| Nonpositive target for smooth derivative | $\min_j \tilde{q}_{t,j} \leq 0$ without declared floor                                           | target failure     |
| Mass failure                             | $\hat{Z}_t \leq 0$ or not finite                                                                 | normalizer failure |
| Finite-difference branch mismatch        | $\mathcal{M}_{\text{frozen}}(\beta - h) \neq \mathcal{M}_{\text{frozen}}(\beta + h)$             | branch mismatch    |

This table is part of the mathematical method. A failed branch is not a successful derivative with a warning attached; it is a declared failure of the fixed scalar.

### 8.11 One Concrete Branch, Fully Instantiated

This section gives a single fixed-branch construction. It is intentionally narrower than Zhao and Cui [4]’s adaptive code. Its purpose is to remove ambiguity. A coder can implement this construction first, then replace pieces with adaptive or higher performance choices later.

### 8.12 Coordinate Order And Domain

For the fixed-parameter likelihood case, use the two-block coordinate order

$$r_t = (x_t, x_{t-1}) \in \mathbb{R}^2$$

for a scalar state. For vector states, concatenate coordinates as

$$r_t = (x_{t,1}, \dots, x_{t,m_x}, x_{t-1,1}, \dots, x_{t-1,m_x}).$$

Choose a fixed interval for each coordinate,

$$r_{t,k} \in [\ell_{t,k}, u_{t,k}],$$

and map from reference coordinate  $z_k \in [-1, 1]$  by

$$r_{t,k} = a_{t,k} + h_{t,k} z_k, \quad a_{t,k} = \frac{u_{t,k} + \ell_{t,k}}{2}, \quad h_{t,k} = \frac{u_{t,k} - \ell_{t,k}}{2}. \quad (8.49)$$

The Jacobian is constant:

$$J_t = \prod_{k=1}^D h_{t,k}. \quad (8.50)$$

The branch stores  $\ell_{t,k}, u_{t,k}, a_{t,k}, h_{t,k}$ . In the same-scalar derivative these are constants and  $\dot{J}_t = 0$ .

### Domain-Selection Policy

The fixed domain must be chosen before differentiation and then saved. For coordinate  $k$ , let a pilot location and scale be

$$\mu_{t,k}^{\text{pilot}}, \quad s_{t,k}^{\text{pilot}} > 0. \quad (8.51)$$

They may come from prior moments, transition moments, a pilot particle cloud, or a Gaussian bridge. A deterministic interval is

$$[\ell_{t,k}, u_{t,k}] = [\mu_{t,k}^{\text{pilot}} - \gamma s_{t,k}^{\text{pilot}}, \mu_{t,k}^{\text{pilot}} + \gamma s_{t,k}^{\text{pilot}}], \quad (8.52)$$

with expansion factor  $\gamma > 0$ . For Gaussian-like pilot tails,

$$\gamma = 4 \quad \Rightarrow \quad \Pr(|Z| > \gamma) \approx 6.3 \times 10^{-5}. \quad (8.53)$$

If the model supplies hard bounds, intersect with those bounds:

$$[\ell_{t,k}, u_{t,k}] \leftarrow [\ell_{t,k}, u_{t,k}] \cap [\ell_k^{\text{model}}, u_k^{\text{model}}]. \quad (8.54)$$

The out-of-domain diagnostic at fitting or proposal points is

$$\delta_{\text{out}} = \frac{1}{N} \sum_{j=1}^N \mathbf{1}\{r_{t,k}^{(j)} \notin [\ell_{t,k}, u_{t,k}] \text{ for some } k\}. \quad (8.55)$$

The branch is accepted only if

$$\delta_{\text{out}} \leq \delta_{\text{max}}. \quad (8.56)$$

If this fails, enlarge the domain and rebuild the branch. Do not enlarge the domain during a same-branch derivative check:

$$[\ell_{t,k}, u_{t,k}](\beta_0 + h) = [\ell_{t,k}, u_{t,k}](\beta_0) = [\ell_{t,k}, u_{t,k}](\beta_0 - h). \quad (8.57)$$

### 8.13 Basis, Points, Weights, Ranks

Use normalized Legendre basis with the same degree  $p$  in every coordinate:

$$b_k(z_k) = \left( \sqrt{\frac{1}{2}} P_0(z_k), \sqrt{\frac{3}{2}} P_1(z_k), \dots, \sqrt{\frac{2p-1}{2}} P_{p-1}(z_k) \right)^\top. \quad (8.58)$$

Use fixed Halton-type fitting points. Let  $p_k^*$  be the  $k$ -th prime. If

$$j = \sum_{m=0}^M d_m (p_k^*)^m, \quad d_m \in \{0, \dots, p_k^* - 1\},$$

define

$$\text{radinv}_{p_k^*}(j) = \sum_{m=0}^M d_m (p_k^*)^{-(m+1)}, \quad z_k^{(j)} = 2 \text{radinv}_{p_k^*}(j) - 1. \quad (8.59)$$

Use  $j = 1, \dots, N_{\text{fit}}$ , weights

$$W = \frac{1}{N_{\text{fit}}} I, \quad (8.60)$$

and fixed ranks

$$R_0 = R_D = 1, \quad R_1, \dots, R_{D-1} \text{ declared before fitting.} \quad (8.61)$$

No pivot, point, rank, or domain choice is allowed to change during a same-scalar derivative check.

### 8.14 Deterministic Rank Ladder Before Branch Freezing

The rank vector is selected by a small deterministic ladder before it becomes part of the fixed branch. Let

$$\mathcal{R}_{\text{ladder}} = \{(1, \dots, 1), (2, \dots, 2), (4, \dots, 4), (6, \dots, 6), (8, \dots, 8)\}. \quad (8.62)$$

For every candidate  $R \in \mathcal{R}_{\text{ladder}}$ , keep the same domain, basis, fitting points, weights, shift, defensive density, and preconditioner. Fit the square-root TT and compute

$$\varepsilon_{\text{rel}}(R) = \frac{\|A_R g_R - y\|_W}{\max\{\|y\|_W, \epsilon_{\text{abs}}\}}, \quad \kappa_R = \text{cond}(A_R^\top W A_R + \epsilon_{\text{ridge}} I). \quad (8.63)$$

The defensive-mass fraction is

$$\chi_R = \frac{\tau_t}{e^{-c_t} \int \phi_{t,R}(z)^2 dz + \tau_t}. \quad (8.64)$$

The candidate passes if

$$\varepsilon_{\text{rel}}(R) \leq \epsilon_{\text{fit}}, \quad \kappa_R \leq \kappa_{\text{max}}, \quad \chi_R \leq \chi_{\text{max}}. \quad (8.65)$$



The declared branch uses the first passing rank:

$$R_\star = \min\{R \in \mathcal{R}_{\text{ladder}} : R \text{ satisfies (8.65)}\}. \quad (8.66)$$

If no candidate passes, the correct outcome is branch failure,

$$\mathcal{F}_t = \text{rank-ladder failure}, \quad (8.67)$$

not an unrecorded adaptive rank change. Once  $R_\star$  is selected, all finite-difference evaluations and derivative equations keep  $R_\star$  fixed.

### 8.15 Defensive Reference, Mass, And Shift

Use the uniform reference on  $[-1, 1]^D$ :

$$\lambda(z) = 2^{-D}. \quad (8.68)$$

Choose a defensive floor  $\epsilon_\tau > 0$  and set

$$\tau_t = 2^D \epsilon_\tau. \quad (8.69)$$

Then  $\tau_t \lambda(z) = \epsilon_\tau$ . Choose the stabilizing shift at the base parameter  $\beta_0$ :

$$c_t = - \max_{1 \leq j \leq N_{\text{fit}}} \log \tilde{q}_t(z^{(j)}; \beta_0). \quad (8.70)$$

The fitted square-root target is

$$y_j(\beta) = \exp(c_t/2) \sqrt{\tilde{q}_t(z^{(j)}; \beta)}. \quad (8.71)$$

For same-branch finite differences,  $c_t$  is reused for  $\beta_0 + h$  and  $\beta_0 - h$ . Recomputing  $c_t$  changes the scalar.

### 8.16 Boxed Convention: Shifted Fit, Unshifted Density

The concrete branch uses this convention everywhere downstream.

|                            |                                                                                                                 |        |
|----------------------------|-----------------------------------------------------------------------------------------------------------------|--------|
| Fitted TT:                 | $\phi_t(z; \beta) \approx e^{c_t/2} \sqrt{\tilde{q}_t(z; \beta)},$                                              | (8.72) |
| Approximate joint density: | $\hat{q}_t(z; \beta) = e^{-c_t} \phi_t(z; \beta)^2 + \tau_t \lambda_t(z),$                                      |        |
| Retained numerator:        | $a_t(z_t; \beta) = \int [e^{-c_t} \phi_t(z_t, z_{t-1}; \beta)^2 + \tau_t \lambda_t(z_t, z_{t-1})] \, dz_{t-1},$ |        |
| Normalizer:                | $\hat{Z}_t(\beta) = \int \hat{q}_t(z; \beta) \, dz = e^{-c_t} \int \phi_t(z; \beta)^2 \, dz + \tau_t.$          |        |

Every carried filter, proposition, derivative, and finite-difference check uses (8.72). If an implementation instead absorbs  $e^{-c_t/2}$  into the TT cores, then it must remove the explicit  $e^{-c_t}$  factors everywhere. Mixing the two conventions changes the scalar.

## Stabilization Defaults

A minimal deterministic stabilization policy is

$$\lambda_t(z) = \prod_{k=1}^D \frac{1}{2} \mathbf{1}\{-1 \leq z_k \leq 1\}, \quad (8.73)$$

so  $\lambda_t$  has units of inverse reference-volume and integrates to one. Choose a density floor  $\epsilon_{\text{floor}} > 0$  in transformed target units and set

$$\tau_t = 2^D \epsilon_{\text{floor}}. \quad (8.74)$$

Then the defensive density contribution is

$$\tau_t \lambda_t(z) = \epsilon_{\text{floor}} \quad \text{on } [-1, 1]^D. \quad (8.75)$$

The fitting target uses the floored value

$$\tilde{q}_{t,j}^{\text{floor}} = \max\{\tilde{q}_t(z^{(j)}; \beta), \epsilon_{\text{floor}}\}. \quad (8.76)$$

The shift is computed once at the base parameter:

$$c_t = -\max_j \log \tilde{q}_{t,j}^{\text{floor}}(\beta_0). \quad (8.77)$$

The ridge parameter is scaled to the design matrix:

$$\rho_t = \epsilon_{\text{ridge}} \frac{\text{trace}(A_k^\top W A_k)}{\dim(g_k)} \quad \text{for each core update,} \quad (8.78)$$

or fixed to a declared positive scalar if this adaptive scaling is not used. The stabilization diagnostic vector is

$$\mathcal{D}_{\text{stab}} = \left\{ \frac{\tau_t}{\widehat{Z}_t}, \frac{\#\{j : \tilde{q}_{t,j} \leq \epsilon_{\text{floor}}\}}{N_{\text{fit}}}, \kappa(A_k^\top W A_k + \rho_t I), |c_t| \right\}. \quad (8.79)$$

A branch fails stabilization if the defensive mass dominates,

$$\frac{\tau_t}{\widehat{Z}_t} > \delta_\tau, \quad (8.80)$$

or if too many fitting targets are floored,

$$\frac{\#\{j : \tilde{q}_{t,j} \leq \epsilon_{\text{floor}}\}}{N_{\text{fit}}} > \delta_{\text{floor}}. \quad (8.81)$$

These defaults are reproducibility defaults, not production validation.

| constant                         | first-pass value                                             | role                                                                     |
|----------------------------------|--------------------------------------------------------------|--------------------------------------------------------------------------|
| $\lambda_t(z)$                   | $2^{-D} \mathbf{1}\{z \in [-1, 1]^D\}$                       | defensive reference density in reference coordinates                     |
| $\tau_t$                         | $2^D \epsilon_{\text{floor}}$                                | defensive mass giving floor $\tau_t \lambda_t = \epsilon_{\text{floor}}$ |
| $c_t$                            | $-\max_j \log \tilde{q}_{t,j}^{\text{floor}}(\beta_0)$       | base-parameter square-root scaling shift                                 |
| $\epsilon_{\text{floor}}$        | $10^{-12}$                                                   | minimum transformed target value used before square roots                |
| $\epsilon_{\text{ridge}}$        | $10^{-10}$                                                   | relative ridge scale for local normal equations                          |
| $\rho_t$                         | $\epsilon_{\text{ridge}} \text{trace}(A^\top W A) / \dim(g)$ | ridge used in each local normal equation                                 |
| $\delta_\tau$                    | $10^{-2}$                                                    | maximum defensive-mass fraction                                          |
| $\delta_{\text{floor}}$          | $10^{-2}$                                                    | maximum fraction of floored fitting values                               |
| $\gamma$                         | 4                                                            | pilot-scale multiplier for reference domains                             |
| $N_{\text{fit}}$                 | $\max\{20 D p R_\star^2, 200\}$                              | number of fixed fitting points                                           |
| $S_{\text{max}}$                 | 4                                                            | bidirectional sweep count for the deterministic fit                      |
| $\epsilon_{\text{fit}}$          | $10^{-4}$                                                    | relative weighted square-root fit-residual tolerance                     |
| $\epsilon_{\text{sweep}}$        | $10^{-5}$                                                    | relative core-change tolerance across sweeps                             |
| $\epsilon_Z$                     | $10^{-8}$                                                    | relative normalizer cross-check tolerance                                |
| $\epsilon_{\text{norm}}$         | $10^{-8}$                                                    | retained-filter normalization tolerance                                  |
| $\epsilon_Q, \epsilon_{\dot{Q}}$ | $10^{-6}, 10^{-5}$                                           | compressed retained-object validation tolerances                         |
| $\epsilon_{\text{abs}}$          | $10^{-14}$                                                   | absolute scale preventing division by zero in residuals                  |
| $\epsilon_{\text{root}}$         | $10^{-10}$                                                   | conditional CDF inversion tolerance                                      |
| $N_{\text{root}}$                | 80                                                           | maximum bisection steps for inverse conditionals                         |
| $\chi_{\text{max}}$              | $10^{-2}$                                                    | defensive-fraction veto in the rank ladder                               |
| $\kappa_{\text{max}}$            | $10^{12}$                                                    | normal-equation conditioning veto                                        |
| $H_{\text{FD}}$                  | $\{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}$                     | finite-difference step schedule                                          |

(8.82)

The numerical values in (8.82) are reproducibility defaults for a first mathematical implementation. They are not empirical evidence that the method is production-ready.

### 8.17 Initialization And Sweep Order

Initialize every core to zero except the constant channel:

$$C_k[1, 0, 1] = 1, \quad k = 2, \dots, D,$$

and

$$C_1[1, 0, 1] = \bar{y}, \quad \bar{y} = \frac{1}{N_{\text{fit}}} \sum_{j=1}^{N_{\text{fit}}} y_j.$$

If a rank is larger than one, all non-first rank channels start at zero. This plain initialization gives a constant initial approximation and avoids a random initialization branch.

One bidirectional sweep means update cores  $1, 2, \dots, D$ , then  $D, D-1, \dots, 1$ . Fix the number of bidirectional sweeps  $S$ . The core update is the ridge solve (7.29). The branch stores the sweep order,  $S$ ,  $\rho$ , and the final cores.

## 8.18 Forward-Step Object Flow

The fixed-branch forward step is a dependency chain among mathematical objects, not executable code. The dependency flow is

$$\begin{aligned}
\text{reference points} &\longrightarrow z^{(j)} \text{ from (8.59),} \\
\text{physical points} &\longrightarrow r^{(j)} = \Psi_t(z^{(j)}), \\
\text{target values} &\longrightarrow \tilde{q}_{t,j} = \hat{p}_{t-1}(x_{t-1}^{(j)}; \beta) f_\beta(x_t^{(j)} | x_{t-1}^{(j)}) g_\beta(y_t | x_t^{(j)}) J_t, \\
\text{square-root fit values} &\longrightarrow y_j = \exp(c_t/2) \sqrt{\tilde{q}_{t,j}}, \\
\text{initial cores} &\longrightarrow \text{constant-channel initialization,} \\
\text{fixed sweep objects} &\longrightarrow L_{j,k}, R_{j,k}, A_{j,k}, N_k, d_k, C_k, \\
\text{mass objects} &\longrightarrow M_{>k}, R_t, \hat{Z}_t, \\
\text{carried objects} &\longrightarrow a_t, \hat{p}_t.
\end{aligned} \tag{8.83}$$

The fixed sweep environments are

$$L_{j,k} = H_1(z_1^{(j)}) \cdots H_{k-1}(z_{k-1}^{(j)}),$$

$$R_{j,k} = H_{k+1}(z_{k+1}^{(j)}) \cdots H_D(z_D^{(j)}),$$

with  $A_{j,k}$  defined by (7.27) and the core update defined by (7.29). After fitting, square-mass contractions use (7.61)–(7.62):

$$R_t = \int \phi_t(z)^2 dz, \quad \hat{Z}_t = e^{-c_t} R_t + \tau_t.$$

The carried numerator is the old-state marginal

$$a_t(z_t; \beta) = \int [e^{-c_t} \phi_t(z_t, z_{t-1}; \beta)^2 + \tau_t \lambda_t(z_t, z_{t-1})] dz_{t-1},$$

and the carried filter is  $a_t/\hat{Z}_t$ .

## 8.19 Conditional-Map Inversion Protocol

For a conditional coordinate  $r_k$ , compute

$$F_k(s | r_{<k}) = \int_{\ell_k}^s \hat{p}(r'_k | r_{<k}) dr'_k.$$

To sample from the conditional, given  $u \in (0, 1)$ , solve

$$F_k(s | r_{<k}) - u = 0, \quad s \in [\ell_k, u_k].$$

Use bisection as the baseline because it only needs monotonicity. Fix a tolerance  $\varepsilon_{\text{root}}$  and maximum iterations  $N_{\text{root}}$ . The branch stores both. The bisection update is:

$$m = \frac{a+b}{2}, \quad \text{if } F_k(m | r_{<k}) < u \text{ set } a = m, \text{ else set } b = m.$$

Stop when  $b - a \leq \varepsilon_{\text{root}}$  or the iteration count reaches  $N_{\text{root}}$ . Raise a failure diagnostic if the final residual  $|F_k(m | r_{<k}) - u|$  exceeds the declared tolerance.

## 8.20 Fixed-Branch TT Filtering Recursion

The fixed-branch filtering variant chooses all discrete approximation objects before differentiating. It is narrower than the full Zhao and Cui [4] adaptive code, but it is the variant for which an analytical derivative can be stated without changing the scalar under the derivative.

## 8.21 End-To-End Fixed-Branch Squared-TT Filter

The following boxed algorithm gathers the complete mathematical recurrence in one place. It is written as equations and stored objects rather than executable program text.

**Inputs.** Model densities  $p_\beta(x_0)$ ,  $f_\beta(x_t | x_{t-1})$ ,  $g_\beta(y_t | x_t)$ ; observations  $y_{1:T}$ ; reference domains  $\mathcal{Z}_t$ ; domain maps  $\Psi_t$ ; optional preconditioners  $T_t$ ; basis families  $b_{t,k}$ ; candidate ranks  $\mathcal{R}$ ; fitting points  $Z_{\text{fit},t}$ ; weights  $W_t$ ; defensive densities  $\lambda_t$ ; constants  $c_t, \tau_t, \epsilon_{\text{floor}}, \epsilon_{\text{ridge}}$ .

**Initialization.** Set  $\hat{p}_0$  to the declared initial density in retained coordinates and set  $\hat{\ell}_0 = 0$ . Store the initial domain map and, if differentiating,  $\hat{p}_0$ .

**For**  $t = 1, \dots, T$ . Form the transformed target  $\tilde{q}_t(z; \beta) = \hat{p}_{t-1}(z_{t-1}; \beta) f_\beta(x_t | x_{t-1}) g_\beta(y_t | x_t) J_t(z)$ . Evaluate  $y_j = e^{c_t/2} \sqrt{\max\{\tilde{q}_t(z^{(j)}), \epsilon_{\text{floor}}\}}$ . Fit a square-root TT  $\phi_t$  by the declared fixed-rank sweep. Build  $\hat{q}_t = e^{-c_t} \phi_t^2 + \tau_t \lambda_t$ . Contract mass matrices to compute  $\hat{Z}_t = e^{-c_t} \int \phi_t^2 dz + \tau_t$ . Set  $a_t(z_t) = \int \hat{q}_t(z_t, z_{t-1}) dz_{t-1}$  and  $\hat{p}_t(z_t) = a_t(z_t) / \hat{Z}_t$ . Update  $\hat{\ell}_t = \hat{\ell}_{t-1} + \log \hat{Z}_t$ . If requested, build conditional KR maps from marginal ratios. If differentiating, run the derivative pass through target values, sweep environments, core solves, mass contractions, normalizer, and retained filter.

**Outputs.**  $\hat{\ell}_T(\beta; B)$ , retained filters  $\hat{p}_t$ , normalizers  $\hat{Z}_t$ , square-root cores  $C_{t,k}$ , mass matrices, optional KR maps, and dotted counterparts for all objects used by the next target.

**Invariants.**  $\hat{q}_t \geq 0$ ;  $0 < \hat{Z}_t < \infty$ ;  $\int \hat{p}_t = 1$ ; fixed branch  $B$  is identical for all finite difference evaluations; all densities are evaluated in their declared coordinate systems.

**Failure exits.** Reject the branch if target values underflow beyond the declared floor, the least-squares solve is ill-conditioned after ridge regularization, the chosen rank fails the rank ladder, defensive mass dominates the fitted mass, KR conditional CDFs are not monotone enough for inversion, a preconditioner is singular, or finite differences do not use the same branch.

(8.84)

## 8.22 Boxed Algorithm: Fixed-Branch Filter And Derivative Together

For implementation work, the forward scalar and its derivative should be read as one coupled mathematical recurrence. The branch is fixed first:

$$B = \{\Psi_t, T_t, b_{t,k}, Z_{\text{fit},t}, W_t, R_{t,k}, S_t, \rho_{t,k}, c_t, \tau_t, \lambda_t, \text{sweep order}\}_{t,k}. \quad (8.85)$$

For a parameter component  $\beta_i$ , the coupled algorithm is

**Inputs.**  $\beta$ , model density evaluators  $(p_\beta, f_\beta, g_\beta)$ , derivative evaluators  $(\partial_i p_\beta, \partial_i f_\beta, \partial_i g_\beta)$ , observations  $y_{1:T}$ , and fixed branch  $B$ .

**Initial retained object.** Set  $P_0$  to the initial retained density representation and set  $\dot{P}_0 = \partial_i P_0$ . If the initial law is independent of  $\beta_i$ , then  $\dot{P}_0 = 0$ .

**For**  $t = 1, \dots, T$ , **target values.** At each fitting point  $z^{(j)}$ , evaluate

$$\tilde{q}_{t,j} = P_{t-1}(z_{t-1}^{(j)}) f_\beta(x_t^{(j)} | x_{t-1}^{(j)}) g_\beta(y_t | x_t^{(j)}) J_t(z^{(j)}),$$

and

$$\dot{\tilde{q}}_{t,j} = \dot{P}_{t-1} f_\beta g_\beta J_t + P_{t-1} (\partial_i f_\beta) g_\beta J_t + P_{t-1} f_\beta (\partial_i g_\beta) J_t + P_{t-1} f_\beta g_\beta \dot{J}_t.$$

In the fixed-domain specialization,  $\dot{J}_t = 0$ .

**Square-root values.** With the frozen shift and floor,

$$y_j = e^{c_t/2} \sqrt{\max(\tilde{q}_{t,j}, \epsilon_{\text{floor}})}, \quad \dot{y}_j = \frac{e^{c_t/2} \dot{\tilde{q}}_{t,j}}{2\sqrt{\tilde{q}_{t,j}}}$$

when the value is not floored; if the branch declares a nonsmooth floor hit, the derivative check is failed rather than silently differentiated through max.

**Fixed sweeps.** For each declared core update, recompute  $L_{j,k}, R_{j,k}, A_k, N_k, d_k$ , solve  $N_k g_k = d_k$ , reshape  $g_k \mapsto C_{t,k}$ , then recompute the dotted environments and solve

$$N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k.$$

The recomputation order is the one in (8.34)–(8.45); in particular, cached prefixes and suffixes on the wrong side of an updated core are stale until recomputed.

**Mass and normalizer.** Contract the fitted cores to obtain  $R_t = \int \phi_t^2 dz$  and

$$\dot{R}_t = \int 2\phi_t \dot{\phi}_t dz, \quad \hat{Z}_t = e^{-c_t} R_t + \tau_t, \quad \dot{\hat{Z}}_t = e^{-c_t} \dot{R}_t.$$

**Retained filter.** Compute retained numerator matrices or compressed retained evaluators  $(Q_t, \dot{Q}_t)$ , normalize

$$P_t = Q_t / \hat{Z}_t, \quad \dot{P}_t = \dot{Q}_t / \hat{Z}_t - Q_t \dot{\hat{Z}}_t / \hat{Z}_t^2,$$

and save the evaluator pair needed by time  $t + 1$ .

**Scalar update.**

$$\hat{\ell}_t = \hat{\ell}_{t-1} + \log \hat{Z}_t, \quad \dot{\hat{\ell}}_t = \dot{\hat{\ell}}_{t-1} + \dot{\hat{Z}}_t / \hat{Z}_t.$$

**Outputs.**  $\hat{\ell}_T(\beta; B)$ ,  $\partial_i \hat{\ell}_T(\beta; B)$ , retained evaluator pairs  $(P_t, \dot{P}_t)$ , fitted cores  $(C_{t,k}, \dot{C}_{t,k})$ , normalizers, diagnostics, and failure status.

**Invariants and exits.** The branch  $B$  is unchanged; every retained filter integrates to one; every normalizer is finite and positive; every derivative solve uses the same forward normal matrix. Exit with failure if floor hits are nonsmooth, ranks do not pass the declared ladder, conditioned solves exceed  $\kappa_{\max}$ , defensive mass dominates, retained compression breaks normalization, or finite differences do not use the same branch.

There is one important notational separation. In the Zhao and Cui [4] learning algorithm, the unknown parameter  $\alpha$  may be carried as a random variable inside the posterior density. In an HMC-style likelihood calculation, the parameter is instead an external argument that is varied by the sampler. To avoid using the same symbol for both roles, this section writes the external differentiated parameter as

$$\beta \in \mathbb{R}^{m_\beta}.$$

The fixed-branch likelihood variant conditions on a trial  $\beta$  and filters only the latent state. Its unnormalized update is

$$q_t(x_t, x_{t-1}; \beta) = \hat{p}_{t-1}(x_{t-1}; \beta) f_\beta(x_t | x_{t-1}) g_\beta(y_t | x_t).$$

If one also wants to carry a learned static parameter as a random coordinate, that coordinate can be appended to the state; the derivative below is still with respect to the external  $\beta$ , not with respect to an integration variable.

Let  $z \in [-1, 1]^D$  be reference coordinates and

$$r = \Psi_t(z) = (x_t, x_{t-1}).$$

Let  $J_t(z) = |\det \nabla \Psi_t(z)|$ . The transformed target is

$$\tilde{q}_t(z; \beta) = \hat{p}_{t-1}(x_{t-1}; \beta) f_\beta(x_t | x_{t-1}) g_\beta(y_t | x_t) J_t(z). \quad (8.87)$$

The branch fixes points  $z^{(j)}$ , weights  $w_j$ , basis functions, ranks, and a core-construction algorithm. The fitted square-root TT is

$$\phi_t(z; \beta) = G_{t,1}(z_1; \beta) \cdots G_{t,D}(z_D; \beta). \quad (8.88)$$

The fixed-branch approximate density is

$$\hat{q}_t(z; \beta) = e^{-c_t} \phi_t(z; \beta)^2 + \tau_t \lambda_t(z), \quad \int \lambda_t(z) dz = 1. \quad (8.89)$$

Its normalizer is

$$\hat{Z}_t(\beta) = \int \hat{q}_t(z; \beta) dz = e^{-c_t} \int \phi_t(z; \beta)^2 dz + \tau_t. \quad (8.90)$$

This is exactly the boxed convention (8.72). The next filter object is the normalized marginal

$$\hat{p}_t(x_t; \beta) = \frac{\int \hat{q}_t(z_t, z_{t-1}; \beta) dz_{t-1}}{\hat{Z}_t(\beta)} |\det \nabla z_t / \nabla x_t|. \quad (8.91)$$

The Jacobian factor appears if the filter is evaluated in physical coordinates. If the whole recursion is implemented in reference coordinates, it is absorbed into the stored domain maps.

**Proposition 1** (Fixed-branch recursion is a normalized approximate filter). *Fix a time horizon  $T$ , model densities  $f_\beta, g_\beta$ , initial density  $p_\beta(x_0)$ , and fixed branch objects for each time step. Suppose every  $\hat{q}_t(\cdot; \beta)$  defined by (8.89) is nonnegative, integrable, and has  $0 < \hat{Z}_t(\beta) < \infty$ . Define  $\hat{p}_t$  by (8.91), starting from the fixed initial approximation. Then each  $\hat{p}_t$  is a normalized density on the retained state variable  $x_t$ . Moreover, the recursion is exactly the Bayes filtering recursion applied to the declared approximate joint density  $\hat{q}_t$ , not to the exact joint density.*

*Proof.* Nonnegativity follows from  $\phi_t^2 \geq 0$ ,  $\tau_t > 0$ , and  $\lambda_t \geq 0$ . Since  $\widehat{Z}_t = \int \widehat{q}_t$  is positive and finite, the normalized joint density

$$\widehat{p}_t^{\text{joint}}(z) = \widehat{q}_t(z) / \widehat{Z}_t$$

integrates to one. The retained filter is the marginal of this normalized joint density:

$$\int \widehat{p}_t(z_t) dz_t = \int \left[ \int \widehat{p}_t^{\text{joint}}(z_t, z_{t-1}) dz_{t-1} \right] dz_t = \int \widehat{p}_t^{\text{joint}}(z) dz = 1.$$

The change-of-variables factor in physical coordinates is the standard Jacobian factor for density transformation, so normalization is preserved. The construction uses the exact Bayes filtering algebra with  $q_t$  replaced by the declared approximation  $\widehat{q}_t$ . No step asserts that  $\widehat{q}_t = q_t$ , so the result is a normalized approximate filtering recursion rather than an exact-posterior theorem.  $\square$

## 9 Fixed-Branch Derivative Theory

The preceding sections preserved the source-order Zhao and Cui [4] annotation and the fixed-branch likelihood setup. The next part expands the derivative path in small mathematical steps. It deliberately starts again from scalar normalizers before returning to tensor trains, because the key claim is that the gradient differentiates the same approximate scalar computed by the fixed branch.

The merge has one important reading convention. The Zhao and Cui [4] annotation uses  $\theta$  for a random static parameter inside the sequential posterior when the paper studies joint state and parameter learning. The fixed-branch gradient part uses  $\beta$  for an external parameter value at which evaluates an approximate likelihood. These are different roles:

$\theta$  can be an integration coordinate in Zhao and Cui [4],  $\beta$  is the differentiated argument of  $\widehat{\ell}_T$ . (9.1)

The fixed-branch scalar therefore does not claim to differentiate the adaptive Zhao and Cui [4] algorithm with respect to every internal coordinate. It freezes the realized numerical branch and differentiates the scalar produced by that branch as  $\beta$  varies:

$$B = \{\text{domains, points, basis, ranks, shifts, sweeps, tolerances}\}, \quad \beta \mapsto \widehat{\ell}_T(\beta; B). \quad (9.2)$$

This distinction is what makes the next derivation useful rather than merely formal. Without it, changing  $\beta$  could also change ranks, pivots, maps, or fitting points; then two nearby likelihood evaluations would not be values of one mathematical function. With it, the fitted core values still change with  $\beta$ , but they change through the same stored fitting equations:

$$C_{t,k}(\beta; B) = \text{output of the fixed core-fitting rule at parameter } \beta. \quad (9.3)$$

Thus the derivative below is neither a derivative of frozen numbers nor a derivative of an adaptive search procedure. It is the derivative of the declared fixed-branch approximation. This is the identity a finite-difference diagnostic and a later implementation must obey.

### 9.1 Fixed-Branch Gradient Motivation

The filtering algorithm produces, at each time  $t$ , an approximate normalizing constant

$$\widehat{Z}_t(\beta) > 0. \quad (9.4)$$



The parameter  $\beta$  is the model parameter at which we evaluate the approximate likelihood. The total approximate log likelihood is

$$\hat{\ell}_T(\beta) = \sum_{t=1}^T \log \hat{Z}_t(\beta). \quad (9.5)$$

Formally,  $\hat{Z}_t(\beta)$  plays the role of a partition-function-like normalizer: it is an integral of a nonnegative weight. Here  $\hat{Z}_t(\beta)$  is also an integral of a nonnegative weight:

$$\hat{Z}_t(\beta) = \int \hat{q}_t(z; \beta) \, dz. \quad (9.6)$$

The variable  $z$  is a numerical coordinate, usually a transformed coordinate on a fixed box. The function  $\hat{q}_t$  is not the exact Bayesian density. It is the squared tensor-train approximation used by the numerical filter.

The panel cares about the gradient because parameter inference algorithms need to know how the approximate log likelihood changes when  $\beta$  changes:

$$\nabla_{\beta} \hat{\ell}_T(\beta). \quad (9.7)$$

The delicate point is not the derivative of log. The delicate point is that  $\hat{Z}_t$  is produced by a complicated approximation algorithm. If the approximation algorithm changes its grid, ranks, pivots, or scaling shifts when  $\beta$  changes, then the two likelihood evaluations may not be evaluations of the same mathematical formula. A classical analytical derivative can only differentiate one declared formula.

That is why this note uses a fixed branch.

**Fixed branch.** Before differentiating, freeze the computational choices: coordinate domain, fitting points, basis functions, tensor-train ranks, sweep order, ridge parameter, defensive density, stabilizing shifts, and the rule for constructing every fitted core. Do not freeze the fitted core values themselves. Those core values are outputs of the fixed fitting equations, so they remain functions of  $\beta$ . Then differentiate the scalar defined by the saved choices and by the core values obtained from the same saved fitting equations.

The goal is therefore:

$$\text{differentiate the scalar actually computed by the saved forward pass.} \quad (9.8)$$

The goal is not:

$$\text{differentiate every possible adaptive version of the algorithm.} \quad (9.9)$$

## 9.2 Warmup 1: A Normalizing Constant

Start with the simplest possible object. Let

$$Z(\beta) = \int_{\Omega} q(z; \beta) \, dz, \quad q(z; \beta) \geq 0, \quad 0 < Z(\beta) < \infty. \quad (9.10)$$

The normalized density is

$$p(z; \beta) = \frac{q(z; \beta)}{Z(\beta)}. \quad (9.11)$$

The log normalizing constant is

$$L(\beta) = \log Z(\beta). \quad (9.12)$$

Assume that  $q(z; \beta)$  is differentiable in  $\beta$ , and that we may move the derivative inside the integral:

$$\frac{\partial}{\partial \beta} \int_{\Omega} q(z; \beta) dz = \int_{\Omega} \frac{\partial q(z; \beta)}{\partial \beta} dz. \quad (9.13)$$

A sufficient condition is that, near the parameter value of interest, the absolute derivative is bounded by an integrable function:

$$\left| \frac{\partial q(z; \beta)}{\partial \beta} \right| \leq m(z), \quad \int_{\Omega} m(z) dz < \infty. \quad (9.14)$$

Then the derivative of the log normalizer is

$$\begin{aligned} \frac{\partial L(\beta)}{\partial \beta} &= \frac{\partial}{\partial \beta} \log Z(\beta) \\ &= \frac{1}{Z(\beta)} \frac{\partial Z(\beta)}{\partial \beta} \\ &= \frac{1}{Z(\beta)} \int_{\Omega} \frac{\partial q(z; \beta)}{\partial \beta} dz. \end{aligned} \quad (9.15)$$

This is the first idea in the whole gradient. Everything later is only a way to compute the numerator and denominator in (9.15) for the tensor-train approximation.

For the sequential filter, the same formula is used at every time:

$$\frac{\partial \hat{\ell}_T(\beta)}{\partial \beta} = \sum_{t=1}^T \frac{1}{\hat{Z}_t(\beta)} \frac{\partial \hat{Z}_t(\beta)}{\partial \beta}. \quad (9.16)$$

### 9.3 Warmup 2: A Squared Approximation

The fixed-branch squared tensor-train approximation has the form

$$\hat{q}(z; \beta) = e^{-c} \phi(z; \beta)^2 + \tau \lambda(z). \quad (9.17)$$

Here:

$$\begin{aligned} c & \text{ is a frozen stabilizing shift,} \\ \tau > 0 & \text{ is a frozen defensive mass,} \\ \lambda(z) \geq 0 & \text{ is a frozen reference density with } \int \lambda = 1, \\ \phi(z; \beta) & \text{ is the fitted square-root function.} \end{aligned} \quad (9.18)$$

Because  $c, \tau, \lambda$  are frozen, the only object in (9.17) that changes with  $\beta$  is  $\phi$ . Differentiating (9.17) gives

$$\begin{aligned} \dot{\hat{q}}(z; \beta) &= \frac{\partial}{\partial \beta} [e^{-c} \phi(z; \beta)^2 + \tau \lambda(z)] \\ &= e^{-c} \frac{\partial}{\partial \beta} \phi(z; \beta)^2 + \frac{\partial}{\partial \beta} \tau \lambda(z) \\ &= 2e^{-c} \phi(z; \beta) \dot{\phi}(z; \beta). \end{aligned} \quad (9.19)$$

The dot means  $\partial/\partial \beta$ .

The corresponding normalizer is

$$\begin{aligned}
\widehat{Z}(\beta) &= \int \widehat{q}(z; \beta) \, dz \\
&= e^{-c} \int \phi(z; \beta)^2 \, dz + \tau \int \lambda(z) \, dz \\
&= e^{-c} \int \phi(z; \beta)^2 \, dz + \tau.
\end{aligned} \tag{9.20}$$

Using (9.19), its derivative is

$$\begin{aligned}
\dot{\widehat{Z}}(\beta) &= \int \dot{\widehat{q}}(z; \beta) \, dz \\
&= 2e^{-c} \int \phi(z; \beta) \dot{\phi}(z; \beta) \, dz.
\end{aligned} \tag{9.21}$$

Substitution into (9.15) gives

$$\frac{\partial}{\partial \beta} \log \widehat{Z}(\beta) = \frac{2e^{-c} \int \phi(z; \beta) \dot{\phi}(z; \beta) \, dz}{e^{-c} \int \phi(z; \beta)^2 \, dz + \tau}. \tag{9.22}$$

If  $c$ ,  $\tau$ , or  $\lambda$  changed with  $\beta$ , then (9.19) would have extra terms:

$$\dot{\widehat{q}} = -\dot{c} e^{-c} \phi^2 + 2e^{-c} \phi \dot{\phi} + \dot{\tau} \lambda + \tau \dot{\lambda}. \tag{9.23}$$

The fixed-branch derivative does not use (9.23). It uses the fixed-branch formula (9.19). This is not a minor technicality. It is the mathematical line between differentiating one saved scalar and differentiating a changing adaptive procedure.

### 9.4 Warmup 3: Two Coordinates, Rank One

Before a full tensor train, consider a two-coordinate rank-one function:

$$\phi(z_1, z_2; \beta) = h_1(z_1; \beta) h_2(z_2; \beta). \tag{9.24}$$

The square integral is

$$\begin{aligned}
R(\beta) &= \iint \phi(z_1, z_2; \beta)^2 \, dz_1 \, dz_2 \\
&= \iint h_1(z_1; \beta)^2 h_2(z_2; \beta)^2 \, dz_1 \, dz_2 \\
&= \left[ \int h_1(z_1; \beta)^2 \, dz_1 \right] \left[ \int h_2(z_2; \beta)^2 \, dz_2 \right].
\end{aligned} \tag{9.25}$$

Define

$$R_1(\beta) = \int h_1(z_1; \beta)^2 \, dz_1, \quad R_2(\beta) = \int h_2(z_2; \beta)^2 \, dz_2. \tag{9.26}$$

Then

$$R(\beta) = R_1(\beta) R_2(\beta). \tag{9.27}$$

Differentiate by the product rule:

$$\dot{R}(\beta) = \dot{R}_1(\beta) R_2(\beta) + R_1(\beta) \dot{R}_2(\beta). \tag{9.28}$$

Each one-dimensional derivative is

$$\dot{R}_1(\beta) = 2 \int h_1(z_1; \beta) \dot{h}_1(z_1; \beta) dz_1, \quad (9.29)$$

$$\dot{R}_2(\beta) = 2 \int h_2(z_2; \beta) \dot{h}_2(z_2; \beta) dz_2. \quad (9.30)$$

Combining (9.28)–(9.30),

$$\begin{aligned} \dot{R}(\beta) = & 2 \left[ \int h_1 \dot{h}_1 dz_1 \right] \left[ \int h_2^2 dz_2 \right] \\ & + 2 \left[ \int h_1^2 dz_1 \right] \left[ \int h_2 \dot{h}_2 dz_2 \right]. \end{aligned} \quad (9.31)$$

This is the simplest version of the tensor-train mass derivative: when one factor changes, keep the other factors fixed and sum the contributions.

## 9.5 Warmup 5: A Fixed Linear Solve

The fitted tensor-train cores are obtained from finite-dimensional least-squares updates. One such update can be written

$$N(\beta)g(\beta) = d(\beta). \quad (9.32)$$

Here  $g$  is the vector of unknown core coefficients in the current update,  $N$  is the regularized normal matrix, and  $d$  is the right-hand side. Differentiate both sides:

$$\frac{\partial}{\partial \beta} \{N(\beta)g(\beta)\} = \frac{\partial d(\beta)}{\partial \beta}. \quad (9.33)$$

Use the product rule:

$$\dot{N}(\beta)g(\beta) + N(\beta)\dot{g}(\beta) = \dot{d}(\beta). \quad (9.34)$$

Move the first term to the right:

$$N(\beta)\dot{g}(\beta) = \dot{d}(\beta) - \dot{N}(\beta)g(\beta). \quad (9.35)$$

If  $N(\beta)$  is nonsingular, then

$$\dot{g}(\beta) = N(\beta)^{-1} \left[ \dot{d}(\beta) - \dot{N}(\beta)g(\beta) \right]. \quad (9.36)$$

In computation, one should not form the inverse. One solves the linear system (9.35) using the same numerical linear algebra used for the forward solve.

The ridge term is important because the normal matrix has the form

$$N = A^\top W A + \rho I, \quad \rho > 0. \quad (9.37)$$

If  $W$  is positive semidefinite, then

$$v^\top N v = v^\top A^\top W A v + \rho v^\top v \geq \rho \|v\|^2 \quad (9.38)$$

for all  $v$ . Hence  $N$  is positive definite and nonsingular. Without nonsingularity, the derivative of the solve may not be well defined.

## 9.6 The Fixed-Branch Forward Pass

At time  $t$ , the fixed branch constructs a nonnegative approximation

$$\hat{q}_t(z; \beta) = e^{-c_t} \phi_t(z; \beta)^2 + \tau_t \lambda_t(z) \quad (9.39)$$

and its normalizer

$$\hat{Z}_t(\beta) = \int \hat{q}_t(z; \beta) dz. \quad (9.40)$$

The branch is the collection of structural choices that define the scalar as a function of  $\beta$ . These choices include the domain, fitting points, basis, ranks, sweep order, shift, defensive term, and ridge parameter. The branch does not mean that the fitted numerical core values are held constant when  $\beta$  changes. Instead,

$$C_{t,k}(\beta) = \text{the core obtained by the saved fitting rule at parameter } \beta. \quad (9.41)$$

The derivative pass computes  $\dot{C}_{t,k}(\beta)$ . This distinction matters: freezing the core values would usually make the normalizer nearly constant and would not assess the derivative of the fitted approximation.

| Forward object           | What is stored                                | Why it is needed later                                          |
|--------------------------|-----------------------------------------------|-----------------------------------------------------------------|
| Domain map $\Psi_t$      | Coordinate intervals and Jacobian             | Evaluates the same transformed target at every derivative point |
| Fitting points $z^{(j)}$ | Deterministic point array and weights         | Defines the least-squares equations                             |
| Basis $b_k$              | Basis family, degree, mass matrices           | Evaluates cores and mass contractions                           |
| Ranks $R_k$              | Fixed rank vector                             | Fixes core dimensions                                           |
| Sweep order              | Core update order and number of sweeps        | Defines a finite composition of solve maps                      |
| Shift $c_t$              | One frozen scalar                             | Defines the same scaled square-root target                      |
| Defensive term           | $\tau_t, \lambda_t$                           | Ensures nonnegative density floor                               |
| Cores $C_{t,k}(\beta)$   | Parameter-dependent fitted coefficient arrays | Evaluate $\phi_t$ and $\hat{q}_t$                               |
| Mass contractions        | $M_{t,>k}(\beta), M_{t,<k}(\beta)$            | Compute integrals and marginals                                 |
| Normalizer               | $\hat{Z}_t$                                   | Adds $\log \hat{Z}_t$ to the scalar                             |
| Carried filter           | Numerator $a_t$ and normalized $\hat{p}_t$    | Feeds the next time step                                        |

The transformed target fitted at time  $t$  is

$$\tilde{q}_t(z; \beta) = \hat{p}_{t-1}(x_{t-1}(z); \beta) f_\beta(x_t(z) \mid x_{t-1}(z)) g_\beta(y_t \mid x_t(z)) J_t(z). \quad (9.42)$$

The fitted square-root values are

$$y_j(\beta) = e^{c_t/2} \sqrt{\tilde{q}_t(z^{(j)}; \beta)}. \quad (9.43)$$

The tensor train is

$$\phi_t(z; \beta) = H_{t,1}(z_1; \beta) H_{t,2}(z_2; \beta) \cdots H_{t,D}(z_D; \beta), \quad (9.44)$$

where  $H_{t,k}(z_k) \in \mathbb{R}^{R_{k-1} \times R_k}$  and  $R_0 = R_D = 1$ .

The retained numerator for the next filter is

$$a_t(z_t; \beta) = \int \hat{q}_t(z_t, z_{t-1}; \beta) dz_{t-1}. \quad (9.45)$$

The retained normalized filter is

$$\hat{p}_t(z_t; \beta) = \frac{a_t(z_t; \beta)}{\hat{Z}_t(\beta)}. \quad (9.46)$$

If the filter is reported in physical coordinates, multiply by the appropriate change-of-variables factor. The derivative logic is unchanged if the domain map is frozen.

## 9.7 Gradient Teaching Layer: What Is Differentiated

The derivative section is easier to read after separating three objects:

the exact statistical model,      the adaptive approximation policy,      one fixed approximate scalar. (9.47)

Only the third object is differentiated here. The scalar is

$$\boxed{\hat{\ell}_T(\beta; B) = \sum_{t=1}^T \log \hat{Z}_t(\beta; B_t)} \quad (9.48)$$

where  $B = (B_1, \dots, B_T)$  is a saved branch. The branch contains frozen structural fields:

$$B_{t,\text{frozen}} = \{\Omega_t, \Psi_t, Z_{\text{fit}}^{(t)}, W^{(t)}, b_{t,k}, R_{t,k}, c_t, \tau_t, \rho_t, S_t, \text{sweep order}\}. \quad (9.49)$$

The differentiable fields are recomputed from those frozen fields:

$$B_{t,\text{diff}}(\beta) = \{\tilde{q}_{t,j}(\beta), y_{t,j}(\beta), A_{t,k}(\beta), N_{t,k}(\beta), d_{t,k}(\beta), C_{t,k}(\beta), \hat{Z}_t(\beta)\}. \quad (9.50)$$

Thus the derivative is neither

$$\frac{\partial}{\partial \beta} \hat{\ell}_T^{\text{adapt}}(\beta) \quad (9.51)$$

nor the derivative obtained by freezing all fitted core numbers. It is

$$\frac{\partial}{\partial \beta} \hat{\ell}_T(\beta; B) \quad \text{with } B_{\text{frozen}} \text{ fixed and } B_{\text{diff}}(\beta) \text{ recomputed.} \quad (9.52)$$

| Forward scalar path                                                       | Derivative path                                                                                                                            |
|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| $\tilde{q}_{t,j} = \hat{p}_{t-1,j} f_j g_j J_j$                           | $\dot{\tilde{q}}_{t,j} = J_j (\dot{\hat{p}}_{t-1,j} f_j g_j \hat{p}_{t-1,j} \dot{f}_j g_j \hat{p}_{t-1,j} \dot{f}_j \dot{g}_j)$            |
| $y_j = e^{c_t/2} \sqrt{\tilde{q}_{t,j}}$                                  | $\dot{y}_j = e^{c_t/2} \dot{\tilde{q}}_{t,j} / (2\sqrt{\tilde{q}_{t,j}})$                                                                  |
| $A_k \rightarrow N_k = A_k^\top W A_k + \rho I, \quad d_k = A_k^\top W y$ | $\dot{A}_k \rightarrow \dot{N}_k = \dot{A}_k^\top W A_k + A_k^\top W \dot{A}_k, \quad \dot{d}_k = \dot{A}_k^\top W y + A_k^\top W \dot{y}$ |
| $N_k g_k = d_k$                                                           | $N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k$                                                                                                |
| $g_k \rightarrow C_k \rightarrow M_{>k} \rightarrow \hat{Z}_t$            | $\dot{g}_k \rightarrow \dot{C}_k \rightarrow \dot{M}_{>k} \rightarrow \dot{\hat{Z}}_t$                                                     |
| $\hat{p}_t = a_t / \hat{Z}_t$                                             | $\dot{\hat{p}}_t = \dot{a}_t / \hat{Z}_t - a_t \dot{\hat{Z}}_t / \hat{Z}_t^2$                                                              |

A finite-dimensional analog shows why the solve derivative is the correct object. Suppose the entire approximation at one time were a two-coefficient fit

$$\phi(z; \beta) = g_0(\beta)b_0(z) + g_1(\beta)b_1(z), \quad (9.53)$$

with fixed fitting matrix  $A$  and target values  $y(\beta)$ . Ridge least squares gives

$$g(\beta) = (A^\top W A + \rho I)^{-1} A^\top W y(\beta). \quad (9.54)$$

If  $A, W, \rho$  are frozen, then

$$\dot{g}(\beta) = (A^\top W A + \rho I)^{-1} A^\top W \dot{y}(\beta). \quad (9.55)$$

If  $A$  depends on already-updated neighboring cores, the same equation gains the  $\dot{A}$  and  $\dot{N}$  terms shown in the table. The conceptual point is unchanged: the derivative propagates through the fitted coefficients, not around them.

The derivative is useful for three reasons:

$$\begin{aligned} &\text{finite-difference verification of the declared scalar,} \\ &\text{local sensitivity analysis of the approximation,} \\ &\text{possible gradient-based exploration of a fixed approximation.} \end{aligned} \quad (9.56)$$

It is deliberately narrow for one reason:

$$B(\beta + h) \neq B(\beta) \quad \Rightarrow \quad \widehat{\ell}_T(\beta + h; B(\beta + h)) \text{ is a different function evaluation path.} \quad (9.57)$$

That adaptive derivative may be studied separately, but it is not what is proved in Proposition 2.

## 9.8 The Fixed-Branch Derivative Pass

The derivative pass computes the derivative of each forward object that depends on  $\beta$ . It does not create a new branch.

| Derivative object                 | Forward object differentiated | Formula source                       |
|-----------------------------------|-------------------------------|--------------------------------------|
| $\tilde{q}_{t,j}$                 | target value at fitting point | product rule                         |
| $\dot{y}_j$                       | square-root fit value         | chain rule                           |
| $\dot{A}_k, \dot{N}_k, \dot{d}_k$ | least-squares system          | environment derivatives              |
| $\dot{C}_{t,k}$                   | fitted core                   | fixed linear solve                   |
| $\dot{M}_{t,>k}$                  | mass contraction              | product rule in core indices         |
| $\dot{\hat{Z}}_t$                 | normalizer                    | squared-integral derivative          |
| $\dot{a}_t$                       | carried numerator             | marginal contraction derivative      |
| $\dot{\hat{p}}_t$                 | normalized carried filter     | quotient rule                        |
| $\dot{\tilde{q}}_{t+1}$           | next target                   | product rule using $\dot{\hat{p}}_t$ |

## 9.9 Target And Square-Root Target

Differentiate (9.42). At fitting point  $z^{(j)}$ , abbreviate

$$\hat{p}_{t-1,j} = \hat{p}_{t-1}(x_{t-1}^{(j)}; \beta), \quad f_j = f_\beta(x_t^{(j)} | x_{t-1}^{(j)}), \quad g_j = g_\beta(y_t | x_t^{(j)}). \quad (9.58)$$

With  $J_{t,j}$  frozen, the derivative is

$$\dot{\tilde{q}}_{t,j} = J_{t,j} \left[ \dot{\hat{p}}_{t-1,j} f_j g_j + \hat{p}_{t-1,j} \dot{f}_j g_j + \hat{p}_{t-1,j} f_j \dot{g}_j \right]. \quad (9.59)$$

If the domain map depended on  $\beta$ , an additional  $\hat{p} f g \dot{J}$  term would appear. That is outside the fixed branch used here.

From (9.43),

$$\dot{y}_j = e^{c_t/2} \frac{1}{2\sqrt{\tilde{q}_{t,j}}} \dot{\tilde{q}}_{t,j}, \quad (9.60)$$

provided  $\tilde{q}_{t,j} > 0$ . A numerical implementation should use a positive floor if the target is close to zero. If such a floor is used, the flooring rule becomes part of the declared scalar. For example,

$$y_j(\beta) = e^{c_t/2} \sqrt{\max\{\tilde{q}_{t,j}(\beta), \varepsilon_{\text{floor}}\}} \quad (9.61)$$

has derivative zero through the floor whenever  $\tilde{q}_{t,j} < \varepsilon_{\text{floor}}$ . The formulas below describe the smooth positive-target case; a floored implementation must differentiate the floored scalar it actually evaluates.

## 9.10 Design Matrix And Core Solve

The design row is the first place where tensor-train notation can look more mysterious than it is. Fix one sample point  $z^{(j)}$  and one core  $k$ . All cores except core  $k$  are treated as the known left and right environments for this local least-squares update. Write

$$L_{j,k}[a] = \left[ H_1(z_1^{(j)}) \cdots H_{k-1}(z_{k-1}^{(j)}) \right]_{1a}, \quad (9.62)$$

$$R_{j,k}[b] = \left[ H_{k+1}(z_{k+1}^{(j)}) \cdots H_D(z_D^{(j)}) \right]_{b1}. \quad (9.63)$$

The local core evaluated at  $z_k^{(j)}$  is

$$H_k(z_k^{(j)})[a, b] = \sum_{\ell=1}^{m_k} C_k[a, \ell, b] b_k(z_k^{(j)})[\ell], \quad (9.64)$$

where  $m_k$  is the number of one-dimensional basis functions in coordinate  $k$ . The value of the tensor train at this one point is therefore

$$\begin{aligned} \phi_t(z^{(j)}; \beta) &= \sum_{a=1}^{R_{k-1}} \sum_{b=1}^{R_k} L_{j,k}[a] H_k(z_k^{(j)})[a, b] R_{j,k}[b] \\ &= \sum_{a=1}^{R_{k-1}} \sum_{\ell=1}^{m_k} \sum_{b=1}^{R_k} L_{j,k}[a] b_k(z_k^{(j)})[\ell] R_{j,k}[b] C_k[a, \ell, b]. \end{aligned} \quad (9.65)$$



Equation (9.65) is the whole reason the local fit is a linear least-squares problem. At the fixed sample point, the unknown numbers are the entries of the core  $C_k$ . Every coefficient multiplying  $C_k[a, \ell, b]$  is already known from the branch and from the other cores.

If  $g_k = \text{vec}(C_k)$  stacks entries in the order  $(a, \ell, b)$ , the row coefficient of  $g_k[a, \ell, b]$  is

$$A_{j,k}[a, \ell, b] = L_{j,k}[a] b_k(z_k^{(j)})[\ell] R_{j,k}[b]. \quad (9.66)$$

The compact Kronecker expression in (9.72) is just (9.66) written without three sums:

$$A_{j,k} = R_{j,k}^\top \otimes b_k(z_k^{(j)})^\top \otimes L_{j,k}. \quad (9.67)$$

To differentiate the row, keep the basis value fixed and differentiate the two environments:

$$\begin{aligned} \dot{A}_{j,k}[a, \ell, b] &= \dot{L}_{j,k}[a] b_k(z_k^{(j)})[\ell] R_{j,k}[b] \\ &\quad + L_{j,k}[a] b_k(z_k^{(j)})[\ell] \dot{R}_{j,k}[b]. \end{aligned} \quad (9.68)$$

Equation (9.73) is (9.68) written in the same Kronecker shorthand. If the basis or fitting point moved with  $\beta$ , there would be an additional term with  $\dot{b}_k(z_k^{(j)})$ . That term is absent because the branch keeps basis and fitting points fixed.

For one core  $k$ , the least-squares relation has rows

$$\phi_t(z^{(j)}; \beta) = A_{j,k}(\beta) g_k(\beta), \quad (9.69)$$

where  $g_k = \text{vec}(C_{t,k})$ . Let

$$L_{j,k} = H_1(z_1^{(j)}) \cdots H_{k-1}(z_{k-1}^{(j)}), \quad (9.70)$$

$$R_{j,k} = H_{k+1}(z_{k+1}^{(j)}) \cdots H_D(z_D^{(j)}). \quad (9.71)$$

Then

$$A_{j,k} = R_{j,k}^\top \otimes b_k(z_k^{(j)})^\top \otimes L_{j,k}. \quad (9.72)$$

Because basis values and fitting points are frozen,

$$\dot{A}_{j,k} = \dot{R}_{j,k}^\top \otimes b_k(z_k^{(j)})^\top \otimes L_{j,k} + R_{j,k}^\top \otimes b_k(z_k^{(j)})^\top \otimes \dot{L}_{j,k}. \quad (9.73)$$

The environments are differentiated by product rule:

$$\dot{L}_{j,1} = 0, \quad \dot{L}_{j,k+1} = \dot{L}_{j,k} H_k(z_k^{(j)}) + L_{j,k} \dot{H}_k(z_k^{(j)}), \quad (9.74)$$

$$\dot{R}_{j,D} = 0, \quad \dot{R}_{j,k-1} = \dot{H}_k(z_k^{(j)}) R_{j,k} + H_k(z_k^{(j)}) \dot{R}_{j,k}. \quad (9.75)$$

The local core matrix derivative is

$$\dot{H}_k(z_k^{(j)})[a, b] = \sum_{\ell} \dot{C}_k[a, \ell, b] b_k(z_k^{(j)})[\ell]. \quad (9.76)$$

The fixed ridge solve is

$$N_k g_k = d_k, \quad N_k = A_k^\top W A_k + \rho I, \quad d_k = A_k^\top W y. \quad (9.77)$$

Using (9.35),

$$N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k, \quad (9.78)$$

where

$$\dot{N}_k = \dot{A}_k^\top W A_k + A_k^\top W \dot{A}_k, \quad (9.79)$$

$$\dot{d}_k = \dot{A}_k^\top W y + A_k^\top W \dot{y}. \quad (9.80)$$

The derivative update is therefore another linear solve with the same left-hand matrix  $N_k$ . This is why the fixed branch is implementable: the forward pass and derivative pass use the same stored computational path.

### 9.11 Mass Contraction And Normalizer

The full right-mass recursion is the same idea as the rank- $R$  warmup in (9.72)–(9.80). In the warmup,  $M_1[r, s]$  stored the inner products of the first-coordinate factors and  $M_2[r, s]$  stored the inner products of the second-coordinate factors. In the full tensor train,  $M_{>j}[b, b']$  plays the role of the already-integrated right factor,  $B_j[\ell, \ell']$  is the one-coordinate basis inner product, and the two copies of  $C_j$  are the two factors coming from squaring  $\phi_t$ .

The right mass contraction is

$$M_{>j-1}[a, a'] = \sum_{b, b', \ell, \ell'} C_j[a, \ell, b] M_{>j}[b, b'] C_j[a', \ell', b'] B_j[\ell, \ell']. \quad (9.81)$$

Here  $B_j$  is the basis mass matrix. It is frozen. Differentiate (9.81):

$$\begin{aligned} \dot{M}_{>j-1}[a, a'] &= \sum_{b, b', \ell, \ell'} \dot{C}_j[a, \ell, b] M_{>j}[b, b'] C_j[a', \ell', b'] B_j[\ell, \ell'] \\ &\quad + \sum_{b, b', \ell, \ell'} C_j[a, \ell, b] \dot{M}_{>j}[b, b'] C_j[a', \ell', b'] B_j[\ell, \ell'] \\ &\quad + \sum_{b, b', \ell, \ell'} C_j[a, \ell, b] M_{>j}[b, b'] \dot{C}_j[a', \ell', b'] B_j[\ell, \ell']. \end{aligned} \quad (9.82)$$

Start from

$$M_{>D} = 1, \quad \dot{M}_{>D} = 0. \quad (9.83)$$

Then (9.81)–(9.82) move from the right end of the chain to the left end. The final square integral is

$$R_t(\beta) = \int \phi_t(z; \beta)^2 dz = M_{>0}. \quad (9.84)$$

Its derivative is

$$\dot{R}_t(\beta) = \dot{M}_{>0}. \quad (9.85)$$

By (17)–(18),

$$\hat{Z}_t(\beta) = e^{-c_t} R_t(\beta) + \tau_t, \quad (9.86)$$

$$\dot{\hat{Z}}_t(\beta) = e^{-c_t} \dot{R}_t(\beta). \quad (9.87)$$

### 9.12 Carried Filter And Next Time Step

The carried numerator is

$$a_t(z_t; \beta) = \int \hat{q}_t(z_t, z_{t-1}; \beta) dz_{t-1}. \quad (9.88)$$

Its derivative is

$$\dot{a}_t(z_t; \beta) = \int \dot{\hat{q}}_t(z_t, z_{t-1}; \beta) dz_{t-1}. \quad (9.89)$$

For the squared part of the approximation, this marginal is another tensor contraction. To make the formula concrete, suppose the retained coordinate is the last coordinate  $z_D = z_t$ , and the previous-state coordinates are  $z_1, \dots, z_{D-1}$ . First integrate out the previous-state coordinates by the same left-to-right mass recursion:

$$M_{<0} = 1, \quad M_{<j}[b, b'] = \sum_{a, a', \ell, \ell'} M_{<j-1}[a, a'] C_j[a, \ell, b] C_j[a', \ell', b'] B_j[\ell, \ell'] \quad (9.90)$$

for  $j = 1, \dots, D - 1$ . The derivative of this marginal environment is

$$\begin{aligned}\dot{M}_{<j}[b, b'] &= \sum_{a, a', \ell, \ell'} \dot{M}_{<j-1}[a, a'] C_j[a, \ell, b] C_j[a', \ell', b'] B_j[\ell, \ell'] \\ &+ \sum_{a, a', \ell, \ell'} M_{<j-1}[a, a'] \dot{C}_j[a, \ell, b] C_j[a', \ell', b'] B_j[\ell, \ell'] \\ &+ \sum_{a, a', \ell, \ell'} M_{<j-1}[a, a'] C_j[a, \ell, b] \dot{C}_j[a', \ell', b'] B_j[\ell, \ell'].\end{aligned}\tag{9.91}$$

Start with  $\dot{M}_{<0} = 0$ . After the recursion reaches  $D - 1$ , the retained-coordinate numerator is the function

$$\begin{aligned}a_t(z_D; \beta) &= e^{-c_t} \sum_{b, b', \ell, \ell'} M_{<D-1}[b, b'] C_D[b, \ell, 1] C_D[b', \ell', 1] b_D(z_D)[\ell] b_D(z_D)[\ell'] \\ &+ \tau_t \int \lambda_t(z_D, z_{1:D-1}) dz_{1:D-1}.\end{aligned}\tag{9.92}$$

Its derivative, with  $c_t, \tau_t, \lambda_t$  and the basis fixed, is

$$\begin{aligned}\dot{a}_t(z_D; \beta) &= e^{-c_t} \sum_{b, b', \ell, \ell'} \dot{M}_{<D-1}[b, b'] C_D[b, \ell, 1] C_D[b', \ell', 1] b_D(z_D)[\ell] b_D(z_D)[\ell'] \\ &+ e^{-c_t} \sum_{b, b', \ell, \ell'} M_{<D-1}[b, b'] \dot{C}_D[b, \ell, 1] C_D[b', \ell', 1] b_D(z_D)[\ell] b_D(z_D)[\ell'] \\ &+ e^{-c_t} \sum_{b, b', \ell, \ell'} M_{<D-1}[b, b'] C_D[b, \ell, 1] \dot{C}_D[b', \ell', 1] b_D(z_D)[\ell] b_D(z_D)[\ell'].\end{aligned}\tag{9.93}$$

Equations (9.90)–(9.93) are the coding recipe behind the short integral identity (9.89). If the retained state is not the last coordinate, reorder the coordinates or use left and right environments around the retained block; the same product-rule pattern applies.

The normalized carried filter is

$$\hat{p}_t(z_t; \beta) = \frac{a_t(z_t; \beta)}{\hat{Z}_t(\beta)}.\tag{9.94}$$

Differentiate by quotient rule:

$$\dot{\hat{p}}_t(z_t; \beta) = \frac{\dot{a}_t(z_t; \beta)}{\hat{Z}_t(\beta)} - \frac{a_t(z_t; \beta) \dot{\hat{Z}}_t(\beta)}{\hat{Z}_t(\beta)^2}.\tag{9.95}$$

This is the crucial bridge between time steps. At time  $t + 1$ , the target contains  $\hat{p}_t$ , so its derivative contains  $\dot{\hat{p}}_t$ . Without saving the carried-filter derivative, the next-step gradient is incomplete.

### 9.13 Concrete One-Coordinate Carried-Filter Storage

The quotient formula (9.95) is a mathematical identity. A later implementation also needs a concrete stored object. In the minimal two-coordinate fixed branch, use the normalized Legendre basis  $b_1(z) \in \mathbb{R}^p$  on the retained coordinate and store the carried numerator as a coefficient matrix:

$$Q_t[\ell, m] = e^{-c_t} \sum_{r, s=1}^R C_1[0, \ell, r] S_2[r, s] C_1[0, m, s] + \tau_t \mathbf{1}_{\ell=0} \mathbf{1}_{m=0}, \quad \text{shape}(Q_t) = (p, p).\tag{9.96}$$

The retained numerator is then the quadratic form

$$a_t(z_1; \beta) = b_1(z_1)^\top Q_t b_1(z_1). \quad (9.97)$$

The defensive term in (9.96) gives  $\tau_t/2$  because the normalized Legendre constant satisfies  $b_1(z)[0] = 1/\sqrt{2}$ , hence

$$\tau_t b_1(z_1)[0]^2 = \tau_t/2. \quad (9.98)$$

The derivative coefficient matrix is

$$\begin{aligned} \dot{Q}_t[\ell, m] &= e^{-c_t} \sum_{r,s} \dot{C}_1[0, \ell, r] S_2[r, s] C_1[0, m, s] \\ &\quad + e^{-c_t} \sum_{r,s} C_1[0, \ell, r] \dot{S}_2[r, s] C_1[0, m, s] \\ &\quad + e^{-c_t} \sum_{r,s} C_1[0, \ell, r] S_2[r, s] \dot{C}_1[0, m, s], \quad \text{shape}(\dot{Q}_t) = (p, p). \end{aligned} \quad (9.99)$$

The normalized carried filter is stored as

$$P_t = \frac{Q_t}{\hat{Z}_t}, \quad \dot{P}_t = \frac{\dot{Q}_t}{\hat{Z}_t} - \frac{Q_t \dot{\hat{Z}}_t}{\hat{Z}_t^2}, \quad \text{shape}(P_t) = \text{shape}(\dot{P}_t) = (p, p). \quad (9.100)$$

For a query vector  $z^{\text{query}} \in [-1, 1]^M$ , build

$$B^{\text{query}}[j, \ell] = b_1(z_j^{\text{query}})[\ell], \quad \text{shape}(B^{\text{query}}) = (M, p). \quad (9.101)$$

The carried evaluator returns

$$\begin{aligned} \hat{p}_t^{\text{ref}}[j] &= \sum_{\ell, m} B^{\text{query}}[j, \ell] P_t[\ell, m] B^{\text{query}}[j, m], \\ \dot{\hat{p}}_t^{\text{ref}}[j] &= \sum_{\ell, m} B^{\text{query}}[j, \ell] \dot{P}_t[\ell, m] B^{\text{query}}[j, m], \end{aligned} \quad \text{shape}(\hat{p}_t^{\text{ref}}) = \text{shape}(\dot{\hat{p}}_t^{\text{ref}}) = (M, ). \quad (9.102)$$

At the next time step, the query points are the previous-state coordinate of the fitting table:

$$z_j^{\text{query}} = Z_{\text{fit}}[j, 2], \quad p_j = \hat{p}_{t-1}^{\text{ref}}[j], \quad \dot{p}_j = \dot{\hat{p}}_{t-1}^{\text{ref}}[j]. \quad (9.103)$$

The stored object is therefore not an informal function name. It is a pair of coefficient matrices and a declared query rule. This is what makes the next-step target derivative mechanically reproducible.

## Multidimensional Retained-Filter Contract

For retained state dimension  $m > 1$ , let the retained block have coordinates

$$z_t = (z_{t,1}, \dots, z_{t,m}), \quad b_t(z_t) = b_{t,1}(z_{t,1}) \otimes \dots \otimes b_{t,m}(z_{t,m}). \quad (9.104)$$

If coordinate  $j$  uses  $p_j$  basis functions, then

$$b_t(z_t) \in \mathbb{R}^{p_{\text{ret}}}, \quad p_{\text{ret}} = \prod_{j=1}^m p_j. \quad (9.105)$$

The retained numerator is represented as a quadratic form

$$a_t(z_t; \beta) = b_t(z_t)^\top Q_t(\beta) b_t(z_t), \quad Q_t \in \mathbb{R}^{p_{\text{ret}} \times p_{\text{ret}}}. \quad (9.106)$$

The derivative object has the same shape:

$$\dot{a}_t(z_t; \beta) = b_t(z_t)^\top \dot{Q}_t(\beta) b_t(z_t), \quad \dot{Q}_t \in \mathbb{R}^{p_{\text{ret}} \times p_{\text{ret}}}. \quad (9.107)$$

Here is how  $Q_t$  is obtained from the inherited squared-TT contractions. Split the square-root TT coordinates into retained coordinates  $z_t$  and integrated coordinates  $w$ :

$$\phi_t(z_t, w) = \sum_{\alpha, \gamma} L_\alpha(z_t) S_{\alpha, \gamma} R_\gamma(w). \quad (9.108)$$

Expand the retained interface functions in the retained product basis:

$$L_\alpha(z_t) = \sum_{\ell=1}^{p_{\text{ret}}} U_{\alpha, \ell} b_{t, \ell}(z_t), \quad b_t(z_t) = (b_{t, 1}(z_t), \dots, b_{t, p_{\text{ret}}}(z_t))^\top. \quad (9.109)$$

The integrated square-mass over the non-retained block is

$$G_{\gamma, \gamma'} = \int R_\gamma(w) R_{\gamma'}(w) dw. \quad (9.110)$$

Therefore

$$\int \phi_t(z_t, w)^2 dw = \sum_{\ell, \ell'} b_{t, \ell}(z_t) \left[ \sum_{\alpha, \alpha', \gamma, \gamma'} U_{\alpha, \ell} S_{\alpha, \gamma} G_{\gamma, \gamma'} S_{\alpha', \gamma'} U_{\alpha', \ell'} \right] b_{t, \ell'}(z_t). \quad (9.111)$$

The bracketed matrix, plus the retained defensive contribution, is  $Q_t$ :

$$Q_t[\ell, \ell'] = e^{-c_t} \sum_{\alpha, \alpha', \gamma, \gamma'} U_{\alpha, \ell} S_{\alpha, \gamma} G_{\gamma, \gamma'} S_{\alpha', \gamma'} U_{\alpha', \ell'} + \tau_t Q_{\lambda, t}[\ell, \ell']. \quad (9.112)$$

Thus  $Q_t$  is an exact coefficient matrix for the retained numerator under the chosen product basis if the retained block is stored in the full product basis and the non-retained square-mass contractions are exact. It becomes a storage surrogate only if a later implementation compresses  $Q_t$  into a TT, low-rank, or sparse representation; such compression must preserve the evaluator in (9.115).

Normalize by

$$P_t = \frac{Q_t}{\hat{Z}_t}, \quad \dot{P}_t = \frac{\dot{Q}_t}{\hat{Z}_t} - \frac{Q_t \hat{Z}_t}{\hat{Z}_t^2}. \quad (9.113)$$

For  $M$  next-step query points, build the basis matrix

$$B^{\text{query}}[j, :] = b_t(z_t^{\text{query}}(j))^\top, \quad B^{\text{query}} \in \mathbb{R}^{M \times p_{\text{ret}}}. \quad (9.114)$$

The retained evaluator returns

$$\hat{p}_t^{\text{ref}}[j] = B^{\text{query}}[j, :] P_t B^{\text{query}}[j, :]^\top, \quad (9.115)$$

and

$$\dot{\hat{p}}_t^{\text{ref}}[j] = B^{\text{query}}[j, :] \dot{P}_t B^{\text{query}}[j, :]^\top. \quad (9.116)$$

The next-step query rule is

$$z_t^{\text{query}}(j) = \text{the previous-state block of } Z_{\text{fit}}^{(t+1)}[j, :]. \quad (9.117)$$

This full matrix form can be large when  $m$  is large. A later implementation may store  $Q_t$  in TT or low-rank form, but then it must provide the same evaluator and derivative evaluator as (9.115)–(9.116).

## Retained-Filter Storage When The Product Basis Is Too Large

The full retained matrix has

$$\#Q_t = p_{\text{ret}}^2 = \left( \prod_{j=1}^m p_j \right)^2 = \prod_{j=1}^m p_j^2 \quad (9.118)$$

entries. If  $m = 10$  and  $p_j = 4$ , then

$$p_{\text{ret}} = 4^{10} = 1,048,576, \quad p_{\text{ret}}^2 \approx 1.1 \times 10^{12}, \quad (9.119)$$

so full matrix storage is no longer a mathematical implementation plan. The retained object should then be stored as an evaluator, not necessarily as a dense matrix.

One exact evaluator form is the marginal TT itself. Let the retained coordinates be  $z_{t,1:m}$ . Store cores  $G_{t,j}$  representing the retained numerator:

$$a_t(z_t) = G_{t,1}(z_{t,1})G_{t,2}(z_{t,2}) \cdots G_{t,m}(z_{t,m}), \quad (9.120)$$

with TT ranks  $S_0 = S_m = 1$ . Store dotted cores or a dotted evaluator

$$\dot{a}_t(z_t) = \sum_{r=1}^m G_{t,1}(z_{t,1}) \cdots \dot{G}_{t,r}(z_{t,r}) \cdots G_{t,m}(z_{t,m}). \quad (9.121)$$

The normalized retained evaluator is then

$$\hat{p}_t(z_t) = a_t(z_t)/\hat{Z}_t, \quad \dot{\hat{p}}_t(z_t) = \dot{a}_t(z_t)/\hat{Z}_t - a_t(z_t)\dot{\hat{Z}}_t/\hat{Z}_t^2. \quad (9.122)$$

A second option is a low-rank quadratic basis representation:

$$Q_t \approx U_t S_t U_t^\top, \quad U_t \in \mathbb{R}^{p_{\text{ret}} \times r_Q}, \quad r_Q \ll p_{\text{ret}}. \quad (9.123)$$

Then

$$a_t(z_t) \approx \left( U_t^\top b_t(z_t) \right)^\top S_t \left( U_t^\top b_t(z_t) \right), \quad (9.124)$$

and the derivative includes all stored factors:

$$\begin{aligned} \dot{a}_t(z_t) &\approx (\dot{U}_t^\top b_t)^\top S_t (U_t^\top b_t) + (U_t^\top b_t)^\top \dot{S}_t (U_t^\top b_t) \\ &\quad + (U_t^\top b_t)^\top S_t (\dot{U}_t^\top b_t). \end{aligned} \quad (9.125)$$

This compression is acceptable only if it is declared as part of the fixed branch and the compression residual is tested:

$$\varepsilon_Q = \frac{\|Q_t - U_t S_t U_t^\top\|_{\mathcal{V}}}{\max\{\|Q_t\|_{\mathcal{V}}, \epsilon_{\text{abs}}\}} \leq \epsilon_Q, \quad (9.126)$$

where  $\|\cdot\|_{\mathcal{V}}$  may be evaluated on a fixed validation grid or fixed quadrature rule. The derivative compression is tested analogously:

$$\varepsilon_{\dot{Q}} = \frac{\|\dot{Q}_t - \dot{Q}_t^{\text{comp}}\|_{\mathcal{V}}}{\max\{\|\dot{Q}_t\|_{\mathcal{V}}, \epsilon_{\text{abs}}\}} \leq \epsilon_{\dot{Q}}. \quad (9.127)$$

Thus the retained-filter contract is not “store a dense matrix.” It is

$$\mathcal{E}_t = \{z_t \mapsto \hat{p}_t(z_t), \ z_t \mapsto \dot{\hat{p}}_t(z_t), \ \int \hat{p}_t(z_t) dz_t = 1, \ \int \dot{\hat{p}}_t(z_t) dz_t = 0\}. \quad (9.128)$$

Any dense, TT, sparse, or low-rank storage format is admissible only if it implements  $\mathcal{E}_t$  and passes the normalization and derivative-mass checks in (9.128).

The compressed retained object has its own failure diagnostics:

| diagnostic                      | failure test                                                                       |         |
|---------------------------------|------------------------------------------------------------------------------------|---------|
| normalization drift             | $\left  \int \hat{p}_t(z_t) dz_t - 1 \right  > \epsilon_{\text{norm}}$             | (9.129) |
| derivative-mass drift           | $\left  \int \dot{\hat{p}}_t(z_t) dz_t \right  > \epsilon_{\text{norm}}$           |         |
| value compression residual      | $\epsilon_Q > \epsilon_Q$                                                          |         |
| derivative compression residual | $\epsilon_{\dot{Q}} > \epsilon_{\dot{Q}}$                                          |         |
| query mismatch                  | $\max_j  \hat{p}_t^{\text{comp}}(z_j) - \hat{p}_t^{\text{ref}}(z_j)  > \epsilon_Q$ |         |

Normalization drift means the retained density no longer integrates to one. Derivative-mass drift means the quotient derivative or compression is inconsistent. Compression residuals mean the value or derivative evaluator is not accurate enough. Query mismatch means the next-step target would use the wrong retained factor. The query mismatch test uses a fixed validation table of retained-coordinate points. If no dense reference is available, compare two independently built evaluators or a higher-rank retained representation on the same fixed table.

#### 9.14 Proposition 1: The Fixed-Branch Recursion Is A Normalized Approximate Filter

**Proposition 2.** *Fix a parameter value  $\beta$ , a time horizon  $T$ , and fixed branch objects for all times. Suppose that each approximate joint density*

$$\hat{q}_t(z_t, z_{t-1}; \beta) = e^{-c_t} \phi_t(z_t, z_{t-1}; \beta)^2 + \tau_t \lambda_t(z_t, z_{t-1}) \quad (9.130)$$

*is nonnegative and integrable, and that*

$$0 < \hat{Z}_t(\beta) = \int \hat{q}_t(z_t, z_{t-1}; \beta) dz_t dz_{t-1} < \infty. \quad (9.131)$$

*Define*

$$\hat{p}_t(z_t; \beta) = \frac{\int \hat{q}_t(z_t, z_{t-1}; \beta) dz_{t-1}}{\hat{Z}_t(\beta)}. \quad (9.132)$$

*Then  $\hat{p}_t(\cdot; \beta)$  is a normalized density in  $z_t$ . The recursion is a Bayesian filtering recursion for the declared approximate joint density  $\hat{q}_t$ , not a proof that  $\hat{q}_t$  equals the exact Bayesian joint density.*

*Proof.* Nonnegativity follows immediately:

$$e^{-c_t} \phi_t^2 \geq 0, \quad \tau_t \lambda_t \geq 0. \quad (9.133)$$

Therefore  $\hat{q}_t \geq 0$ . By assumption,  $\hat{Z}_t$  is positive and finite, so

$$\hat{p}_t^{\text{joint}}(z_t, z_{t-1}; \beta) = \frac{\hat{q}_t(z_t, z_{t-1}; \beta)}{\hat{Z}_t(\beta)} \quad (9.134)$$

is a normalized joint density:

$$\begin{aligned} \iint \hat{p}_t^{\text{joint}}(z_t, z_{t-1}; \beta) dz_t dz_{t-1} &= \frac{1}{\hat{Z}_t(\beta)} \iint \hat{q}_t(z_t, z_{t-1}; \beta) dz_t dz_{t-1} \\ &= 1. \end{aligned} \quad (9.135)$$

The retained filter is the marginal of this normalized joint density:

$$\hat{p}_t(z_t; \beta) = \int \hat{p}_t^{\text{joint}}(z_t, z_{t-1}; \beta) dz_{t-1}. \quad (9.136)$$

Integrating (9.136) over  $z_t$ ,

$$\begin{aligned} \int \hat{p}_t(z_t; \beta) dz_t &= \int \left[ \int \hat{p}_t^{\text{joint}}(z_t, z_{t-1}; \beta) dz_{t-1} \right] dz_t \\ &= \int \int \hat{p}_t^{\text{joint}}(z_t, z_{t-1}; \beta) dz_t dz_{t-1} \\ &= 1. \end{aligned} \quad (9.137)$$

Thus  $\hat{p}_t$  is normalized. The proof used only the declared approximation  $\hat{q}_t$ . It did not use the exact Bayesian density except as motivation for constructing  $\hat{q}_t$ . Hence the theorem is a normalization theorem for the approximate filtering recursion.  $\square$

### 9.15 The Story Of The Fixed-Branch Derivative

Before the formal proposition, it is useful to separate the idea from the notation. The adaptive Zhao and Cui method chooses approximation structure as it runs [4]. A rank may change, a pivot may change, or an interpolation set may change. Such choices are excellent for approximation quality, but they are discrete choices. A discrete choice can jump when  $\beta$  is perturbed:

$$B_{\text{adaptive}}(\beta_0 - h) \neq B_{\text{adaptive}}(\beta_0) \neq B_{\text{adaptive}}(\beta_0 + h). \quad (9.138)$$

Across such jumps, there is no single smooth scalar being differentiated. The fixed-branch construction deliberately chooses a narrower scalar:

$$\hat{\ell}_T(\beta; B) = \sum_{t=1}^T \log \hat{Z}_t(\beta; B), \quad (9.139)$$

where

$$B(\beta_0 - h) = B(\beta_0) = B(\beta_0 + h) = B. \quad (9.140)$$

This is why the derivative is useful but narrower. It does not claim to differentiate the adaptive rank-changing algorithm. It differentiates the declared approximate likelihood obtained after the branch has been fixed.

The forward pass at one time step is a chain of deterministic maps:

$$\beta \mapsto \tilde{q}_{t,j}(\beta) \mapsto y_j(\beta) \mapsto (A_k(\beta), N_k(\beta), d_k(\beta)) \mapsto C_{t,k}(\beta) \mapsto \hat{Z}_t(\beta) \mapsto \hat{p}_t(\cdot; \beta). \quad (9.141)$$

The derivative pass follows the same chain with the product rule, chain rule, linear-solve derivative, and quotient rule:

$$\dot{\tilde{q}}_{t,j} \mapsto \dot{y}_j \mapsto (\dot{A}_k, \dot{N}_k, \dot{d}_k) \mapsto \dot{C}_{t,k} \mapsto \dot{\hat{Z}}_t \mapsto \dot{\hat{p}}_t. \quad (9.142)$$

The only difficult-looking step is the fitted core derivative. But each core update is just a fixed linear solve:

$$N_k(\beta)g_k(\beta) = d_k(\beta). \quad (9.143)$$

Differentiating the left side gives

$$\dot{N}_k g_k + N_k \dot{g}_k = \dot{d}_k, \quad (9.144)$$



and therefore

$$N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k. \quad (9.145)$$

After this, the normalizer derivative is only

$$\widehat{Z}_t = e^{-c_t} \int \phi_t^2 dz + \tau_t, \quad \dot{\widehat{Z}}_t = e^{-c_t} \int 2\phi_t \dot{\phi}_t dz, \quad (9.146)$$

and the carried filter derivative is only

$$\widehat{p}_t = a_t / \widehat{Z}_t, \quad \dot{\widehat{p}}_t = \dot{a}_t / \widehat{Z}_t - a_t \dot{\widehat{Z}}_t / \widehat{Z}_t^2. \quad (9.147)$$

Thus Proposition 2 has a simple message. If the branch is held fixed and all objects are recomputed by the same declared equations, then the dotted recursion is the derivative of the same scalar produced by the forward recursion:

$$\text{forward scalar } \widehat{\ell}_T(\beta; B) \iff \text{dotted scalar } \partial_\beta \widehat{\ell}_T(\beta; B). \quad (9.148)$$

The proposition is not a claim that the approximate scalar equals the exact Bayesian evidence, not a claim that the adaptive algorithm is globally smooth, and not a claim that the numerical linear algebra is stable. Those are separate approximation and diagnostics questions.

## 9.16 Proposition 2: The Fixed-Branch Derivative Differentiates The Same Scalar Equation Dependency Diagram For Proposition 2

The derivative proposition follows one computational graph. In equation form, the graph is

$$\widetilde{q}_{t,j}(\beta) \xrightarrow{\sqrt{\cdot}} y_j(\beta) \xrightarrow{\text{fixed LS rows}} (A_k(\beta), d_k(\beta), N_k(\beta)) \xrightarrow{N_k g_k = d_k} C_{t,k}(\beta). \quad (9.149)$$

The differentiated graph is

$$\dot{\widetilde{q}}_{t,j} \rightarrow \dot{y}_j \rightarrow (\dot{A}_k, \dot{d}_k, \dot{N}_k) \rightarrow N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k \rightarrow \dot{C}_{t,k}. \quad (9.150)$$

This is a mathematical same-scalar derivative statement, not a numerical stability theorem. If

$$\kappa(N_k) = \|N_k\| \|N_k^{-1}\| \gg 1, \quad (9.151)$$

then the solve derivative  $N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k$  can amplify small target or roundoff errors. The branch therefore records the condition numbers already listed in (8.79), and a branch with unstable solves may differentiate the same scalar mathematically while being numerically unusable.

The fitted cores define mass contractions:

$$C_{t,k} \rightarrow M_{>k} \rightarrow R_t = \int \phi_t^2 \rightarrow \widehat{Z}_t = e^{-c_t} R_t + \tau_t, \quad (9.152)$$

and the derivative contractions are

$$(C_{t,k}, \dot{C}_{t,k}) \rightarrow \dot{M}_{>k} \rightarrow \dot{R}_t \rightarrow \dot{\widehat{Z}}_t = e^{-c_t} \dot{R}_t. \quad (9.153)$$

Mass contractions have the same caveat. Repeated products of poorly scaled core matrices can make  $R_t$  and  $\dot{R}_t$  sensitive even when the algebra is correct. This is why rescaling in (7.44), stabilization diagnostics in (8.79)–(8.81), and bridge diagnostics in (6.98) are evidence gates rather than decorative bookkeeping.

The carried filter closes the time recursion:

$$(C_{t,k}, M_{<k}) \rightarrow a_t \rightarrow \hat{p}_t = a_t / \hat{Z}_t \rightarrow \tilde{q}_{t+1,j}, \quad (9.154)$$

and

$$(\dot{C}_{t,k}, \dot{M}_{<k}) \rightarrow \dot{a}_t \rightarrow \dot{\hat{p}}_t = \dot{a}_t / \hat{Z}_t - a_t \dot{\hat{Z}}_t / \hat{Z}_t^2 \rightarrow \dot{\tilde{q}}_{t+1,j}. \quad (9.155)$$

Finally,

$$\{\hat{Z}_t, \dot{\hat{Z}}_t\}_{t=1}^T \rightarrow \hat{\ell}_T(\beta; B) = \sum_t \log \hat{Z}_t, \quad \dot{\hat{\ell}}_T = \sum_t \dot{\hat{Z}}_t / \hat{Z}_t. \quad (9.156)$$

Proposition 2 proves that the derivative path (9.150), (9.153), (9.155), and (9.156) differentiates the same scalar computed by (9.149), (9.152), (9.154), and (9.156). For the running example, the first link in (9.150) is

$$\begin{aligned} \dot{\tilde{q}}_{t,j} &= \dot{\hat{p}}_{t-1}(x_{t-1}^{(j)}) \varphi(x_t^{(j)}; \beta x_{t-1}^{(j)}, \sigma^2) \varphi(y_t; \sin x_t^{(j)}, \tau_y^2) \\ &\quad + \hat{p}_{t-1}(x_{t-1}^{(j)}) \partial_\beta \varphi(x_t^{(j)}; \beta x_{t-1}^{(j)}, \sigma^2) \varphi(y_t; \sin x_t^{(j)}, \tau_y^2), \end{aligned} \quad (9.157)$$

with the observation factor independent of  $\beta$  in this example. This concrete derivative is then passed through the same fixed sweep rows and normal equations as the generic graph above.

**Lemma 2** (Fixed ridge sweeps are differentiable maps). *Fix one branch: the coordinate domains, basis functions, fitting points, weights, ranks, sweep order, ridge parameter, defensive terms, and shifts are all held fixed. Consider one core update written as a ridge least-squares normal equation*

$$N_k(\beta) g_k(\beta) = d_k(\beta), \quad N_k(\beta) = A_k(\beta)^\top W_k A_k(\beta) + \varepsilon_k I,$$

where  $\varepsilon_k > 0$ ,  $W_k$  is fixed positive semidefinite, and  $N_k(\beta)$  is nonsingular in the neighborhood being differentiated. If the entries of  $A_k(\beta)$  and  $d_k(\beta)$  are differentiable, then

$$\dot{g}_k(\beta) = N_k(\beta)^{-1} [\dot{d}_k(\beta) - \dot{N}_k(\beta) g_k(\beta)].$$

Thus this fixed core update is differentiable. A fixed finite sweep is a finite composition of such updates, so the fitted cores produced by the fixed sweep rule are differentiable functions of  $\beta$ .

*Proof.* Because  $N_k(\beta)$  is nonsingular, the core vector is

$$g_k(\beta) = N_k(\beta)^{-1} d_k(\beta).$$

Equivalently, it satisfies  $N_k g_k = d_k$ . Differentiate this identity:

$$\dot{N}_k g_k + N_k \dot{g}_k = \dot{d}_k.$$

Solving for  $\dot{g}_k$  gives the displayed formula. The assumptions that the branch, points, weights, ranks, ridge parameter, shifts, and sweep order are fixed ensure that no discrete choice changes while differentiating. Therefore each update is an ordinary differentiable map, and composing finitely many updates gives an ordinary differentiable fitted-core map.  $\square$

**Proposition 3.** *Fix the branch  $B = (B_1, \dots, B_T)$ . Assume:*

$$\hat{\ell}_T(\beta; B) = \sum_{t=1}^T \log \hat{Z}_t(\beta; B_t), \quad (9.158)$$

each  $\widehat{Z}_t$  is positive and finite, all model density evaluations are differentiable in  $\beta$ , every least-squares update uses a fixed nonsingular ridge system, and differentiation under the finite-dimensional contractions and integrals is valid. Then the derivative pass defined by (9.59)–(9.95) computes

$$\frac{\partial}{\partial \beta} \widehat{\ell}_T(\beta; B). \quad (9.159)$$

*Proof.* The proof follows the computational graph of the fixed branch.

First, the transformed target (9.42) is a product of the previous carried filter, transition density, observation density, and frozen Jacobian. The product rule therefore gives (9.59). If  $t = 1$ , the previous filter is the initial density; for  $t > 1$ , its derivative is supplied by the previous carried-filter derivative (9.95).

Second, the fitted target values (9.43) are differentiable functions of  $\widetilde{q}_{t,j}$ . The chain rule gives (9.60). The stabilizing shift  $c_t$  is frozen, so no  $\dot{c}_t$  term appears.

Third, each core update is a fixed linear solve. Equations (9.32)–(9.36) prove that differentiating the solve yields

$$N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k. \quad (9.160)$$

Equations (9.69)–(9.80) identify  $N_k, d_k, \dot{N}_k, \dot{d}_k$  for the fixed least-squares update. Since the number and order of solves are fixed, a finite composition of differentiable solve maps gives differentiable fitted cores.

Fourth, the square integral  $R_t = \int \phi_t^2$  is computed by the mass recursion (9.81). Its derivative is the product-rule recursion (9.82). Therefore

$$\dot{R}_t = \frac{\partial}{\partial \beta} \int \phi_t(z; \beta)^2 dz. \quad (9.161)$$

With frozen  $c_t, \tau_t, \lambda_t$ , (9.86)–(9.87) give the derivative of  $\widehat{Z}_t$ .

Fifth, the carried numerator and carried filter are defined by (9.88)–(9.94). Their derivatives are (9.89) and (9.95), obtained by differentiating under the integral and by the quotient rule. This supplies the derivative object needed by the next time step.

Finally, since each time contribution is  $\log \widehat{Z}_t(\beta; B_t)$ , Warmup 1 gives

$$\frac{\partial}{\partial \beta} \log \widehat{Z}_t(\beta; B_t) = \frac{\dot{\widehat{Z}}_t(\beta; B_t)}{\widehat{Z}_t(\beta; B_t)}. \quad (9.162)$$

Summing (9.162) over  $t = 1, \dots, T$  gives (9.159).

Every object differentiated in this proof is an object in the saved branch  $B$ . The proof never differentiates a changed rank, changed grid, changed shift, changed pivot set, or changed domain. Therefore the derivative is the derivative of the same scalar  $\widehat{\ell}_T(\beta; B)$ , not the derivative of an adaptive algorithm that chooses a new  $B$  when  $\beta$  changes.  $\square$

## 9.17 What This Does Not Prove

The result above is deliberately narrow.

It does not prove exact posterior accuracy:

$$\widehat{q}_t = q_t \quad \text{is not claimed.} \quad (9.163)$$

It proves normalization and differentiation of the declared approximation.

It does not prove ranks stay small:

$$R_k \text{ moderate for all } k \text{ is a diagnostic requirement, not a theorem here.} \quad (9.164)$$

It does not differentiate adaptive decisions:

$$\frac{\partial}{\partial \beta} \{\text{chosen ranks, pivots, domains, shifts, fitting points}\} \text{ is not part of the formula.} \quad (9.165)$$

It does not prove HMC convergence:

$$\text{correct local gradient of an approximate scalar} \neq \text{posterior sampling guarantee.} \quad (9.166)$$

These caveats make the result more honest, not weaker. The fixed-branch gradient is a precise mathematical statement. Empirical accuracy, rank stability, and sampling performance must be assessed separately.

## 9.18 What Must Be Held Fixed For The Derivative

The finite-difference diagnostic needs a precise meaning for "the same branch." The fixed-field record is

$$\mathcal{H}(B) = \{\mathcal{H}_{\text{fixed}}, \mathcal{H}_{\text{diff}}, \mathcal{H}_{\text{diag}}\}. \quad (9.167)$$

The frozen fields are

$$\begin{aligned} \mathcal{H}_{\text{fixed}} = \{ & \Omega_{t,k}, \Psi_{t,k}, J_t, Z_{\text{fit}}^{(t)}, W^{(t)}, p_{t,k}, R_{t,k}, S_t, \rho_t, \\ & c_t, \tau_t, \lambda_t, \epsilon_{\text{floor}}, \epsilon_{\text{root}}, N_{\text{root}}, \text{sweep order, initialization rule}\}_{t,k}. \end{aligned} \quad (9.168)$$

The differentiable fields are

$$\begin{aligned} \mathcal{H}_{\text{diff}}(\beta) = \{ & y_j(\beta), A_k(\beta), N_k(\beta), d_k(\beta), \\ & C_{t,k}(\beta), M_{>k}(\beta), \hat{Z}_t(\beta), \\ & a_t(\beta), \hat{p}_t(\beta) \}. \end{aligned} \quad (9.169)$$

The diagnostic fields are

$$\begin{aligned} \mathcal{H}_{\text{diag}} = \{ & \text{fit residuals, condition numbers, rank vector,} \\ & \text{domain failures, KR failures, stabilization failures} \}. \end{aligned} \quad (9.170)$$

Two evaluations use the same branch if and only if their frozen fields match:

$$B(\beta_a) = B(\beta_b) \iff \mathcal{H}_{\text{fixed}}(\beta_a) = \mathcal{H}_{\text{fixed}}(\beta_b). \quad (9.171)$$

The equality is a mathematical identity of declared fields. A software implementation may use hashes or serialized labels, but the mathematical object being compared is the field tuple in (9.168).

The finite-difference branch comparison is

$$\mathcal{H}_{\text{fixed}}(\beta_0 - h) = \mathcal{H}_{\text{fixed}}(\beta_0) = \mathcal{H}_{\text{fixed}}(\beta_0 + h), \quad (9.172)$$

while the differentiable fields are recomputed:

$$\mathcal{H}_{\text{diff}}(\beta_0 - h), \quad \mathcal{H}_{\text{diff}}(\beta_0), \quad \mathcal{H}_{\text{diff}}(\beta_0 + h) \text{ may differ.} \quad (9.173)$$

If a frozen field differs, the centered difference is comparing different functions. If a differentiable field is copied rather than recomputed, the centered difference is not evaluating the fixed fitting rule.

## 9.19 Four Checklists For Reading The Derivative

This section collects four compact checklists for the fixed-branch part. They state, after the derivation, what has been established and what a later implementation must preserve.

### Squared-density block

|                |                                                                         |
|----------------|-------------------------------------------------------------------------|
| established    | $\hat{q}_t = e^{-c_t} \phi_t^2 + \tau_t \lambda_t \geq 0$ ,             |
| stored         | $\phi_t$ through $C_{t,k}$ and $\dot{\phi}_t$ through $\dot{C}_{t,k}$ , |
| integrated     | $e^{-c_t} \phi_t(z)^2 + \tau_t \lambda_t(z)$ ,                          |
| differentiated | $e^{-c_t} \phi_t(z)^2 \mapsto 2e^{-c_t} \phi_t(z) \dot{\phi}_t(z)$ ,    |
| fixed          | $c_t, \tau_t, \lambda_t$ , basis, domain, ranks, points,                |
| failure mode   | differentiating a different fitted square root than the one squared.    |

(9.174)

### Mass-contraction block

|                |                                                                  |
|----------------|------------------------------------------------------------------|
| established    | $R_t = \int \phi_t(z)^2 dz$ is computed by TT mass contractions, |
| stored         | $C_{t,k}, \dot{C}_{t,k}, B_{t,k}, M_{>k}, \dot{M}_{>k}$ ,        |
| integrated     | $\phi_t^2$ one coordinate at a time,                             |
| differentiated | each product factor once, then summed,                           |
| fixed          | $B_{t,k}$ , coordinate order, rank pattern, basis family,        |
| failure mode   | omitting a product-rule term in $\dot{M}_{>k}$ .                 |

(9.175)

### Fixed-solve block

|                |                                                                              |
|----------------|------------------------------------------------------------------------------|
| established    | $N_k g_k = d_k \mapsto N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k$ ,          |
| stored         | $A_k, N_k, d_k, g_k$ and dotted counterparts,                                |
| integrated     | nothing; this is a finite-dimensional fitting solve,                         |
| differentiated | the fixed normal equations,                                                  |
| fixed          | $W, \rho, Z_{\text{fit}}, B_{\text{val}}$ , sweep order,                     |
| failure mode   | changing the solve system or copying $g_k(\beta_0)$ at perturbed parameters. |

(9.176)

### Carried-filter block

|                |                                                                                                                   |
|----------------|-------------------------------------------------------------------------------------------------------------------|
| established    | $\hat{p}_t = a_t / \hat{Z}_t$ and $\dot{\hat{p}}_t = \dot{a}_t / \hat{Z}_t - a_t \dot{\hat{Z}}_t / \hat{Z}_t^2$ , |
| stored         | $Q_t, \dot{Q}_t, P_t, \dot{P}_t$ with shape $(p, p)$ ,                                                            |
| integrated     | $\hat{q}_t(z_t, z_{t-1})$ over $z_{t-1}$ ,                                                                        |
| differentiated | $Q_t / \hat{Z}_t$ by the quotient rule,                                                                           |
| fixed          | basis, query coordinate, branch, defensive term,                                                                  |
| failure mode   | feeding time $t + 1$ without $\dot{P}_t$ or with an undeclared query rule.                                        |

(9.177)

## A Validation And Diagnostics

The finite-difference diagnostic checks that the value path and derivative path agree for the same branch. Choose a base parameter  $\beta_0$ . Build the branch once and save the structural choices

$$B = \{\text{domains, points, basis, ranks, sweeps, shifts,} \\ \text{defensive terms, ridge parameters,} \\ \text{deterministic initialization rules}\}. \quad (\text{A.1})$$

Then evaluate the same scalar at perturbed parameters while reusing those structural choices and recomputing the parameter-dependent fitted values by the same fixed equations:

$$\widehat{\ell}_T(\beta_0 + he_i; B), \quad \widehat{\ell}_T(\beta_0 - he_i; B). \quad (\text{A.2})$$

That means the domains, points, ranks, shifts, and sweep order are unchanged, but the fitted core values

$$C_{t,k}(\beta_0 + he_i; B) \quad \text{and} \quad C_{t,k}(\beta_0 - he_i; B) \quad (\text{A.3})$$

are recomputed from the same saved fitting rule. They are not copied from  $\beta_0$ . The centered finite difference is

$$D_i(h) = \frac{\widehat{\ell}_T(\beta_0 + he_i; B) - \widehat{\ell}_T(\beta_0 - he_i; B)}{2h}. \quad (\text{A.4})$$

For the one-parameter minimal case, the same field is

$$D(h) = \frac{\widehat{\ell}_2(\beta_0 + h; B) - \widehat{\ell}_2(\beta_0 - h; B)}{2h}. \quad (\text{A.5})$$

Compare it to the analytical derivative

$$G_i = \partial_i \widehat{\ell}_T(\beta_0; B). \quad (\text{A.6})$$

The one-parameter derivative field is

$$G = \partial_\beta \widehat{\ell}_2(\beta_0; B). \quad (\text{A.7})$$

Record

$$e_i(h) = |D_i(h) - G_i|, \quad (\text{A.8})$$

and

$$e_i^{\text{rel}}(h) = \frac{|D_i(h) - G_i|}{1 + |G_i|}. \quad (\text{A.9})$$

For the one-parameter minimal case,

$$e(h) = |D(h) - G|, \quad e_{\text{rel}}(h) = \frac{|D(h) - G|}{1 + |G|}. \quad (\text{A.10})$$

Use a small schedule such as

$$h \in \{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}. \quad (\text{A.11})$$

The step-size field is

$$H_{\text{FD}} = \{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}. \quad (\text{A.12})$$

The recompute-core rule is

$$C_{t,k}(\beta_0 \pm h; B) = \text{the core produced by the fixed fitting rule at } \beta_0 \pm h, \quad (\text{A.13})$$

not the core copied from  $\beta_0$ . The report should be a declared mathematical object, not an informal output list. For the one-parameter  $T = 2$  minimal case, record

$$\mathcal{R}_{\text{FD}} = \{\widehat{\ell}_2(\beta_0; B), G, (h_i, D(h_i), e(h_i), e_{\text{rel}}(h_i))_{i=1}^4, I_-(h_i), I_+(h_i), K_-(h_i), K_+(h_i), W_i, \text{status}\}. \quad (\text{A.14})$$

Here

$$I_{\pm}(h_i) = \mathbf{1}\{B(\beta_0 \pm h_i) = B\} \quad (\text{A.15})$$

checks fixed-field equality, and

$$K_{\pm}(h_i) = \mathbf{1}\{C_k(\beta_0 \pm h_i; B) \text{ was recomputed by the fixed fitting rule}\} \quad (\text{A.16})$$

checks that perturbed cores were not copied from  $\beta_0$ . A decreasing error window is recorded by

$$W_i = \mathbf{1}\{e(h_i) > e(h_{i+1}) > e(h_{i+2})\}, \quad i = 1, 2. \quad (\text{A.17})$$

The status rule is

$$\text{status} = \begin{cases} \text{branch identity failure,} & \min_{i,\pm} I_{\pm}(h_i) = 0, \\ \text{copied-core failure,} & \min_{i,\pm} K_{\pm}(h_i) = 0, \\ \text{decreasing-window agreement,} & \max(W_1, W_2) = 1, \\ \text{inconclusive: no decreasing window,} & \text{otherwise.} \end{cases} \quad (\text{A.18})$$

The status is a derivative-consistency diagnostic for the fixed branch. It is not a posterior-accuracy theorem, a rank-stability theorem, or an HMC convergence result.

$$\mathcal{N}_{\text{FD}} = \{\text{not exact posterior accuracy,} \\ \text{not rank-stability certification,} \\ \text{not HMC convergence certification}\}. \quad (\text{A.19})$$

A healthy derivative check typically shows decreasing error over a range of  $h$ , followed by roundoff or fitting-noise limitations. If the error does not decrease on any reasonable range, the derivative path is not yet aligned with the value path.

$$\exists i \in \{1, 2\} : W_i = 1 \iff \text{there is a decreasing finite-difference error window.} \quad (\text{A.20})$$

The local branch identity condition is mandatory:

$$B(\beta_0) = B(\beta_0 + h) = B(\beta_0 - h) = B. \quad (\text{A.21})$$

If the implementation rebuilds ranks, points, shifts, or domains at the perturbed values, then the centered difference is no longer assessing the derivative of  $\widehat{\ell}_2(\beta; B)$ . It is comparing a different adaptive calculation. If the implementation copies the numerical cores from  $\beta_0$  without resolving the fixed fitting equations at the perturbed parameter, then it is also assessing the wrong object. The correct diagnostic freezes the branch choices but allows all differentiable outputs of those choices to change with  $\beta$ .

$$\begin{aligned} \min_{i,\pm} I_{\pm}(h_i) = 0 & \Rightarrow \text{the compared values use different branches,} \\ \min_{i,\pm} K_{\pm}(h_i) = 0 & \Rightarrow \text{the perturbed values copied old numerical cores,} \\ \max(W_1, W_2) = 0 & \Rightarrow \text{the diagnostic is inconclusive for derivative agreement.} \end{aligned} \quad (\text{A.22})$$

## A.1 Minimal Mathematical Example

The smallest useful mathematical example should be nonlinear but one-dimensional:

$$x_t = \beta x_{t-1} + \sigma \epsilon_t, \quad \epsilon_t \sim N(0, 1), \quad (\text{A.23})$$

$$y_t = \sin(x_t) + \eta_t, \quad \eta_t \sim N(0, r^2). \quad (\text{A.24})$$

Use  $T = 2$ . The first time step defines

$$q_1(x_1, x_0; \beta) = p_\beta(x_0) f_\beta(x_1 | x_0) g_\beta(y_1 | x_1). \quad (\text{A.25})$$

It fits  $\phi_1$ , computes

$$\widehat{Z}_1(\beta) = e^{-c_1} \int \phi_1(z_1, z_0; \beta)^2 dz_1 dz_0 + \tau_1, \quad (\text{A.26})$$

and stores

$$\widehat{p}_1(z_1; \beta) = \frac{\int [e^{-c_1} \phi_1(z_1, z_0; \beta)^2 + \tau_1 \lambda_1(z_1, z_0)] dz_0}{\widehat{Z}_1(\beta)}. \quad (\text{A.27})$$

The second time step depends on that stored object:

$$q_2(x_2, x_1; \beta) = \widehat{p}_1(x_1; \beta) f_\beta(x_2 | x_1) g_\beta(y_2 | x_2). \quad (\text{A.28})$$

The scalar checked by the example is

$$\widehat{\ell}_2(\beta) = \log \widehat{Z}_1(\beta) + \log \widehat{Z}_2(\beta). \quad (\text{A.29})$$

The numerical derivative table must contain

$$\widehat{\ell}_2(\beta_0), \quad \partial_\beta \widehat{\ell}_2(\beta_0), \quad D(h), \quad |D(h) - \partial_\beta \widehat{\ell}_2(\beta_0)|. \quad (\text{A.30})$$

It must also contain the fixed-branch record used for  $\beta_0$ ,  $\beta_0 + h$ , and  $\beta_0 - h$ . The mathematical pass condition is that those fixed-branch records are identical and the derivative error has a stable decreasing window before numerical roundoff dominates.

## A.2 Large-Scale Validation Models And Test Protocol

The derivation above explains what the squared-TT filter computes and what the fixed-branch derivative differentiates. A large-scale state-space model also needs empirical validation: the approximation must be accurate enough, the memory growth must be controlled, the runtime must be credible, and the diagnostics must reveal failure before the posterior summaries are trusted. This section specifies such a validation protocol; it does not report BayesFilter benchmark outcomes. The natural backbone for this validation is the numerical model suite used by Zhao and Cui [4]. Those examples form a diverse benchmark suite: an exact-reference linear Gaussian model, a long-horizon nonlinear stochastic-volatility model, a moderate-dimensional spatial epidemic model, and a nonlinear dynamical system where preconditioning is essential. The large-scale BayesFilter claim would come only after the additional dimension, horizon, rank, and memory stress ladders below have also been run.

The validation ladder should be read from bottom to top:

$$\begin{aligned} \mathcal{V}_0 &= \text{algebraic unit identities,} \\ \mathcal{V}_1 &= \text{small exact state-space models,} \\ \mathcal{V}_2 &= \text{Zhao-Cui model-suite reproduction,} \\ \mathcal{V}_3 &= \text{dimension, horizon, rank, and basis stress ladders,} \\ \mathcal{V}_4 &= \text{robustness and failure-mode tests,} \\ \mathcal{V}_5 &= \text{fixed-branch derivative finite-difference tests.} \end{aligned} \quad (\text{A.31})$$



The order matters. A large epidemic run cannot rescue a failed mass-matrix identity; a good finite-difference derivative cannot prove posterior accuracy; and an accurate small Kalman run does not prove scalability. The validation claim at level  $\mathcal{V}_j$  is conditional on the lower levels being healthy:

$$\text{credible at level } \mathcal{V}_j \implies \bigcap_{i < j} \{\mathcal{V}_i \text{ has no veto failure}\}. \quad (\text{A.32})$$

### A.3 What The Tests Must Measure

For a fixed time horizon  $T$ , state dimension  $m$ , parameter dimension  $d$ , basis size  $p$ , maximum rank  $R$ , and number of alternating sweeps  $S_{\text{ALS}}$ , the implementation should record the mathematical quantities that control the computation:

$$\begin{aligned} \text{target evaluations} & N_{\text{eval}} = \sum_{t=1}^T \sum_{k=1}^{D_t} n_{t,k}^{\text{fit}}, \\ \text{core storage} & B_{\text{core}} = \sum_{t=1}^T \sum_{k=1}^{D_t} r_{t,k-1} p_{t,k} r_{t,k}, \\ \text{derivative-core storage} & B_{\text{core}} = \sum_{t=1}^T \sum_{k=1}^{D_t} r_{t,k-1} p_{t,k} r_{t,k}, \\ \text{largest normal equation} & n_{\text{solve,max}} = \max_{t,k} r_{t,k-1} p_{t,k} r_{t,k}, \\ \text{mass-contraction count} & N_{\text{mass}} = \sum_{t=1}^T D_t, \\ \text{KR inversion count} & N_{\text{KR}} = \sum_{t=1}^T N_{\text{samp},t} D_t. \end{aligned} \quad (\text{A.33})$$

Here  $D_t$  is the dimension of the density fitted at time  $t$ . For the unpreconditioned adjacent-state target  $q_t(x_t, \theta, x_{t-1})$ ,

$$D_t = 2m + d,$$

while after a retained-filter specialization without static parameter integration, a smaller  $D_t$  may be used. The quantities in (A.33) are not merely software counters. They correspond to the places where the mathematical approximation can fail: too many target evaluations indicate an unaffordable design, large cores indicate rank growth, large normal equations indicate conditioning risk, and many KR inversions amplify monotonicity and quadrature errors.

The peak memory should be reported in both array units and bytes. If one floating-point scalar occupies  $b_{\text{fp}}$  bytes, then a first-order memory model is

$$M_{\text{peak}} \approx b_{\text{fp}} \max_t [B_{\text{live}}(t) + B_{\text{live}}(t) + B_{\text{env}}(t) + B_{\text{fit}}(t) + B_{\text{retained}}(t)], \quad (\text{A.34})$$

where the terms denote live forward cores, live derivative cores, sweep environments, current fitting matrices, and retained-filter representations. For a fixed-branch derivative run the derivative storage must be counted separately:

$$M_{\text{peak}}^{\text{value+grad}} - M_{\text{peak}}^{\text{value}} \approx b_{\text{fp}} \max_t [B_{\text{live}}(t) + B_{\text{env}}(t) + B_{\text{retained}}(t)]. \quad (\text{A.35})$$

This difference is the memory price of the analytical derivative.

Runtime should be split into mathematically meaningful components:

$$T_{\text{wall}} = T_{\text{target}} + T_{\text{fit}} + T_{\text{mass}} + T_{\text{KR}} + T_{\text{retained}} + T_{\text{overhead}}. \quad (\text{A.36})$$

The same total time can be interpreted very differently depending on which term dominates. If  $T_{\text{target}}$  dominates, the model likelihood or ODE solve is the bottleneck. If  $T_{\text{fit}}$  dominates, the rank/basis/sweep protocol is the bottleneck. If  $T_{\text{KR}}$  dominates, smoothing or path sampling is the bottleneck. If  $T_{\text{retained}}$  dominates, the carried-filter storage format is probably too expensive.

#### A.4 Accuracy Metrics With Exact Or Reference Targets

The first accuracy metric should be used only when an exact or high-resolution reference density is available. If  $p_{\text{ref}}$  and  $\hat{p}$  are normalized densities on the same coordinate system, use the Hellinger distance

$$D_H(p_{\text{ref}}, \hat{p}) = \left[ \frac{1}{2} \int \left( \sqrt{p_{\text{ref}}(u)} - \sqrt{\hat{p}(u)} \right)^2 du \right]^{1/2}, \quad (\text{A.37})$$

and the relative  $L^1$  error

$$E_{L^1} = \frac{\int |\hat{\pi}(u) - \pi_{\text{ref}}(u)| du}{\int |\pi_{\text{ref}}(u)| du}. \quad (\text{A.38})$$

For posterior means, variances, and trajectory summaries, use normalized moment discrepancies:

$$E_{\text{mean}} = \frac{\|\hat{\mu} - \mu_{\text{ref}}\|_2}{1 + \|\mu_{\text{ref}}\|_2}, \quad E_{\text{var}} = \frac{\|\text{diag}(\hat{\Sigma}) - \text{diag}(\Sigma_{\text{ref}})\|_2}{1 + \|\text{diag}(\Sigma_{\text{ref}})\|_2}. \quad (\text{A.39})$$

For hidden trajectories in synthetic data, report the root mean squared error of the posterior median  $\hat{x}_{t,0.5}$ :

$$\text{RMSE}_{\text{med}} = \left[ \frac{1}{T+1} \sum_{t=0}^T \|x_t^{\text{true}} - \hat{x}_{t,0.5}\|_2^2 \right]^{1/2}, \quad (\text{A.40})$$

and empirical credible-band coverage:

$$\text{Cov}_{90} = \frac{1}{T+1} \sum_{t=0}^T \mathbf{1}\{x_t^{\text{true}} \in [\hat{x}_{t,0.05}, \hat{x}_{t,0.95}]\}. \quad (\text{A.41})$$

For vector states, (A.41) should be reported coordinatewise and through a simultaneous ellipsoid or box event; coordinatewise coverage alone can hide joint undercoverage.

When importance-weighted path samples are produced from the approximate transport, report the normalized effective sample size

$$\text{ESS} = \frac{\left( \sum_{i=1}^N w_i \right)^2}{N \sum_{i=1}^N w_i^2} \in (0, 1], \quad (\text{A.42})$$

where  $w_i$  are the correction weights. Zhao and Cui report ESS quantiles over repeated experiments [4]. The same convention is useful here:

$$Q_{\text{ESS}}(q, t) = \text{Quantile}_q \left( \text{ESS}_t^{(1)}, \dots, \text{ESS}_t^{(B)} \right), \quad q \in \{0.25, 0.50, 0.75\}. \quad (\text{A.43})$$

ESS is a diagnostic for the proposal and correction weights, not an exact posterior-distance theorem. A high ESS with a biased approximate target is still possible, and a low ESS can occur even when the fitted density has useful marginals.

#### A.5 The Exact-Reference Linear Gaussian Benchmark

The first large-scale validation model should be the linear Gaussian benchmark used by Zhao and Cui [4]. The state and observation equations are

$$X_t - \mu \mathbf{1} = b(X_{t-1} - \mu \mathbf{1}) + a \epsilon_t^{(x)}, \quad \epsilon_t^{(x)} \sim N(0, I_m), \quad (\text{A.44})$$

$$Y_t = CX_t + d\epsilon_t^{(y)}, \quad \epsilon_t^{(y)} \sim N(0, I_n), \quad (\text{A.45})$$

with independent state and observation noises. The initial state prior is

$$X_0 \mid \mu \sim N(\mu \mathbf{1}, I_m), \quad (\text{A.46})$$

and the unknown parameter vector is

$$\theta = (\mu, a, b, d). \quad (\text{A.47})$$

In the paper's numerical instance  $m = n = 3$ ,  $T = 50$ ,  $\mu = 0$ , and  $a^2 + b^2 = 1$ , so the estimated parameter is effectively

$$\theta_{\text{est}} = (a, d) \in [0.4, 1]^2. \quad (\text{A.48})$$

For a fixed  $\theta$ , the Kalman recursion gives the exact conditional Gaussian filtering law

$$X_t \mid \theta, y_{1:t} \sim N(m_t(\theta), P_t(\theta)). \quad (\text{A.49})$$

The one-step predictive density is also Gaussian:

$$Y_t \mid \theta, y_{1:t-1} \sim N\left(Cm_{t|t-1}(\theta), CP_{t|t-1}(\theta)C^\top + d^2I_n\right). \quad (\text{A.50})$$

Therefore the exact parameter posterior is available up to normalization:

$$p(\theta \mid y_{1:t}) \propto p(\theta) \prod_{j=1}^t p(y_j \mid \theta, y_{1:j-1}). \quad (\text{A.51})$$

This is the most important accuracy benchmark because it tests the TT approximation against a non-Monte-Carlo reference. The required comparison is

$$\begin{aligned} \text{density accuracy} & D_H(p_{\text{Kalman}}(\theta \mid y_{1:t}), \widehat{p}_{\text{TT}}(\theta \mid y_{1:t})), \\ \text{relative density error} & E_{L^1, t} = \|\widehat{\pi}_t - \pi_{\text{Kalman}, t}\|_1 / \|\pi_{\text{Kalman}, t}\|_1, \\ \text{evidence error} & E_{Z, t} = |\log \widehat{Z}_{1:t} - \log Z_{1:t}^{\text{Kalman}}|, \\ \text{sampling quality} & Q_{\text{ESS}}(q, t), \quad q \in \{0.25, 0.50, 0.75\}. \end{aligned} \quad (\text{A.52})$$

The rank and basis ladder should include the Zhao–Cui choices:

$$\ell \in \{17, 25, 33, 41, 49\}, \quad R \in \{10, 15, 20, 25, 30\}, \quad (\text{A.53})$$

with the piecewise Lagrange basis on four subintervals and repeated ALS sweeps as in their experiments [4]. For BayesFilter, add a gradient column for the fixed-branch scalar: Here  $\beta$  denotes the scalar coordinate selected for fixed-branch differentiation and

$$\widehat{\ell}_T(\beta; B) = \sum_{t=1}^T \log \widehat{Z}_t(\beta; B). \quad (\text{A.54})$$

$$G_{\text{FB}} = \partial_\beta \widehat{\ell}_T(\beta; B), \quad D_h = \frac{\widehat{\ell}_T(\beta + h; B) - \widehat{\ell}_T(\beta - h; B)}{2h}. \quad (\text{A.55})$$

The linear Gaussian benchmark passes the derivative check only if

$$B(\beta + h) = B(\beta) = B(\beta - h) \quad \text{and} \quad |D_h - G_{\text{FB}}| \text{ decreases over a stable window of } h. \quad (\text{A.56})$$

The branch record  $B$  is the discrete transcript of every choice that can change the value path without being differentiable:

$$B = \{\text{coordinate order, domains, basis families, ranks,} \\ \text{selected fitting points, pivot choices, rank-truncation choices,} \\ \text{shifts, floors, KR inversion branches, solver fallbacks}\}. \quad (\text{A.57})$$

Branch equality means exact equality of this transcript at  $\beta - h$ ,  $\beta$ , and  $\beta + h$ . The default finite-difference ladder is

$$\mathcal{H}_{\text{FD}} = \{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}. \quad (\text{A.58})$$

Let  $e(h) = |D_h - G_{\text{FB}}|$ . A branch-stable decreasing window exists if there are at least two consecutive ladder values  $h_i > h_{i+1}$  such that

$$B(\beta - h_j) = B(\beta) = B(\beta + h_j), \quad e(h_{i+1}) < e(h_i), \quad j \in \{i, i+1\}. \quad (\text{A.59})$$

If no such window exists, the derivative validation fails and no gradient-accuracy claim is made. If roundoff later causes  $e(h)$  to stop decreasing, the reported diagnostic is the minimum  $e(h)$  over the branch-stable window before the plateau.

## A.6 The Long-Horizon Stochastic-Volatility Benchmark

The stochastic-volatility model tests whether the method remains stable over a long sequence with nonlinear observations. The model is

$$X_t = \gamma X_{t-1} + \sigma \epsilon_t^{(x)}, \quad \epsilon_t^{(x)} \sim N(0, 1), \quad (\text{A.60})$$

$$Y_t = \beta \exp(X_t/2) \epsilon_t^{(y)}, \quad \epsilon_t^{(y)} \sim N(0, 1), \quad (\text{A.61})$$

with

$$X_0 \mid \gamma, \sigma \sim N\left(0, \frac{\sigma^2}{1 - \gamma^2}\right). \quad (\text{A.62})$$

For synthetic data Zhao and Cui fix  $\sigma = 1$ , estimate  $\theta = (\gamma, \beta)$ , use  $T = 1000$ , and generate data at  $(\gamma, \beta) = (0.6, 0.4)$  [4]. The joint posterior dimension is

$$\dim(\theta, x_{0:T}) = 2 + (T + 1) = 1003 \quad \text{when } T = 1000. \quad (\text{A.63})$$

If one instead counts  $x_{1:T}$  with  $x_0$  absorbed into the initial density, the state-parameter dimension is 1002. The validation report must state which convention is used.

The transformed coordinates used for unbounded-domain preconditioning should be recorded explicitly and separately for the synthetic and real-data cases. In the synthetic fixed- $\sigma$  benchmark,  $\sigma \equiv 1$  and the transformed parameter coordinates are

$$\Theta'_{\text{syn}} = (\Phi^{-1}(\gamma), \log \beta), \quad X'_{t,\text{syn}} = X_t. \quad (\text{A.64})$$

In the real-data free- $\sigma$  benchmark following Zhao and Cui [4], the transformed coordinates are

$$\Theta'_{\text{real}} = \left(\Phi^{-1}(\gamma), \log \sigma, \frac{\log \beta}{\sigma}\right), \quad X'_{t,\text{real}} = \frac{X_t}{\sigma}. \quad (\text{A.65})$$

Here  $\Phi$  is the standard normal distribution function. Each benchmark row must state whether it uses (A.64) or (A.65).

Because an analytical marginal posterior is not available for this nonlinear long-horizon model, the main diagnostics are path-sample ESS, trajectory recovery, posterior concentration around the synthetic truth, and comparison with an independent SMC or SMC2 reference where feasible:

$$\begin{array}{ll}
\text{joint ESS} & Q_{\text{ESS}}(q, t; R), \\
\text{state-path RMSE} & \text{RMSE}_{\text{med}}(R), \\
\text{credible-band coverage} & \text{Cov}_{90}(R), \\
\text{parameter concentration} & |\hat{\gamma}_{0.5,t} - \gamma_{\text{true}}|, \quad |\hat{\beta}_{0.5,t} - \beta_{\text{true}}|, \\
\text{reference discrepancy} & D_{\text{ref}}(\hat{p}_{\text{TT}}(\gamma, \beta \mid y_{1:t}), \hat{p}_{\text{SMC}}(\gamma, \beta \mid y_{1:t})).
\end{array} \tag{A.66}$$

The rank ladder should include

$$R \in \{5, 10, 20\}, \quad \ell = 33, \quad S_{\text{ALS}} = 5, \tag{A.67}$$

matching the Zhao–Cui experimental scale [4]. For large-scale implementation work, record the memory and runtime curves

$$R \mapsto (M_{\text{peak}}(R), T_{\text{wall}}(R), Q_{\text{ESS}}(0.50, T; R), \text{RMSE}_{\text{med}}(R)). \tag{A.68}$$

A larger rank should not be treated as better if it produces ill-conditioned solves, unstable KR inversions, or a memory blow-up that would make the method unusable.

For a real-data volatility run, such as daily S&P 500 returns, the parameter priors should be recorded in the transformed and physical coordinates. A typical form, following the Zhao–Cui example, is

$$\frac{\gamma + 1}{2} \sim \text{Beta}(\omega_1, \omega_2), \quad \sigma^2 \sim \text{IG}(\zeta/2, S_\sigma/2), \quad \log \beta \mid \sigma \sim N(\beta_0, \sigma^2/q_0), \tag{A.69}$$

with

$$X_0 \mid \gamma, \sigma \sim N\left(0, \frac{\sigma^2}{1 - \gamma^2}\right). \tag{A.70}$$

Since no true hidden volatility path is observed in real data, the real-data run is not an accuracy proof. It tests stability, posterior reasonableness, rank sensitivity, and whether inferred volatility responds to observed return bursts.

## A.7 The Spatial SIR Benchmark For High-Dimensional State Filtering

The spatial susceptible-infectious-removed model tests dimension scalability. For  $J$  spatial compartments, let

$$(S_j(t), I_j(t), R_j(t)), \quad j = 1, \dots, J, \tag{A.71}$$

denote susceptible, infectious, and removed populations. The coupled dynamics are

$$\frac{dS_j}{dt} = -\kappa_j S_j I_j + \frac{1}{2} \sum_{i \in \mathcal{I}_j} (S_i - S_j), \tag{A.72}$$

$$\frac{dI_j}{dt} = \kappa_j S_j I_j - \nu_j I_j + \frac{1}{2} \sum_{i \in \mathcal{I}_j} (I_i - I_j), \tag{A.73}$$

$$\frac{dR_j}{dt} = \nu_j I_j + \frac{1}{2} \sum_{i \in \mathcal{I}_j} (R_i - R_j), \tag{A.74}$$

where  $\mathcal{I}_j$  is the neighbor set of compartment  $j$ . Because  $S_j + I_j + R_j$  determines the third compartment once two are known, the state can be represented as

$$x(t) = (S_1(t), I_1(t), \dots, S_J(t), I_J(t)) \in \mathbb{R}^{2J}. \quad (\text{A.75})$$

Let  $G(x)$  denote the right-hand side of the  $S$  and  $I$  equations. The discrete state transition is

$$X_k = X_{k-1} + \int_{(k-1)\Delta}^{k\Delta} G(x(s)) ds + \epsilon_k^{(x)}, \quad \epsilon_k^{(x)} \sim N(0, \Sigma_x), \quad (\text{A.76})$$

where the integral is evaluated numerically, for example by fourth-order Runge–Kutta. The observation model sees only infectious components:

$$Y_{k,j} = X_{k,2j} + \epsilon_{k,j}^{(y)}, \quad \epsilon_k^{(y)} \sim N(0, \Sigma_y), \quad j = 1, \dots, J. \quad (\text{A.77})$$

Zhao and Cui use  $J = 9$ , corresponding to an 18-dimensional state, partial observations,  $T = 20$ ,  $\kappa_j = 0.1$ ,  $\nu_j = 18$ , and rank ladder

$$R \in \{10, 20, 40\}. \quad (\text{A.78})$$

Their reproduction settings are

$$\Delta = 0.02, \quad \text{RK4 internal step} = 0.005, \quad \Sigma_x = I_{2J}, \quad \Sigma_y = 100I_J, \quad (\text{A.79})$$

with synthetic initial conditions

$$S_j(0) = 485 + j, \quad I_j(0) = 15 - j, \quad j = 1, \dots, 9, \quad (\text{A.80})$$

and prior

$$X_0 \sim N(\mu_0, I_{18}), \quad \mu_0 = (S_1(0), I_1(0), \dots, S_9(0), I_9(0)). \quad (\text{A.81})$$

This benchmark should report accuracy separately for observed and unobserved state components. Define

$$\mathcal{O} = \{2, 4, \dots, 2J\}, \quad \mathcal{U} = \{1, 3, \dots, 2J - 1\}. \quad (\text{A.82})$$

Then report

$$\text{RMSE}_{\mathcal{O}} = \left[ \frac{1}{(T+1)|\mathcal{O}|} \sum_{t=0}^T \sum_{j \in \mathcal{O}} (x_{t,j}^{\text{true}} - \hat{x}_{t,j,0.5})^2 \right]^{1/2}, \quad (\text{A.83})$$

and

$$\text{RMSE}_{\mathcal{U}} = \left[ \frac{1}{(T+1)|\mathcal{U}|} \sum_{t=0}^T \sum_{j \in \mathcal{U}} (x_{t,j}^{\text{true}} - \hat{x}_{t,j,0.5})^2 \right]^{1/2}. \quad (\text{A.84})$$

The unobserved susceptible components are important: if the filter only tracks the observed infectious series, it has not solved the high-dimensional latent state problem.

The SIR model is also where memory and rank growth should be most visible. A dimension ladder should vary

$$J \in \{3, 5, 9, J_{\text{large}}\}, \quad D = 2J, \quad R \in \{10, 20, 40, R_{\text{max}}\}, \quad (\text{A.85})$$

and report

$$(J, R, T) \mapsto [M_{\text{peak}}, T_{\text{wall}}, B_{\text{core}}, n_{\text{solve,max}}, Q_{\text{ESS}}(0.25, T), \text{RMSE}_{\mathcal{O}}, \text{RMSE}_{\mathcal{U}}]. \quad (\text{A.86})$$

A credible large-scale result should show not only a good final trajectory, but also explain whether the cost is driven by state dimension  $2J$ , rank  $R$ , ODE solves, retained-filter storage, or KR sampling.

## A.8 Rank-Calibrated Spatial SIR Route For Fixed-Branch Filtering

The spatial SIR benchmark exposes a distinction that is easy to miss. A fixed design is desirable for HMC because it defines one deterministic approximate likelihood, but a dense all-pairs retained-grid transition is not a viable high-dimensional fixed design. If  $d = 2J$  and each state coordinate uses  $n$  retained points, the dense retained state grid has

$$N_{\text{dense}} = n^d \quad (\text{A.87})$$

points, and a direct previous/current transition table has

$$N_{\text{pairs}} = N_{\text{dense}}^2 = n^{2d} \quad (\text{A.88})$$

entries. For the Zhao–Cui spatial SIR setting  $J = 9$ ,  $d = 18$ , and  $n = 3$ , this is

$$N_{\text{dense}} = 3^{18} = 387420489, \quad N_{\text{pairs}} = 3^{36} = 150094635296999121. \quad (\text{A.89})$$

This count is a route blocker, not a reason to abandon fixed branches. The replacement route must keep the likelihood branch fixed while applying the transition through a streamed, local, or tensor-factorized contraction whose rank and workspace are declared before the likelihood is evaluated.

For a tensor train with dimension  $d$ , per-coordinate basis or grid size  $n$ , float64 storage, and fixed rank cap  $r$ , the state-core storage forecast is

$$M_{\text{state}} \approx 8 d n r^2. \quad (\text{A.90})$$

This quantity is not enough for a production decision because the transition application creates intermediate operator ranks, autodiff tapes, contraction buffers, and orthogonalization workspace. Let  $R_{\text{eff}}$  denote the effective transition-rank multiplier and let  $\omega \geq 1$  be the workspace multiplier. The single-step transition workspace forecast is

$$M_{\text{step}} \approx 8 d n (R_{\text{eff}} r)^2 \omega. \quad (\text{A.91})$$

Given a single-step workspace cap  $M_{\text{step}}^{\text{cap}}$ , the hard rank ceiling is

$$r_{\text{max}} = \left\lfloor \sqrt{\frac{M_{\text{step}}^{\text{cap}}}{8 d n \omega R_{\text{eff}}^2}} \right\rfloor. \quad (\text{A.92})$$

The source of  $R_{\text{eff}}$  must be recorded before the run: a lower-rung measurement, an operator-core estimate, or a conservative declared bound. If  $R_{\text{eff}}$  is unknown, the row is a preflight blocker rather than an invitation to launch a large rank ladder.

The fixed-rank target is the approximate likelihood defined by a complete branch specification

$$\mathcal{B} = (r, b_{1:d}, c_{1:d}, s_{1:d}, \Pi_{\text{contract}}, \Pi_{\text{trunc}}), \quad (\text{A.93})$$

where  $r$  is the rank cap,  $b_{1:d}$  are the coordinate bases,  $c_{1:d}$  are the grid centers,  $s_{1:d}$  are the coordinate scales,  $\Pi_{\text{contract}}$  is the contraction path, and  $\Pi_{\text{trunc}}$  is the truncation policy. These choices are part of the scalar being differentiated. They may be calibrated offline, but they cannot change during an HMC likelihood call or during a same-branch derivative check.

An unscented Kalman filter may be used as a scout for centers, scales, and covariance structure. For sigma points  $\chi_i$  and weights  $(W_i^{(m)}, W_i^{(c)})$ , the prediction equations are

$$m_{t|t-1} = \sum_i W_i^{(m)} f_\theta(\chi_i), \quad (\text{A.94})$$

$$P_{t|t-1} = Q_t + \sum_i W_i^{(c)} \{f_\theta(\chi_i) - m_{t|t-1}\} \{f_\theta(\chi_i) - m_{t|t-1}\}^\top. \quad (\text{A.95})$$

Applying the observation map  $h_\theta$  to sigma points from  $(m_{t|t-1}, P_{t|t-1})$  gives

$$\hat{y}_t = \sum_i W_i^{(m)} h_\theta(\chi_i^-), \quad (\text{A.96})$$

$$S_t = R_t + \sum_i W_i^{(c)} \{h_\theta(\chi_i^-) - \hat{y}_t\} \{h_\theta(\chi_i^-) - \hat{y}_t\}^\top, \quad (\text{A.97})$$

$$C_t = \sum_i W_i^{(c)} \{\chi_i^- - m_{t|t-1}\} \{h_\theta(\chi_i^-) - \hat{y}_t\}^\top. \quad (\text{A.98})$$

The update is

$$K_t = C_t S_t^{-1}, \quad m_{t|t} = m_{t|t-1} + K_t(y_t - \hat{y}_t), \quad P_{t|t} = P_{t|t-1} - K_t S_t K_t^\top. \quad (\text{A.99})$$

These equations are useful for scale discovery, effective-dimension spectra, and local-correlation diagnostics. They are not a correctness oracle for the Zhao–Cui filter, not an exact likelihood, and not evidence of HMC readiness.

The rank-selection protocol is therefore:

1. Run the UKF scout and record  $(m_{t|t}, P_{t|t})$  only as scout evidence.
  2. Set  $(c_{1:d}, s_{1:d})$  from scout moments or a declared lower-rung rule.
  3. Estimate or declare  $R_{\text{eff}}$  and compute  $r_{\text{max}}$  by (A.92).
  4. Filter the candidate set  $\mathcal{R}_0 = \{2, 4, 8, 16, 32\}$  to  $\mathcal{R} = \{r \in \mathcal{R}_0 : r \leq r_{\text{max}}\}$ .
  5. If  $\mathcal{R} = \emptyset$ , stop with a rank-budget blocker.
  6. Run candidates in increasing  $r$  with the same branch definition except rank.
  7. Select the first rank passing the declared reference or same-route self-convergence criteria.
  8. Freeze  $\mathcal{B}$  and evaluate the HMC-facing likelihood only on that frozen branch.
- (A.100)

The ladder cannot pass self-convergence at its largest feasible rank unless a separate reviewed comparator exists. If two consecutive ranks produce nonfinite values, nonfinite gradients, memory-preflight failures, or worsening diagnostics without a model-invariant explanation, the outcome is a recorded rank, coordinate, factorization, or reference-strategy blocker. It is not an unbounded rank increase.

The dimension policy for spatial SIR is also part of the claim boundary:

| $d$ | $J$ | first allowed role                                          |         |
|-----|-----|-------------------------------------------------------------|---------|
| 18  | 9   | filtering and calibration candidate after route repair      | (A.101) |
| 50  | 25  | high-dimensional stress and rank self-convergence candidate |         |
| 100 | 50  | UKF scout and memory preflight by default.                  |         |

Thus  $d = 50$  is the first reasonable full filtering stress target under a 32 GB practical memory contract. Dimension  $d = 100$  may become a bounded filtering stress row only after the lower gates pass and the memory predictor clears; otherwise it remains scout and preflight evidence.



## A.9 P53 Route-Class Repair For Spatial SIR Transition Application

The failed P52 execution adds one more design rule. A route contract and a lower-rung streaming evaluator are useful, but they are not the same evidence as a high-dimensional scaling route. We therefore split the transition application evidence into two route classes:

$$\mathbf{C}_{\text{low}} = \text{lower-rung dense-equivalent streaming route}, \quad \mathbf{C}_{\text{scale}} = \text{local-neighborhood or TT-MPO scaling route}. \quad (\text{A.102})$$

The lower-rung route  $\mathbf{C}_{\text{low}}$  computes the same predictive quantity as the dense retained-grid update while avoiding a full in-memory transition matrix. For current points  $x_i$ , previous retained points  $z_j$ , previous log density  $\ell_{t-1}(z_j)$ , quadrature weights  $w_j$ , and Jacobian term  $a_j$ , the predictive log density is

$$\ell_t^-(x_i) = \log \sum_{j=1}^{N_{\text{prev}}} \exp \{ \log w_j + a_j + \ell_{t-1}(z_j) + \log p_\theta(x_i | z_j) \}. \quad (\text{A.103})$$

The streaming evaluator partitions current and previous indices into fixed blocks  $B_c, B_p$ , evaluates  $\log p_\theta(x_i | z_j)$  only inside one block pair at a time, and accumulates (A.103) by a stable log-sum-exp reduction. Its replay identity is

$$\mathcal{I}_{\text{low}} = (\mathbf{C}_{\text{low}}, B_c, B_p, \text{point ordering, sum order, dtype, branch id}), \quad (\text{A.104})$$

which is part of the frozen branch. The peak storage scales like

$$M_{\text{low}} = O(8 B_c B_p + 8(B_c + B_p)d), \quad (\text{A.105})$$

plus autodiff workspace. This route is appropriate for interface hardening and tiny-grid dense tie-out. It is not a scalability proof: its arithmetic still has dense-equivalent pair semantics.

The scaling route  $\mathbf{C}_{\text{scale}}$  must remove dense-equivalent semantics from the production transition application. Two admissible designs are:

$$\mathbf{C}_{\text{loc}} : \quad \log p_\theta(x | z) \approx \sum_{m=1}^M \phi_m(x_{\mathcal{N}_m}, z_{\mathcal{N}_m}; \theta), \quad (\text{A.106})$$

where  $\mathcal{N}_m$  are deterministic spatial neighborhoods, and

$$\mathbf{C}_{\text{mpo}} : \quad K_\theta(x, z) \approx \sum_{\alpha_0, \dots, \alpha_d} A_{\alpha_0, \alpha_1}^{(1)}(x_1, z_1; \theta) \cdots A_{\alpha_{d-1}, \alpha_d}^{(d)}(x_d, z_d; \theta), \quad (\text{A.107})$$

with  $\alpha_0 = \alpha_d = 1$ . In either case the route must emit a conservative route-width bound  $R_{\text{eff}}$  and memory metadata before the rank ladder is run. Its replay identity is

$$\mathcal{I}_{\text{scale}} = (\mathbf{C}_{\text{scale}}, \mathcal{N}_{1:M} \text{ or MPO cores}, R_{\text{eff}}, \Pi_{\text{contract}}, \Pi_{\text{trunc}}, \text{dtype, branch id}). \quad (\text{A.108})$$

The scaling route must be tested on lower rungs against (A.103) or the dense reference before it can unlock rank selection. The gate is

$$\begin{aligned} & \text{if only } \mathbf{C}_{\text{low}} \text{ has passed,} && \text{block rank/scaling phases;} \\ & \text{if } \mathbf{C}_{\text{scale}} \text{ has passed design, implementation,} && \text{lower-rung tie-out, replay, and metadata checks,} \\ & && \text{then rank selection may consume } R_{\text{eff}}. \end{aligned} \quad (\text{A.109})$$

Thus  $R_{\text{eff}}$  in (A.91) must come from  $\mathbf{C}_{\text{scale}}$ , not from the lower-rung dense-equivalent streaming route. Any implementation that has only  $\mathbf{C}_{\text{low}}$  evidence may claim lower-rung tie-out and interface correctness, but it must emit a scaling-route blocker rather than a spatial SIR  $d = 18$ ,  $d = 50$ , or  $d = 100$  filtering claim.

## A.10 The Predator-Prey Benchmark For Nonlinear Preconditioning

The predator-prey model tests the preconditioning story. Let  $P(t)$  be prey population and  $Q(t)$  predator population. The continuous dynamics are

$$\frac{dP}{dt} = rP \left(1 - \frac{P}{K}\right) - s \frac{PQ}{a + P}, \quad (\text{A.110})$$

$$\frac{dQ}{dt} = u \frac{PQ}{a + P} - vQ, \quad (\text{A.111})$$

with parameter vector

$$\theta = (r, K, a, s, u, v). \quad (\text{A.112})$$

Writing  $x = (P, Q)$  and  $G(x; \theta)$  for the right-hand side, the discrete state equation is

$$X_k = X_{k-1} + \int_{(k-1)\Delta}^{k\Delta} G(x(s); \theta) ds + \epsilon_k^{(x)}, \quad \epsilon_k^{(x)} \sim N(0, \Sigma_x). \quad (\text{A.113})$$

The observations are

$$Y_k = X_k + \epsilon_k^{(y)}, \quad \epsilon_k^{(y)} \sim N(0, \Sigma_y). \quad (\text{A.114})$$

Zhao and Cui use the synthetic truth

$$\theta_{\text{true}} = (0.6, 114, 25, 0.3, 0.5, 0.5), \quad x_0^{\text{true}} = (50, 5), \quad (\text{A.115})$$

with  $\Delta = 2$ , RK4 internal step 0.1, and

$$\Sigma_x = 4I_2, \quad \Sigma_y = 4I_2. \quad (\text{A.116})$$

The parameter prior is uniform on the box

$$[a, b] = [0.1, 1.1] \times [110, 130] \times [20, 30] \times [0.1, 1.1] \times [0, 1] \times [0, 1], \quad (\text{A.117})$$

and the initial-state prior is

$$X_0 \sim N((50, 5), I_2). \quad (\text{A.118})$$

The validation should compare linear and nonlinear preconditioning at the same nominal rank:

$$\mathcal{P}_{\text{lin}} = \text{linear Gaussian bridge}, \quad \mathcal{P}_{\text{nonlin}} = \text{tempered nonlinear bridge}, \quad (\text{A.119})$$

with the same basis size, rank, and sweep count. The main preconditioning effect is measured by

$$\Delta_{\text{ESS}}(t) = Q_{\text{ESS}}^{\text{nonlin}}(0.50, t) - Q_{\text{ESS}}^{\text{lin}}(0.50, t), \quad (\text{A.120})$$

and by the cost-normalized improvement

$$\Delta_{\text{cost}}(t) = \frac{Q_{\text{ESS}}^{\text{nonlin}}(0.50, t)}{T_{\text{wall}}^{\text{nonlin}}(t)} - \frac{Q_{\text{ESS}}^{\text{lin}}(0.50, t)}{T_{\text{wall}}^{\text{lin}}(t)}. \quad (\text{A.121})$$

The second quantity prevents a misleading conclusion: nonlinear preconditioning may improve ESS but still be unattractive if it makes the computation too expensive.

### A.11 BayesFilter Stress Ladders Beyond The Paper

The Zhao–Cui models should be the core suite, but a BayesFilter validation suite reproduction alone is not enough for a BayesFilter large-scale claim. A BayesFilter validation program also needs controlled stress ladders that isolate one difficulty at a time. The generic nonlinear state-space model is

$$X_t = F_\theta(X_{t-1}) + \Sigma_x^{1/2} \epsilon_t^{(x)}, \quad Y_t = H_\theta(X_t) + \Sigma_y^{1/2} \epsilon_t^{(y)}. \quad (\text{A.122})$$

Use it to define four increasing ladders:

$$\begin{aligned} \text{dimension ladder} & \quad m \in \{2, 4, 8, 16, 32, 64\}, \\ \text{horizon ladder} & \quad T \in \{10, 50, 100, 500, 1000\}, \\ \text{rank ladder} & \quad R \in \{4, 8, 16, 32, 64\}, \\ \text{basis ladder} & \quad p \in \{9, 17, 33, 65\}. \end{aligned} \quad (\text{A.123})$$

Only one ladder should be varied at a time unless the interaction is the question being tested. The reported response surface is

$$\mathcal{R}(m, T, R, p) = [E_{\text{acc}}, Q_{\text{ESS}}(0.25, T), M_{\text{peak}}, T_{\text{wall}}, n_{\text{solve, max}}, \kappa_{\text{max}}, \rho_{\text{mass}}], \quad (\text{A.124})$$

where  $E_{\text{acc}}$  is the best available accuracy metric,  $\kappa_{\text{max}}$  is the largest observed least-squares condition number, and  $\rho_{\text{mass}}$  is a mass-contraction residual such as

$$\rho_{\text{mass}} = \max_t \frac{|\hat{Z}_t^{\text{left}} - \hat{Z}_t^{\text{right}}|}{1 + |\hat{Z}_t^{\text{left}}| + |\hat{Z}_t^{\text{right}}|}. \quad (\text{A.125})$$

Here  $\hat{Z}_t^{\text{left}}$  and  $\hat{Z}_t^{\text{right}}$  are the same normalizer computed by two contraction orders when both are available. They should agree up to numerical error; disagreement suggests a contraction, mass-matrix, or conditioning defect.

### A.12 Robustness Tests And Veto Conditions

The validation report should mark a run as failed before interpreting posterior plots if any mathematical veto fires. Let

$$\mathcal{F}_t = \{Z_t, \hat{Z}_t, \hat{p}_t, C_{t,k}, \dot{C}_{t,k}, w_i, D_h, G_{\text{FB}}\} \quad (\text{A.126})$$

collect the scalar, tensor, weight, and derivative objects used at time  $t$ . The finite-value veto is

$$V_{\text{finite}} = \mathbf{1} \{ \exists t, \exists a \in \mathcal{F}_t : a \text{ contains NaN or Inf} \}. \quad (\text{A.127})$$

The normalization veto is

$$V_{\text{norm}} = \mathbf{1} \left\{ \exists t : \hat{Z}_t \leq 0 \text{ or } \left| \int \hat{p}_t(z) dz - 1 \right| > \varepsilon_{\text{norm}} \right\}. \quad (\text{A.128})$$

The least-squares conditioning veto is

$$V_{\text{cond}} = \mathbf{1} \left\{ \max_{t,k} \kappa(N_{t,k}) > \kappa_{\text{max}}^{\text{allowed}} \right\}. \quad (\text{A.129})$$

The rank-saturation veto is

$$V_{\text{rank}} = \mathbf{1} \{ \exists t : R_t = R_{\text{max}} \text{ and residual}_t > \varepsilon_{\text{fit}} \}. \quad (\text{A.130})$$

The KR monotonicity veto is

$$V_{\text{KR}} = \mathbf{1} \{ \exists j, z_{<j}, a < b : F_j(b \mid z_{<j}) < F_j(a \mid z_{<j}) \}. \quad (\text{A.131})$$

Finally, the fixed-branch derivative veto is

$$V_{\text{grad}} = \mathbf{1} \{ B(\beta + h) \neq B(\beta) \text{ or } B(\beta - h) \neq B(\beta) \text{ or no branch-stable decreasing window in } \mathcal{H}_{\text{FD}} \}. \quad (\text{A.132})$$

A run should be interpreted only if

$$V_{\text{finite}} + V_{\text{norm}} + V_{\text{cond}} + V_{\text{rank}} + V_{\text{KR}} + V_{\text{grad}}^{\text{required}} = 0, \quad (\text{A.133})$$

where  $V_{\text{grad}}^{\text{required}} = V_{\text{grad}}$  only for runs whose purpose includes validating the derivative. The tolerances in these vetoes are declared before a run. A minimal default table is

| quantity                               | default                                                      | meaning                            |
|----------------------------------------|--------------------------------------------------------------|------------------------------------|
| $\varepsilon_{\text{norm}}$            | $10^2 \epsilon_{\text{mach}} + 10 \varepsilon_{\text{quad}}$ | normalization tolerance            |
| $\kappa_{\text{max}}^{\text{allowed}}$ | $10^{10}$ in double precision                                | least-squares conditioning ceiling |
| $\varepsilon_{\text{fit}}$             | $\max(10^{-8}, 10 \text{tol}_{\text{ALS}})$                  | rank-saturation residual ceiling   |
| $\varepsilon_{\text{KR}}$              | $10^2 \epsilon_{\text{mach}} + 10 \varepsilon_{\text{quad}}$ | monotonicity/inversion tolerance   |

(A.134)

Here  $\epsilon_{\text{mach}}$  is machine precision,  $\varepsilon_{\text{quad}}$  is the declared quadrature or interpolation error proxy, and  $\text{tol}_{\text{ALS}}$  is the ALS stopping tolerance. These are implementation tolerances, not scientific outcomes; changing them after seeing posterior plots invalidates the benchmark interpretation.

### A.13 A Compact Benchmark Table Template

For each model and each configuration, the validation table should contain the same core columns, while making clear that the accuracy quantities are model-specific rather than directly comparable across rows:

| model | $D$         | $T$  | $(p, R)\mathcal{R}_{\text{src}}$ |                                      |
|-------|-------------|------|----------------------------------|--------------------------------------|
| LG    | $2m + d$    | 50   | (33, 15)                         | $(M_{\text{peak}}, T_{\text{wall}})$ |
| SV    | $T + d + 1$ | 1000 | (33, 10)                         | $(M_{\text{peak}}, T_{\text{wall}})$ |
| SIR   | $2J$        | 20   | (33, 40)                         | $(M_{\text{peak}}, T_{\text{wall}})$ |
| PP    | $2 + d$     | 20   | (33, 20)                         | $(M_{\text{peak}}, T_{\text{wall}})$ |

(A.135)

The corresponding accuracy-diagnostic table is

| model | diagnostic class                | $\mathcal{E}_{\text{model}}$                           |
|-------|---------------------------------|--------------------------------------------------------|
| LG    | exact reference                 | $D_H, E_{L^1}, E_Z$                                    |
| SV    | synthetic truth/reference       | $\text{RMSE}_{\text{med}}, \text{Cov}_{90}$            |
| SIR   | synthetic truth                 | $\text{RMSE}_{\mathcal{O}}, \text{RMSE}_{\mathcal{U}}$ |
| PP    | synthetic truth/preconditioning | $\text{RMSE}_{\text{med}}, \text{Cov}_{90}$            |

(A.136)

In a completed empirical study the measured values and uncertainty summaries are reported beside these symbols. A separate derivative table should be attached whenever the fixed-branch gradient is part of the claim:

| $h$       | $D_h$    | $G_{\text{FB}}$ | $ D_h - G_{\text{FB}} $ |
|-----------|----------|-----------------|-------------------------|
| $10^{-2}$ | measured | measured        | measured                |
| $10^{-3}$ | measured | measured        | measured                |
| $10^{-4}$ | measured | measured        | measured                |
| $10^{-5}$ | measured | measured        | measured                |

(A.137)

The same empirical table should also record  $\hat{\ell}_T(\beta + h; B)$  and  $\hat{\ell}_T(\beta - h; B)$  for each  $h$ . The derivative table is meaningful only when the branch record is identical in the three columns  $\beta - h, \beta, \beta + h$ .

## A.14 What Each Benchmark Can And Cannot Establish

The linear Gaussian model can establish exact-reference agreement for the implemented approximation at the tested dimensions and horizons, conditional on the declared seeds, tolerances, and uncertainty summaries:

$$\text{LG pass} \Rightarrow \text{exact-reference accuracy is healthy under the declared run contract.} \quad (\text{A.138})$$

It cannot establish nonlinear posterior accuracy. The stochastic-volatility model can establish long-horizon nonlinear stability and credible agreement with references or synthetic truth, again under the declared run contract:

$$\text{SV pass} \Rightarrow \text{long-horizon nonlinear filtering is plausible under the run contract.} \quad (\text{A.139})$$

It cannot by itself establish high-dimensional scalability. The spatial SIR model can establish whether the method remains useful as the latent state dimension grows, conditional on replicated runs or an explicit uncertainty justification:

$$\text{SIR pass} \Rightarrow \text{partial-observation filtering is credible at the tested } J, T, R, p. \quad (\text{A.140})$$

It cannot prove that all spatial graphs or all epidemic models have moderate TT rank. The predator-prey model can establish that nonlinear preconditioning has a measurable benefit on a nonlinear dynamical system:

$$\text{PP pass} \Rightarrow \Delta_{\text{ESS}} > 0 \text{ and } \Delta_{\text{cost}} > 0 \text{ support the preconditioning choice.} \quad (\text{A.141})$$

It cannot prove that nonlinear preconditioning is always worth its cost. One-seed demonstrations are diagnostic unless the empirical result explicitly justifies why stronger interpretation is appropriate.

The complete validation statement therefore has the form

$$\begin{aligned} &\mathcal{V}_0, \dots, \mathcal{V}_5 \text{ pass on declared models, seeds, hardware, and tolerances} \\ &\implies \text{the implementation is empirically credible for the tested large-scale SSM regimes.} \end{aligned} \quad (\text{A.142})$$

The right-hand side is deliberately empirical and conditional. It is the right conclusion for a large-scale approximation algorithm: the mathematics defines the scalar and the derivative, while the validation suite measures whether the approximation is accurate, stable, and affordable enough to use.

## B Positioning, Limits, And Relation To Zhao And Cui

The fixed-branch gradient is built from five elementary ideas:

$$\begin{aligned} \text{normalizer derivative} & \quad \partial \log Z = \dot{Z}/Z, \\ \text{squared density derivative} & \quad \partial(\phi^2) = 2\phi\dot{\phi}, \\ \text{mass contraction derivative} & \quad \text{differentiate every factor once,} \\ \text{linear solve derivative} & \quad N\dot{g} = \dot{d} - \dot{N}g, \\ \text{carried filter derivative} & \quad \partial(a/Z) = \dot{a}/Z - a\dot{Z}/Z^2. \end{aligned} \quad (\text{B.1})$$

The tensor-train notation makes these ideas look large, but it does not change their nature. Once the branch is fixed, the forward pass defines one scalar. The derivative pass differentiates that scalar. What remains for empirical work is to assess whether the approximation is accurate enough, stable enough, and useful enough for the target scientific model.

## B.1 Relation To Zhao And Cui

The main text reconstructs the displayed mathematical spine of Zhao and Cui [4] Sections 1–3 and 5. Equations (1)–(26) and (30)–(35), Algorithms 1–5, the notation formulas, the square-root error inequalities, the upper and lower conditional map formulas, the particle and smoothing weight normalizations, and the preconditioning composition formulas are represented in the notation of this note notation. Section 4 error propagation and Section 6 numerical examples are not reconstructed as the central object of this note; they are used only to explain what the method does and does not prove.

## References

- [1] Tiangang Cui and Sergey Dolgov. Deep composition of tensor-trains using squared inverse rosenblatt transports. *arXiv preprint arXiv:2007.06968*, 2021.
- [2] Ivan V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33(5): 2295–2317, 2011. doi: 10.1137/090752286.
- [3] Murray Rosenblatt. Remarks on a multivariate transformation. *The Annals of Mathematical Statistics*, 23(3):470–472, 1952.
- [4] Yiran Zhao and Tiangang Cui. Tensor-train methods for sequential state and parameter learning in state-space models. *Journal of Machine Learning Research*, 25:1–51, 2024.